Índice

1. Portada
2. Resumen ejecutivo
    - 2.1. Resum en Valencià
    - 2.2. Resumen en Castellano
    - 2.3. Executive summary in English
3. Introducción y contexto del proyecto
4. Análisis y toma de decisiones
    - 4.1. Necesidad de digitalizar la gestión del gimnasio
    - 4.2. Decisión de separar zona pública, usuario y administración
    - 4.3. Decisión de usar QR para el control de accesos
    - 4.4. Decisión de implementar reservas de clases
    - 4.5. Decisión de incluir caja
    - 4.6. Decisión de simular pagos con Stripe
    - 4.7. Conclusión del análisis inicial
5. Objetivos del proyecto
    - 5.1. Objetivo general
    - 5.2. Objetivos de diseño y experiencia de usuario
    - 5.3. Objetivos técnicos
    - 5.4. Alcance actual del proyecto
    - 5.5. Funcionalidades pendientes o en desarrollo
6. Análisis de requisitos
    - 6.1. Requisitos funcionales
    - 6.2. Requisitos no funcionales
    - 6.3. Roles de usuario
    - 6.4. Casos de uso principales
7. Diseño de interfaces
    - 7.1. Investigación inicial y referencias
    - 7.2. Arquitectura de información
    - 7.3. Mapa de navegación
    - 7.4. Wireframes
    - 7.5. Mockups de alta fidelidad
    - 7.6. Guía de estilo web
    - 7.7. Accesibilidad y responsive design
8. Arquitectura del sistema
    - 8.1. Tecnologías utilizadas
    - 8.2. Justificación de las tecnologías
    - 8.3. Arquitectura general de la aplicación
    - 8.4. Estructura de carpetas y organización del código
    - 8.5. Flujo general de funcionamiento
9. Diseño de base de datos
    - 6.1. Modelo Entidad–Relación

    - 6.2. Modelo lógico relacional
    - 6.3. Descripción de tablas principales
    - 6.4. Relaciones y cardinalidades
    - 6.5. Integridad, normalización e índices
10. Planificación temporal
    - 7.1. Fases del proyecto
    - 7.2. Diagrama de Gantt
11. Desarrollo frontend
    - 9.1. Estructura HTML semántica
    - 9.2. Estilos CSS y organización visual
    - 9.3. Interactividad con JavaScript
    - 9.4. Diseño responsive
    - 9.5. Accesibilidad aplicada
12. Desarrollo backend
    - 10.1. Rutas principales
    - 10.2. Controladores
    - 10.3. Validación de formularios
    - 10.4. Gestión de sesiones y autenticación
    - 10.5. Seguridad básica
    - 10.6. Panel de administración
11. Integración con base de datos
    - 11.1. CRUDs implementados
    - 11.2. Consultas y relaciones
    - 11.3. Control de errores
    - 11.4. Datos iniciales y seeders
    - 11.5. Relación entre funcionalidades y tablas
12. Pruebas
    - 12.1. Pruebas de interfaz
    - 12.2. Pruebas funcionales
    - 12.3. Pruebas de usabilidad
    - 12.4. Pruebas de seguridad
    - 12.5. Errores detectados y solucionados
13. Deploy / publicación
    - 13.1. Plataforma de publicación
    - 13.2. Configuración del entorno
    - 13.3. URL pública
    - 13.4. Instrucciones de uso
    - 13.5. Credenciales de prueba
14. Reflexión final
    - 14.1. Dificultades encontradas
    - 14.2. Soluciones aplicadas
    - 14.3. Mejoras futuras
15. Anexos
    - 15.1. Fragmentos de código relevantes

- 15.2. Capturas de pantalla
- 15.3. Diagramas ampliados
- 15.4. Exportación del Gantt
- 15.5. Manual básico de usuario
- 15.6. Manual básico de administrador

Índice de imágenes

- **Imagen 1:** *Página principal de Iron Pulse*
- **Imagen 2:** Apartado de planes
- **Imagen 3:** QR de acceso del usuario y panel personal de usuario
- **Imagen 4:** Panel de control del administrador
- **Imagen 5:** *Calendario para reserva de clases*
- **Imagen 6:** *Dashboard de rutinas*
- **Imagen 7:** *Formulario de registro*
- **Imagen 8:** *Formulario de inicio de sesión*
- **Imagen 9:** *Pantalla de pago en Iron Pulse*
- **Imagen 10:** *Pantalla de pago en STRIPE*
- **Imagen 11:** *Pantalla de reserva de clase*
- **Imagen 12:** *Gestión de caja con caja abierta*
- **Imagen 13:** *carpetas principales del proyecto*
- **Imagen 14:** *Modelos del proyecto*
- **Imagen 15:** *Modelos del proyecto*
- **Imagen 16:** *Modelos del proyecto*
- **Imagen 17:** *Archivo web.php*
- **Imagen 18:** *Requerimientos implícitos en el composer.json*
- **Imagen 19:** *Diagrama Entidad-Relación de la base de datos del proyecto*
- **Imagen 20:** *Diagrama de flujo de procesos*
- **Imagen 21:** *Diagrama de gantt*
- **Imagen 22:** *Moodboard inicial de Iron Pulse*
- **Imagen 23:** *Mapa de navegación de Iron Pulse*
- **Imagen 24:** *Boceto del HOME, hecho en paint*
- **Imagen 25:** *Boceto del Inicio de Sesión, hecho en paint*
- **Imagen 26:** *Boceto de los Planes, hecho en paint*
- **Imagen 27:** *Boceto del panel de administrador, hecho en paint*
- **Imagen 28:** *Boceto del Panel de usuario con QR, hecho en paint*
- **Imagen 29:** *Mockup del HOME*
- **Imagen 30:** *Mockup del apartado de PLANES*
- **Imagen 31:** *Mockup del apartado de Iniciar Sesión*
- **Imagen 32:** *Mockup del apartado de Clases*
- **Imagen 33:** *Comparativa Home planeado a Home Final*
- **Imagen 34:** *Guía de estilo*

- **Imagen 35:** *Grid utilizado en el proyecto*
- **Imagen 36:** *Estructura base del layout*
- **Imagen 37:** *Captura de una parte del CSS*
- **Imagen 38:** *Ejemplo de importación de las fuentes del proyecto*
- **Imagen 39:** *Demostración de la utilización del grid en el proyecto*
- **Imagen 40:** *captura del apartado en el archivo app.js*
- **Imagen 41:** *captura del apartado en el archivo bootstrap.js*
- **Imagen 42:** *Imagen de parte del calendario*
- **Imagen 43:** *Importación de stripe en el proyecto*
- **Imagen 44:** *Demostración de diseño responsive en el proyecto*
- **Imagen 45:** *Primera parte de rutas principales*
- **Imagen 46:** Segunda parte de rutas principales
- **Imagen 47:** Rutas de autentificación
- **Imagen 48:** *Métodos del EventController*
- **Imagen 49:** Métodos del AdminController
- **Imagen 50:** *Otros métodos del AdminController*
- **Imagen 51:** *Métodos del CashController*
- **Imagen 52:** *Métodos del StripeController*
- **Imagen 53:** *Parte de código de RegisterUserController*
- **Imagen 54:** *Parte de código de LoginRequest*
- **Imagen 55:** *Panel de administración*
- **Imagen 56:** *Escáner QR y accesos*
- **Imagen 57:** *Gestión de caja*
- **Imagen 58:** *Listado de controladores más representativos*
- **Imagen 59:** *Método store del controlador de rutina*
- **Imagen 60.** *Método update del controlador de rutina*
- **Imagen 61:** *Configuración de la conexión a la base de datos en el archivo de entorno*
- **Imagen 62:** *Procesamiento y unificación de colecciones de datos mediante métodos de Eloquent*
- **Imagen 63**. *Consulta de Eloquent utilizando filtros de columna y condiciones sobre relaciones Many-to-Many*
- **Imagen 64**: *Implementación de reglas de validación para asegurar la integridad de los registros*
- **Imagen 65:** *Vista desde el móvil del inicio de sesión*
- **Imagen 66:** *Vista desde el móvil de la página principal*
- **Imagen 67**: *Vista desde el móvil del panel del usuario*
- **Imagen 68**: *Vista desde el móvil del panel de control del administrador*
- **Imagen 69**: *Vista desde la tablet del inicio de sesión*
- **Imagen 70**: *Vista desde la tablet de la página principal*
- **Imagen 71**: *Vista desde la tablet del panel del usuario*
- **Imagen 72**: *Vista desde la tablet del panel de control del administrador*
- **Imagen 73**: *Vista desde el ordenador del inicio de sesión*
- **Imagen 74**: *Vista desde el ordenador de la página principal*

- **Imagen 75**: *Vista desde el ordenador del panel del usuario*
- **Imagen 76**: *Vista desde el ordenador del panel de control del administrador*
- **Imagen 77**: *Redirección a la vista del login si no tienes sesion iniciada*
- **Imagen 78**: *Error al intentar acceder a una ruta que solo el administrador tiene permiso.*
- **Imagen 79**: *Mensajes de error al intentar registrarte omitiendo campos obligatorios.*
- **Imagen 80**: *Conexión segura y cifrada (https) al realizar un pago con Stripe.*

2. Resumen ejecutivo

2.1. Resum executiu – Valencià

Iron Pulse és una plataforma web concebuda per a la gestió integral de gimnasos. El sistema centralitza en una única aplicació processos habituals com l'administració de caixa, el control de mensualitats, la gestió de socis, la consulta de plans, l'organització de classes i rutines, la reserva d'activitats i la regulació d'accessos mitjançant codis QR associats a cada usuari.

El projecte està orientat principalment a dos perfils d'usuari: els clients del gimnàs i el personal administrador. Els usuaris poden registrar-se, iniciar sessió, consultar informació del centre, revisar els plans disponibles, accedir a rutines d'entrenament, reservar classes en el calendari i utilitzar el seu codi QR per a controlar l'entrada i l'eixida del gimnàs. Els administradors, per la seua banda, disposen d'eines per a gestionar usuaris, membresies, activitats, rutines, reserves, accessos i operacions econòmiques vinculades a la caixa.

Una de les característiques principals d'Iron Pulse és la integració de diferents àrees de gestió dins d'un mateix sistema. La caixa es troba connectada amb la base de dades i permet registrar operacions de manera estructurada. A més, la plataforma incorpora un sistema de pagament simulat mitjançant Stripe, útil per a demostrar el flux de contractació d'un pla o abonament, deixant preparada la base per a una integració real en futures versions.

El disseny de l'aplicació s'ha orientat a optimitzar l'experiència tant dels propietaris com dels usuaris, prioritzant l'eficiència operativa, la usabilitat i la coherència visual. El propòsit principal és reduir la dependència de processos manuals, facilitar el seguiment dels abonaments i permetre un control més precís de l'activitat diària del gimnàs, incloent-hi les entrades i eixides mitjançant QR, les reserves de classes i la gestió econòmica bàsica.

2.2. Resumen ejecutivo – Castellano

Iron Pulse es una plataforma web concebida para la gestión integral de gimnasios. El sistema centraliza en una única aplicación procesos habituales como la administración de caja, el control de mensualidades, la gestión de socios, la consulta de planes, la organización de clases y rutinas, la reserva de actividades y la regulación de accesos mediante códigos QR asociados a cada usuario.

El proyecto está orientado principalmente a dos perfiles de usuario: los clientes del gimnasio y el personal administrador. Los usuarios pueden registrarse, iniciar sesión, consultar información del centro, revisar los planes disponibles, acceder a rutinas de entrenamiento, reservar clases en el calendario y utilizar su código QR para controlar la entrada y salida del gimnasio. Los administradores, por su parte, disponen de herramientas para gestionar usuarios, membresías, actividades, rutinas, reservas, accesos y operaciones económicas vinculadas a la caja.

Una de las características principales de Iron Pulse es la integración de diferentes áreas de gestión dentro de un mismo sistema. La caja se encuentra conectada con la base de datos y permite registrar operaciones de forma estructurada. Además, la plataforma incorpora un sistema de pago simulado mediante Stripe, útil para demostrar el flujo de contratación de un plan o abono, dejando preparada la base para una integración real en futuras versiones.

El diseño de la aplicación se ha orientado a optimizar la experiencia tanto de los propietarios como de los usuarios, priorizando la eficiencia operativa, la usabilidad y la coherencia visual. El propósito principal es reducir la dependencia de procesos manuales, facilitar el seguimiento de los abonos y permitir un control más preciso de la actividad diaria del gimnasio, incluyendo entradas y salidas mediante QR, reservas de clases y gestión económica básica.

2.3. Executive summary – English

Iron Pulse is a web platform designed for the comprehensive management of gyms. The system centralizes common processes into a single application, including cash management, membership fee control, member administration, plan consultation, class and workout routine organization, activity booking and access control through QR codes associated with each user.

The project is mainly aimed at two user profiles: gym members and administrative staff. Users can register, log in, view information about the gym, check available plans, access workout routines, book classes through the calendar and use their QR code to control entry and exit from the facility. Administrators, on the other hand, have tools to manage users, memberships, activities, routines, bookings, access records and financial operations linked to the cash register.

One of the main features of Iron Pulse is the integration of different management areas within the same system. The cash register module is connected to the database and allows operations to be recorded in a structured way. In addition, the platform includes a simulated payment system using Stripe, which is useful for demonstrating the process of purchasing a plan or membership, while leaving the foundation ready for a real payment integration in future versions.

The application has been designed to improve the experience of both gym owners and users, prioritizing operational efficiency, usability and visual consistency. Its main purpose is to reduce dependence on manual processes, simplify membership tracking and allow more accurate control of the gym’s daily activity, including QR-based entry and exit, class bookings and basic financial management.

3. Introducción y contexto del proyecto

Iron Pulse nace como una propuesta de aplicación web para mejorar la gestión interna de un gimnasio y digitalizar procesos que habitualmente se realizan de forma manual o mediante herramientas separadas.

En un gimnasio real existen diferentes tareas que deben coordinarse diariamente: alta de socios, control de planes, reservas de clases, registro de accesos, gestión de rutinas, cobros y cierre de caja. Cuando estas tareas se realizan de forma aislada, pueden aparecer errores de organización, falta de trazabilidad, duplicidad de datos o dificultad para conocer el estado real del centro.

A partir de esta problemática se plantea el desarrollo de una plataforma web centralizada, capaz de ofrecer una zona pública para visitantes, una zona privada para socios y un panel de administración para el personal del gimnasio.

4. Análisis y toma de decisiones

4.1. Necesidad de digitalizar la gestión del gimnasio

Durante el análisis inicial se detectó que un gimnasio necesita controlar diferentes procesos diarios: altas de usuarios, planes activos, reservas de clases, rutinas, pagos, accesos y caja. Si estos procesos se gestionan de forma manual o con herramientas separadas, resulta más difícil mantener la información actualizada y controlar la actividad real del centro.

Por este motivo se decidió crear una plataforma web centralizada, donde cada operación importante quedara registrada en la base de datos y pudiera consultarse desde un panel de administración.

4.2. Decisión de separar zona pública, usuario y administración

Se decidió dividir la aplicación en tres zonas principales:

Zona	Motivo
Zona pública	Permite presentar el gimnasio, sus planes, clases e información general sin necesidad de iniciar sesión.

Zona de usuario	Permite al socio gestionar su perfil, consultar planes, rutinas, clases, reservas y QR.
Zona de administración	Permite al personal del gimnasio gestionar usuarios, planes, clases, rutinas, caja y accesos.

Esta separación permite que cada tipo de usuario vea únicamente las funcionalidades que necesita, mejorando la claridad de navegación y reduciendo errores de uso.

4.3. Decisión de utilizar QR para el control de accesos

Una de las necesidades detectadas fue controlar la entrada y salida de los socios del gimnasio. Para resolverlo se analizaron diferentes opciones: registro manual por parte del personal, tarjetas físicas, tornos electrónicos o códigos QR.

Finalmente se eligió el sistema de QR porque ofrece varias ventajas para el alcance del proyecto:

Opción	Ventajas	Inconvenientes
Registro manual	Fácil de entender	Poco eficiente, depende siempre del personal y puede generar errores.
Tarjeta física	Uso sencillo para el socio	Requiere tarjetas, lector físico y mayor coste.
Torno electrónico	Solución profesional	Requiere hardware externo y queda fuera del alcance del proyecto.
Código QR	Bajo coste, fácil de generar, asociado a cada usuario y usable desde móvil	Requiere cámara o escáner desde el panel de administración.

Por tanto, se decidió generar un QR asociado al usuario y procesarlo desde el panel de administración. Cuando el QR se escanea, el sistema identifica al usuario y registra una entrada o una salida en función de su estado actual.

Esta decisión permite controlar cuántos usuarios han accedido al gimnasio, quién se encuentra dentro y mantener un historial de accesos sin necesidad de hardware físico complejo.

4.4 Decisión de implementar reservas de clases

Otra necesidad detectada fue organizar las clases dirigidas y evitar problemas de aforo. En un gimnasio, actividades como spinning, yoga, pilates o boxeo suelen tener plazas limitadas, por lo que es necesario saber qué usuarios están apuntados.

Se decidió implementar un calendario de clases con reserva porque permite:

- Mostrar al usuario las clases disponibles.
- Controlar la capacidad máxima de cada actividad.
- Evitar reservas duplicadas.
- Impedir reservas en clases ya finalizadas.
- Permitir al usuario consultar sus clases reservadas.

Esta funcionalidad mejora la organización interna y ofrece al usuario una experiencia más cómoda.

4.5. Decisión de incluir gestión de caja

Además de gestionar socios y clases, se consideró necesario incorporar una caja porque los gimnasios trabajan con pagos de mensualidades, altas, renovaciones y posibles movimientos diarios.

Se decidió crear un módulo de caja para:

- Abrir caja diaria.
- Registrar ingresos y gastos.
- Asociar pagos a usuarios.
- Diferenciar métodos de pago.
- Cerrar caja.
- Consultar informes.

Esta decisión permite que las operaciones económicas no queden aisladas, sino integradas con la base de datos y con la gestión de usuarios y planes.

4.6. Decisión de simular pagos con Stripe

Para representar la contratación digital de planes se decidió integrar Stripe en modo de pruebas. El objetivo no era gestionar dinero real en producción, sino demostrar un flujo profesional de pago online.

Stripe se eligió porque permite simular pagos con tarjeta, crear sesiones de checkout y preparar el proyecto para una futura integración real.

En la versión actual, el pago se considera una simulación académica

4.7. Conclusión del análisis inicial

Las decisiones tomadas responden a necesidades concretas del proyecto. El QR surge de la necesidad de controlar accesos de forma sencilla y económica; el calendario responde a la necesidad de gestionar clases con aforo; la caja permite mantener trazabilidad económica; y Stripe permite representar un proceso realista de contratación de planes.

A partir de este análisis se definen los objetivos, requisitos y funcionalidades principales de Iron Pulse.

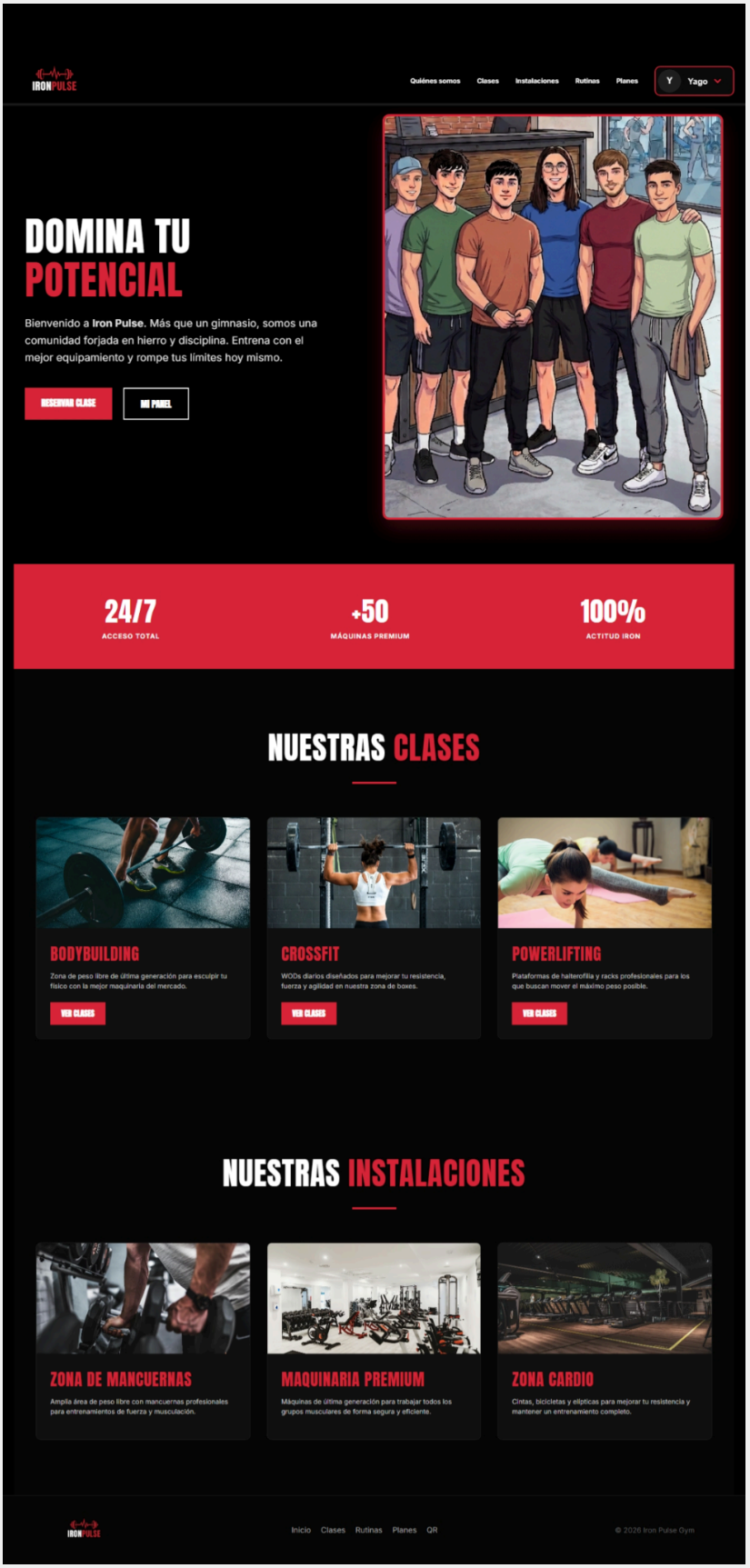


Imagen 1: *Página principal de Iron Pulse*

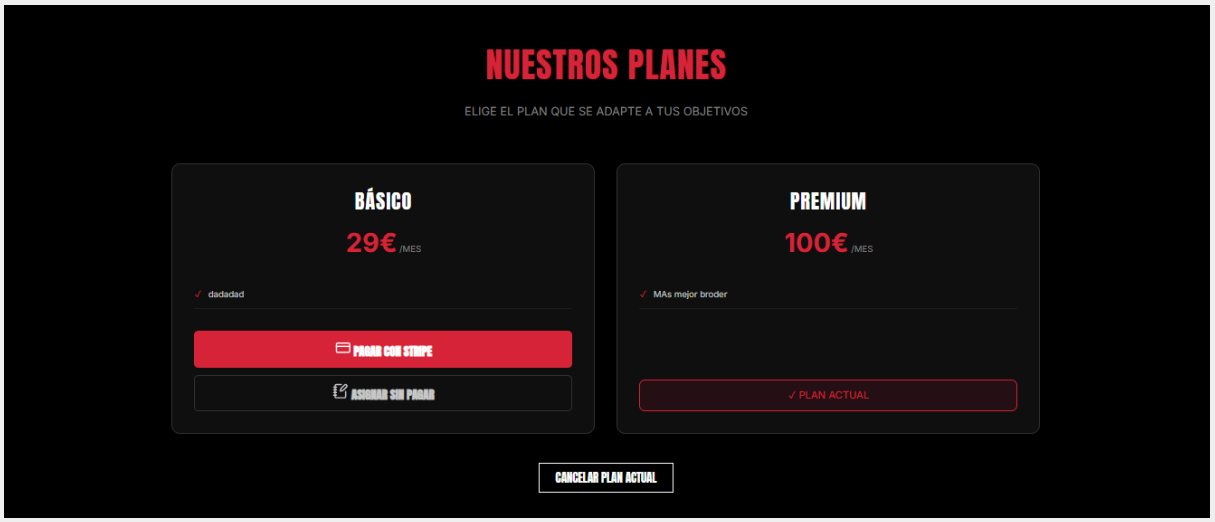


Imagen 2: *Apartado de planes*

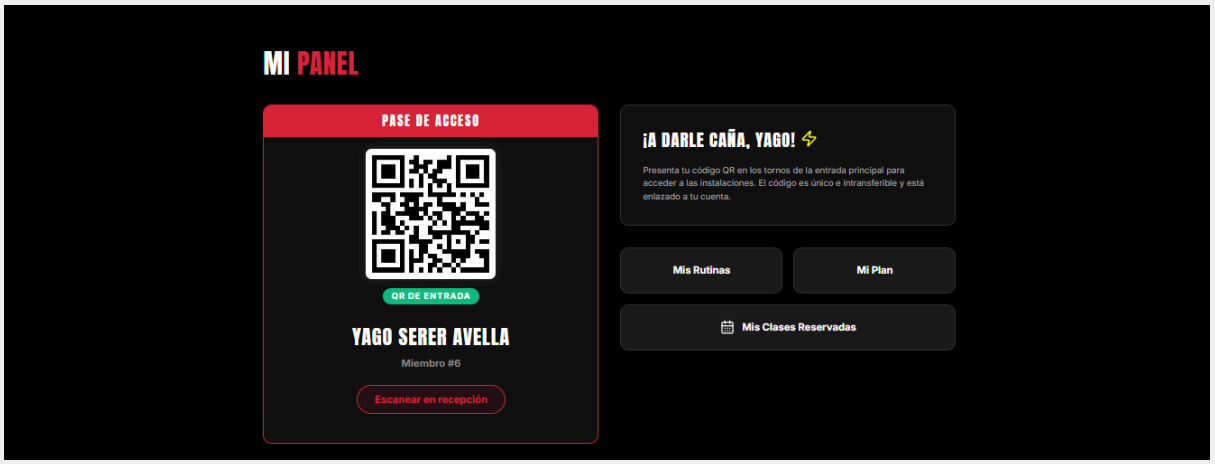


Imagen 3: *QR de acceso del usuario y panel personal de usuario*

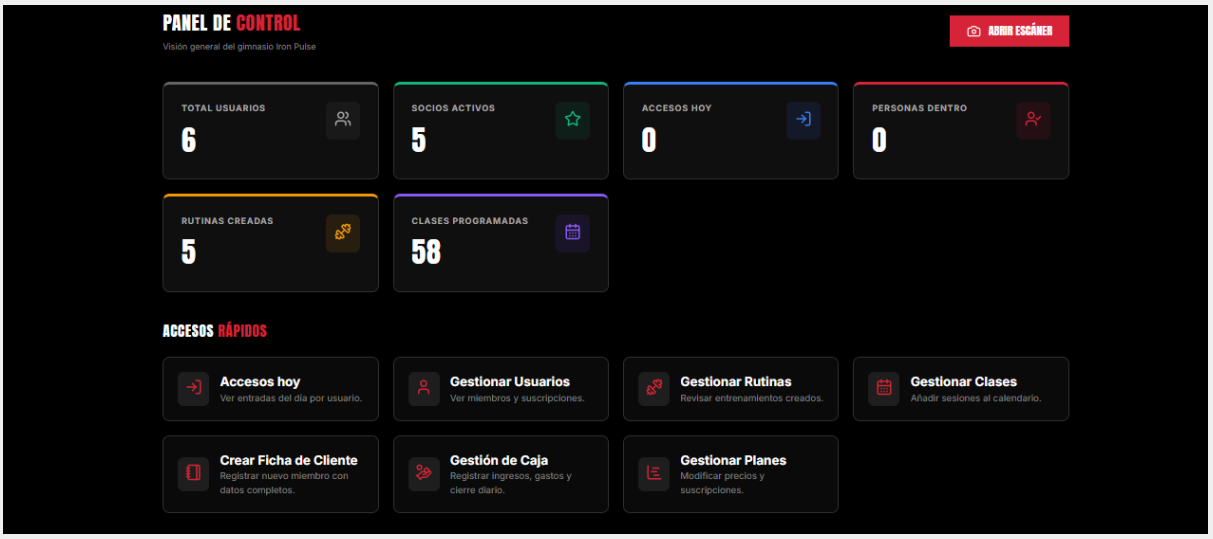


Imagen 4: *Panel de control del administrador*

5. Objetivos del proyecto

5.1. Objetivo general

El objetivo general de Iron Pulse es desarrollar una plataforma web que centralice la gestión de un gimnasio, integrando en un mismo sistema la información pública del centro, la gestión de socios, los planes de membresía, las rutinas, las reservas de clases, el control de accesos y la gestión de caja.

La finalidad del proyecto es reducir la dependencia de procesos manuales, mejorar la organización interna del gimnasio y ofrecer al usuario una experiencia digital clara, moderna y accesible.

5.2. Objetivos derivados del análisis

A partir del análisis inicial se establecen los siguientes objetivos:

Código	Objetivo	Justificación
OA-01	Centralizar la gestión del gimnasio en una única plataforma web.	Se detectó que gestionar usuarios, pagos, clases y accesos por separado puede generar errores y duplicidad de información.
OA-02	Diferenciar los flujos de visitante, usuario registrado y administrador.	Cada perfil necesita acceder a funciones distintas, por lo que era necesario separar zonas y permisos.
OA-03	Digitalizar el control de accesos mediante QR.	Se buscaba una solución económica y viable para registrar entradas y salidas sin depender de hardware físico complejo.
OA-04	Incorporar un calendario de clases con reservas.	Las clases tienen horarios y aforo limitado, por lo que era necesario controlar inscripciones y disponibilidad.

OA-05	Registrar operaciones económicas mediante un módulo de caja.	El gimnasio necesita controlar cobros, ingresos, gastos y cierres diarios.
OA-06	Simular la contratación de planes mediante Stripe.	Se quería representar un proceso de pago online realista sin trabajar con pagos reales en producción.
OA-07	Diseñar una interfaz coherente con el sector fitness.	La aplicación debía transmitir energía, fuerza y profesionalidad, manteniendo una navegación clara.
OA-08	Crear una estructura técnica mantenible y ampliable.	El proyecto debía poder crecer en el futuro añadiendo nuevas funcionalidades sin rehacer la aplicación.

5.3. Objetivos de diseño y experiencia de usuario

Código	Objetivo
OD-01	Crear una identidad visual propia para Iron Pulse antes de desarrollar las pantallas finales.
OD-02	Definir una guía de estilo con colores, tipografías, botones, tarjetas, iconos y sistema de grid.
OD-03	Diseñar una interfaz moderna y deportiva, coherente con el sector fitness.

OD-04	Mantener una navegación clara entre zona pública, zona de usuario y zona de administración.
OD-05	Diseñar pantallas responsive para ordenador, tablet y móvil.
OD-06	Aplicar criterios básicos de accesibilidad, como contraste, legibilidad, etiquetas de formularios y jerarquía visual.

5.4. Objetivos técnicos

Código	Objetivo
OT-01	Desarrollar la aplicación con Laravel siguiendo el patrón Modelo-Vista-Controlador.
OT-02	Utilizar Blade para crear vistas reutilizables y mantener una estructura ordenada.
OT-03	Diseñar una base de datos relacional para almacenar usuarios, planes, rutinas, clases, reservas, accesos y caja.
OT-04	Utilizar migraciones, modelos y relaciones Eloquent para gestionar la base de datos.
OT-05	Implementar autenticación, sesiones y protección de rutas privadas.
OT-06	Validar los formularios para evitar datos incorrectos o incompletos.

OT-07	Integrar librerías externas para calendario, alertas, QR y pagos simulados.
OT-08	Preparar la aplicación para un posible despliegue real y futuras ampliaciones.

5.5. Alcance del proyecto

El proyecto incluye una aplicación web funcional con zona pública, zona privada de usuario y panel de administración.

El alcance actual incluye:

- Página pública del gimnasio.
- Registro e inicio de sesión.
- Gestión de perfil.
- Planes de membresía.
- Pago simulado con Stripe.
- Rutinas.
- Calendario de clases.
- Reservas.
- Código QR de acceso.
- Control de entrada y salida.
- Gestión de usuarios.
- Gestión de planes.
- Gestión de clases.
- Gestión de caja.
- Estadísticas básicas del panel de administración.

Quedan fuera del alcance actual:

- Pagos reales en producción.
- Facturación legal automatizada.
- Integración con tornos físicos.
- Aplicación móvil nativa.
- Sistema avanzado de analítica.
- Gestión completa de instructores como rol independiente.

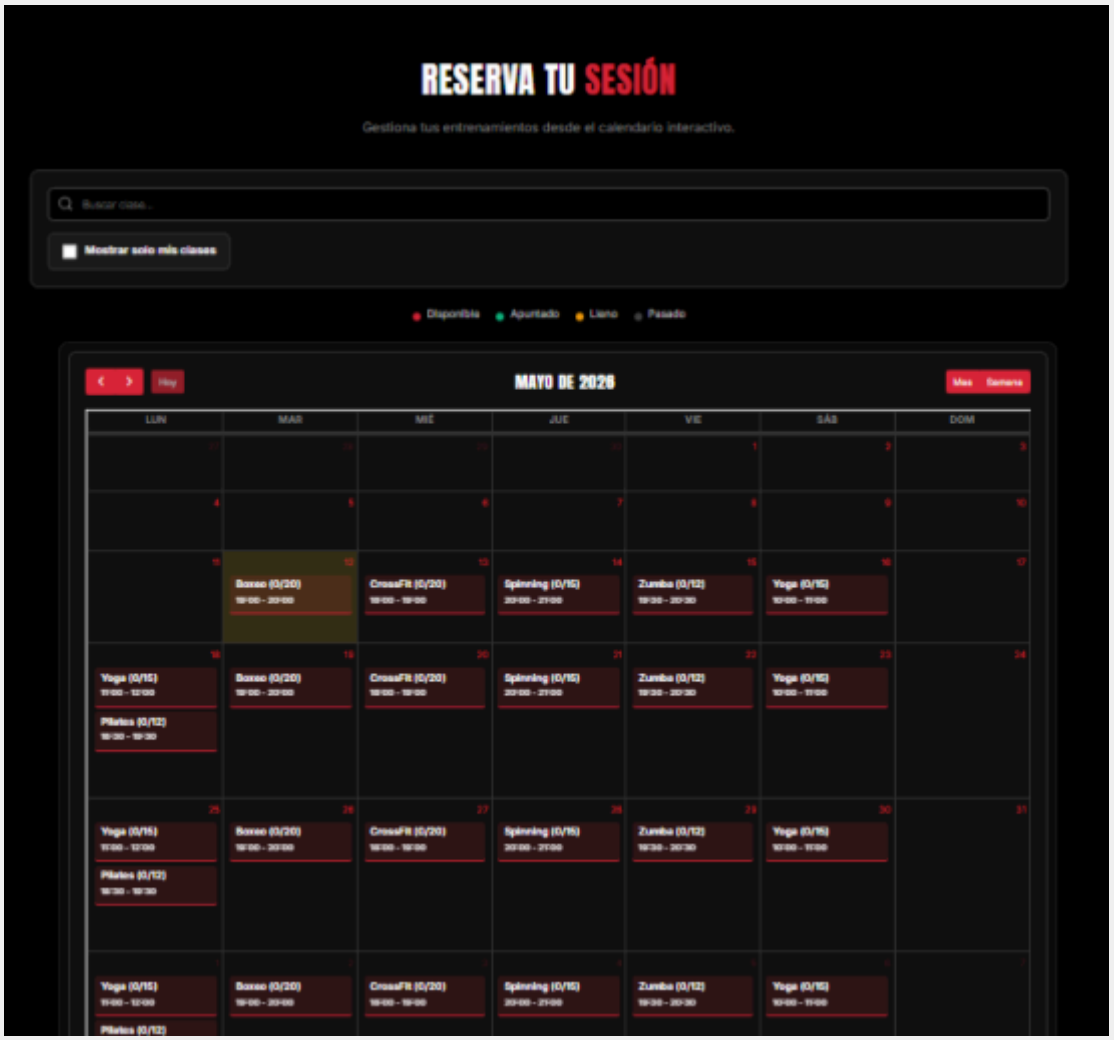


Imagen 5: *Calendario para reserva de clases*

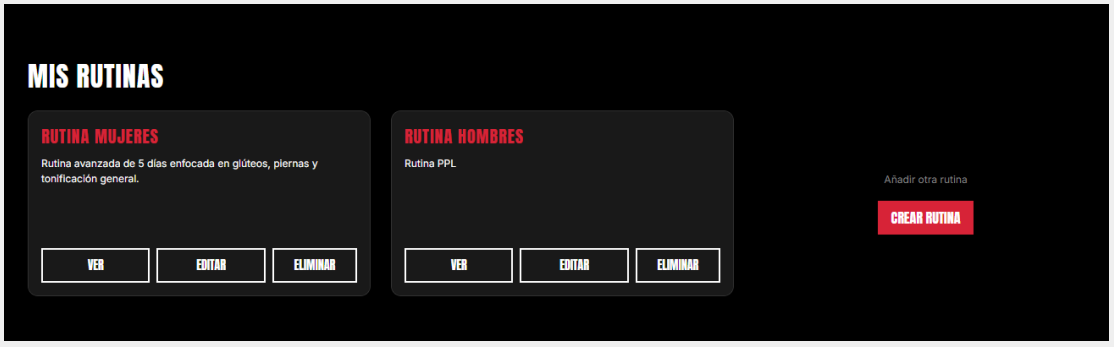


Imagen 6: *Dashboard de rutinas*

6. Análisis de requisitos

6.1. Requisitos funcionales (RF)

Listados y numerados (RF-01, RF-02...). P.ej. sistema de pago, carrito de la compra, etc.

ID	Requisito	Descripción
RF-0	**Registro y Autenticación**	Los usuarios podrán crear una cuenta mediante correo electrónico y contraseña, e iniciar sesión de forma segura.
RF-01	**Alta de usuarios**	El sistema permite dar de alta usuarios desde la propia aplicación, registrando sus datos principales.
RF-02	**Gestión de Perfil**	El usuario podrá consultar y editar sus datos personales, como nombre, correo, teléfono, fecha de nacimiento, objetivos físicos, foto de perfil y contraseña.
RF-03	**Información general del gimnasio**	La web mostrará información pública del gimnasio, incluyendo dirección, horario, contacto, servicios e instalaciones.
RF-04	**Visualización de planes**	El sistema mostrará los diferentes planes de suscripción, con sus características y precios.

RF-05	**Gestión de Membresías**	El sistema permitirá registrar, renovar o cancelar planes de pago asociados a los usuarios.
RF-06	**Visualización de rutinas**	El sistema permitirá a los clientes consultar rutinas disponibles o asignadas por un entrenador.
RF-07	**Rutinas predeterminadas**	La aplicación incluirá rutinas base organizadas por días, nivel o perfil de usuario, pudiendo incorporar vídeos, GIFs o enlaces de referencia.
RF-08	**Calendario de clases**	El sistema mostrará un calendario con clases dirigidas, horarios, tipo de actividad y plazas disponibles.
RF-09	**Reserva de clases**	Los usuarios abonados podrán inscribirse en clases dirigidas con control de aforo.
RF-10	**Código QR de acceso**	El usuario abonado podrá generar o visualizar un código QR de acceso al gimnasio
RF-11	**Control de entrada y salida**	El sistema registrará entradas y salidas mediante QR para conocer el número de usuarios presentes en el gimnasio.
RF-12	**Panel de administración**	El personal autorizado podrá acceder a un panel para gestionar usuarios, rutinas, clases, planes, pagos y accesos.

RF-13	**Gestión de caja**	El sistema permitirá registrar operaciones económicas, pagos de mensualidades, cobros en efectivo o tarjeta y cierres de caja.
RF-14	**Dashboard administrativo**	El panel mostrará indicadores como socios activos, altas, bajas, accesos, socios presentes e importe de ventas. (esto está por ver bien que no me acuerdo como sale)
RF-15	**Gestión de instructores**	El sistema permitirá que los instructores planifiquen y desarrollen actividades formativas o rutinas para los usuarios.
RF-16	**Pago simulado con Stripe**	La aplicación incluye un flujo de pago simulado mediante Stripe para representar la contratación de planes.

6.2. Requisitos no funcionales (RNF)

Listados y numerados como RNF-01, RNF-02, etc. P.ej. accesibilidad, rendimiento, seguridad, diseño, SEO, etc.

ID	Requisito	Descripción
RNF-01	**Diseño Responsive**	La interfaz debe adaptarse correctamente a dispositivos móviles, tablets y escritorio, siguiendo una estrategia responsive.

RNF-02	**Rendimiento**	Las páginas principales deben cargar de forma fluida, optimizando imágenes, estilos y scripts para reducir tiempos de espera.
RNF-03	**Seguridad de Datos**	El sistema debe proteger las contraseñas mediante hashing y restringir el acceso a las zonas privadas mediante autenticación y roles.
RNF-04	**Cifrado en producción**	En un entorno real, la aplicación deberá utilizar HTTPS para proteger la transmisión de datos.
RNF-05	**Cumplimiento legal**	La aplicación debe respetar el Reglamento General de Protección de Datos, especialmente al tratar datos personales y posibles datos vinculados a salud o entrenamiento.
RNF-06	**Usabilidad**	La aplicación debe ser sencilla de utilizar, con navegación clara, formularios comprensibles y mensajes de error útiles.
RNF-07	**Accesibilidad**	La interfaz debe cuidar el contraste, el tamaño de fuente, la estructura semántica, las etiquetas de formularios y los textos alternativos.
RNF-08	**Escalabilidad**	La arquitectura debe permitir añadir nuevas funcionalidades sin rehacer la aplicación desde cero.

RNF-09	**Mantenibilidad**	El código debe organizarse en rutas, controladores, modelos, vistas y migraciones para facilitar su mantenimiento.
RNF-10	**Disponibilidad**	La aplicación debe estar preparada para funcionar de forma estable en un servidor real durante el horario de uso del gimnasio.
RNF-11	**Integridad de datos**	La base de datos debe mantener relaciones coherentes entre usuarios, planes, pagos, clases, rutinas y accesos.
RNF-12	**Trazabilidad**	Las acciones críticas, como pagos, caja, reservas y accesos, deben quedar registradas para poder consultarse posteriormente.

6.3. Roles de usuario

Rol	Descripción	Funcionalidades principales
Visitante	Usuario no autenticado que accede a la parte pública de la web.	Consultar información del gimnasio, planes, contacto y registro.
Usuario abonado	Cliente registrado con acceso a funcionalidades privadas.	Gestionar perfil, consultar rutinas, reservar clases, ver planes y usar QR.
Administrador	Personal con permisos de gestión completa.	Gestionar usuarios, planes, caja, clases, rutinas, accesos y estadísticas.

6.4. Casos de uso principales

Los casos de uso principales de Iron Pulse son los siguientes:

1. Registro de un nuevo usuario.
2. Inicio de sesión de un usuario existente.
3. Alta de usuario desde el sistema.
4. Consulta de información del gimnasio.
5. Consulta de planes de suscripción.
6. Realización de pago simulado mediante Stripe.
7. Alta o renovación de una membresía.
8. Consulta de rutinas de entrenamiento.
9. Consulta de calendario de clases.
10. Reserva de una clase dirigida.
11. Generación o visualización del código QR.
12. Registro de entrada al gimnasio mediante QR.
13. Registro de salida del gimnasio mediante QR.
14. Gestión de usuarios desde el panel de administración.
15. Registro de pagos y operaciones de caja.
16. Consulta de estadísticas del dashboard.

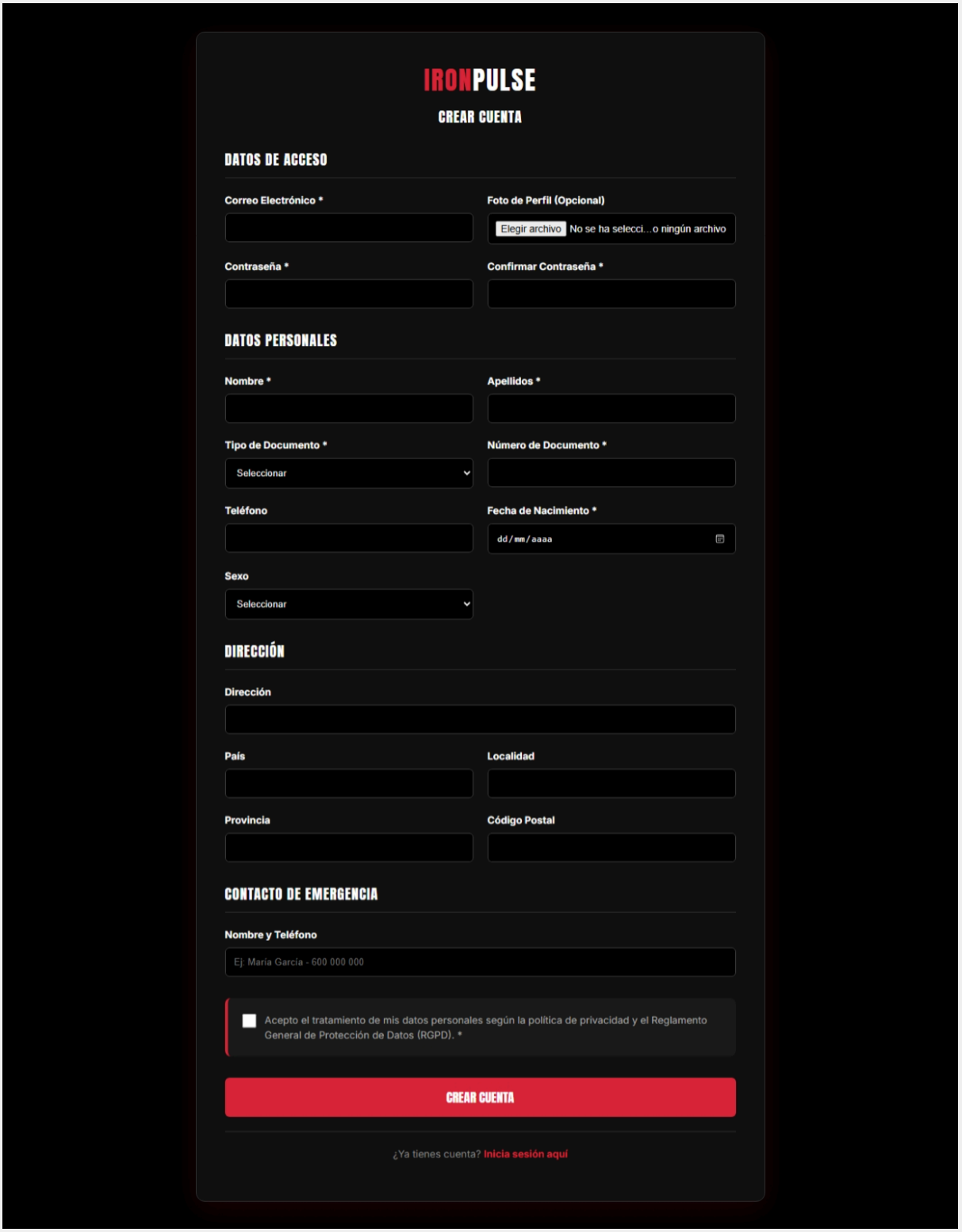


Imagen 7: *Formulario de registro*

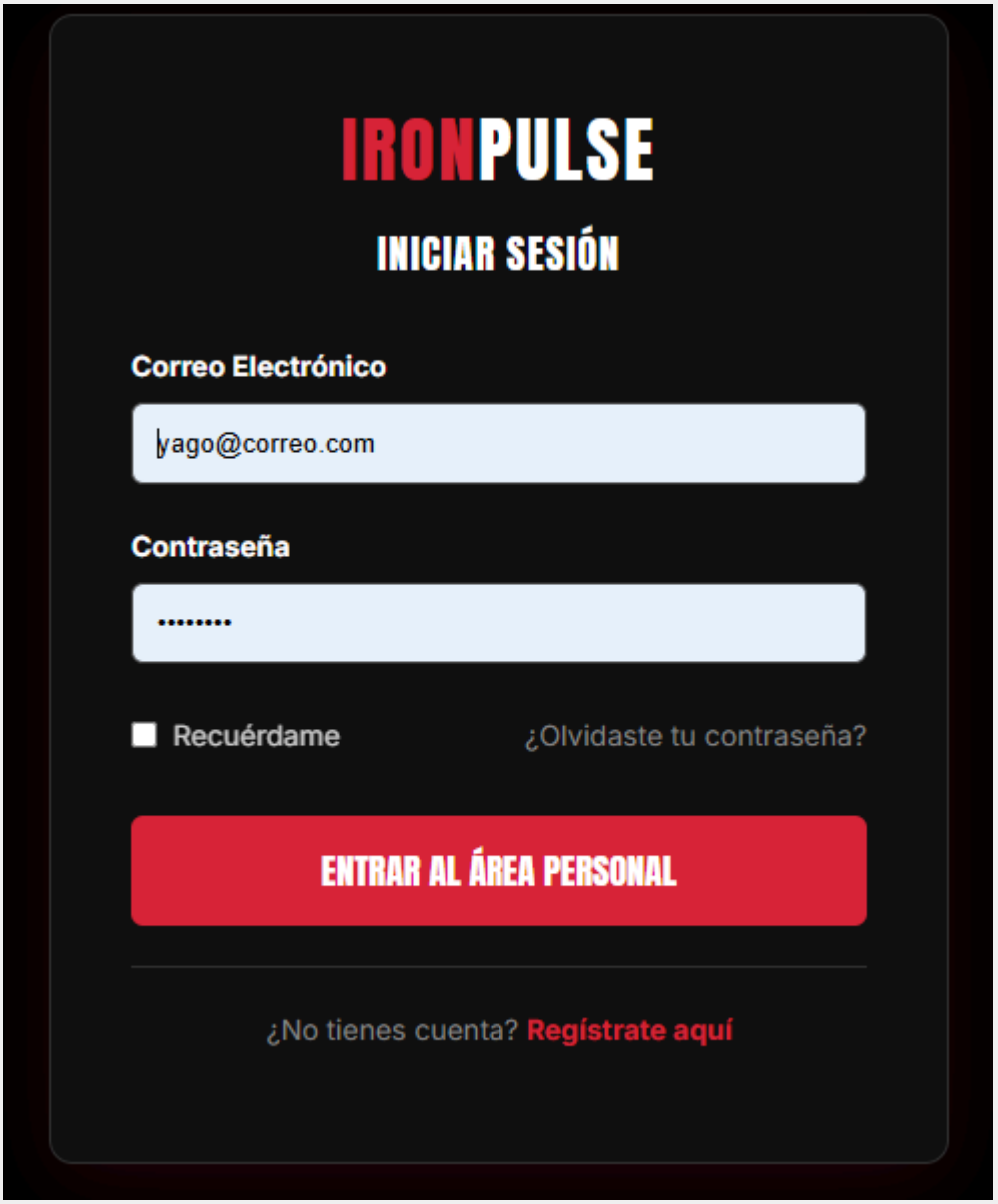


Imagen 8: *Formulario de inicio de sesión*

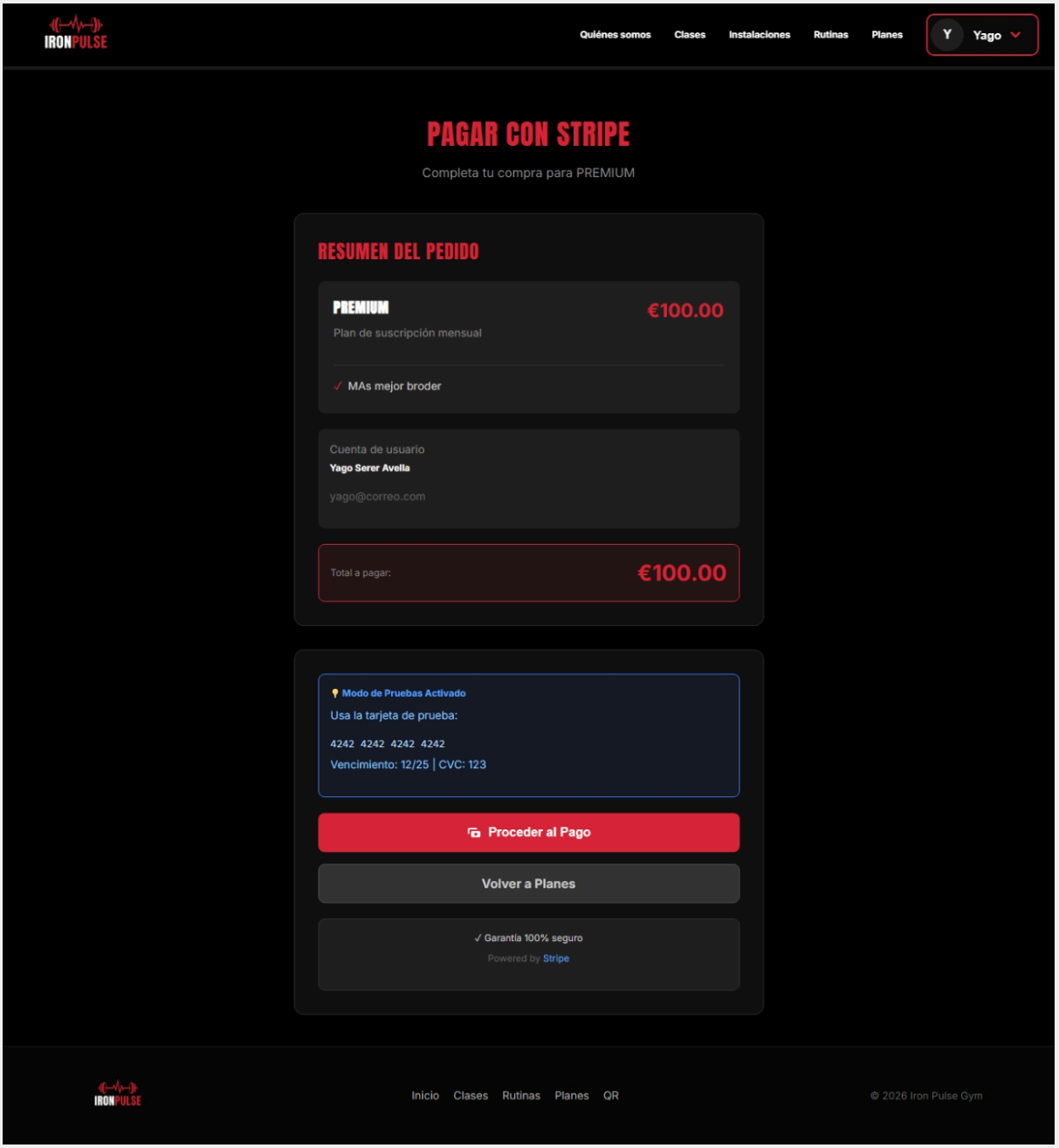


Imagen 9: *Pantalla de pago en Iron Pulse*

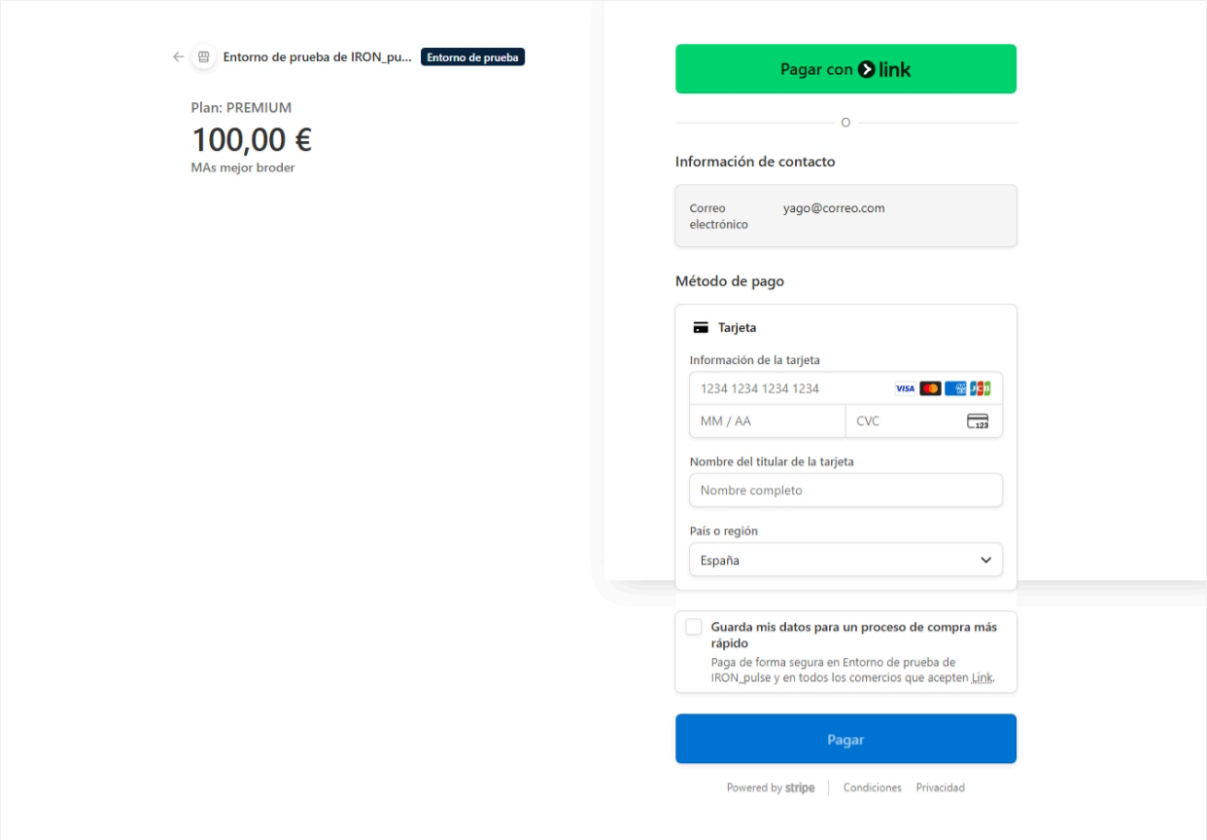


Imagen 10: *Pantalla de pago en STRIPE*

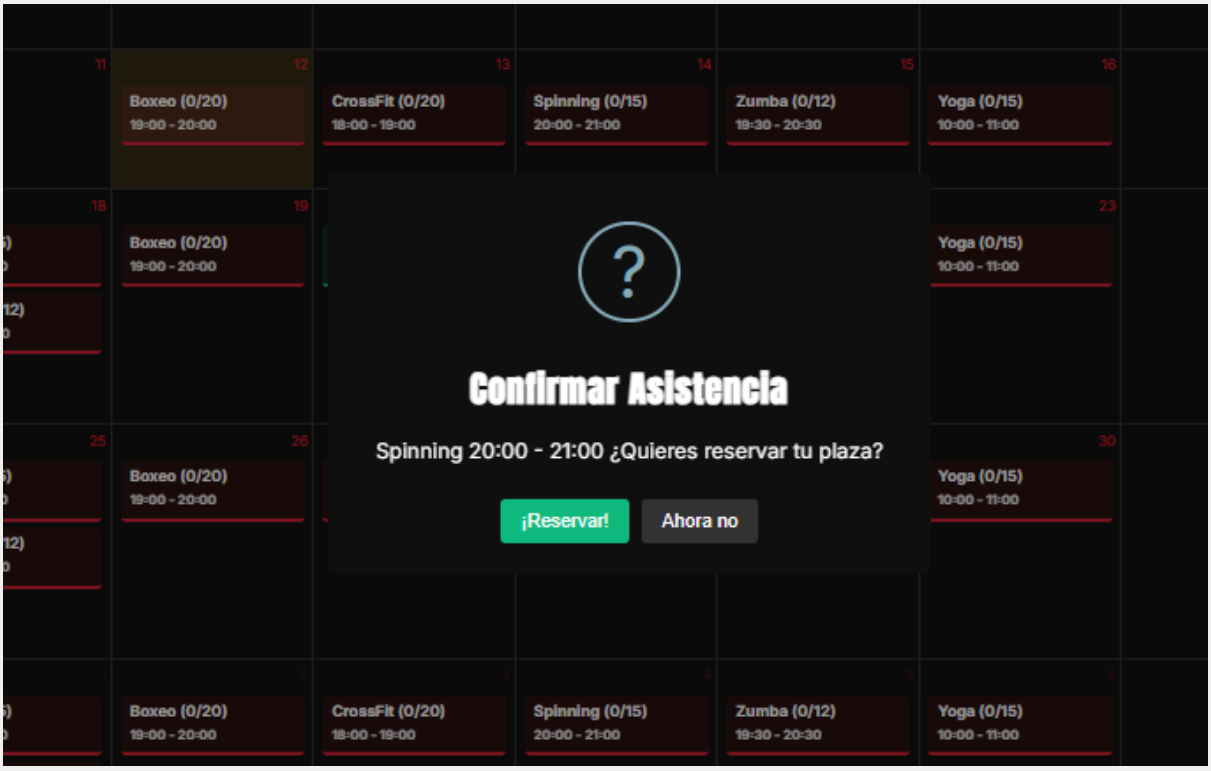


Imagen 11: *Pantalla de reserva de clase*

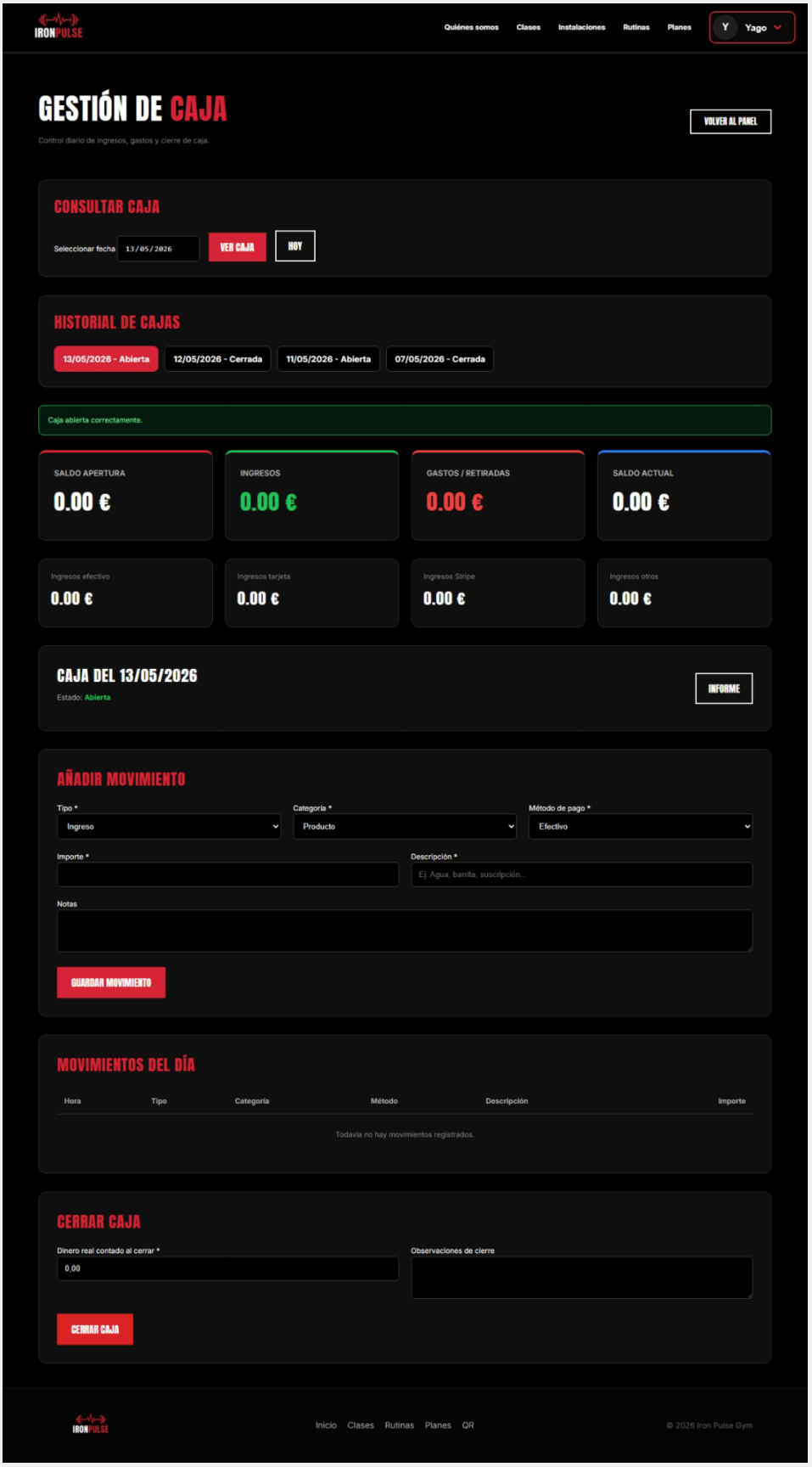


Imagen 12: *Gestión de caja con caja abierta*

5. Arquitectura del sistema

5.1. Tecnologías utilizadas

Para el desarrollo de Iron Pulse se ha seleccionado un stack tecnológico moderno, orientado a la seguridad, la escalabilidad, la mantenibilidad del código y la experiencia de usuario. La aplicación se ha construido sobre Laravel, siguiendo una arquitectura basada en el patrón **Modelo-Vista-Controlador**, lo que permite separar la lógica de negocio, la interfaz y la gestión de datos.

Las tecnologías utilizadas en el proyecto son las siguientes:

Área	Tecnología	Uso dentro del proyecto
Backend	PHP 8.2	Lenguaje principal del lado del servidor.
Framework backend	Laravel 12	Gestión de rutas, controladores, modelos, migraciones, validaciones, sesiones y autenticación.
Arquitectura	MVC	Separación entre modelos, vistas y controladores.
Plantillas	Blade	Motor de plantillas utilizado para crear vistas dinámicas y reutilizables.
Autenticación	Laravel Breeze	Sistema de login, registro, recuperación de contraseña, verificación y gestión de sesión.
Base de datos	MySQL / MariaDB	Almacenamiento de usuarios, planes, rutinas, clases, reservas, caja, pagos y accesos.
ORM	Eloquent	Comunicación entre los

		modelos de Laravel y las tablas de la base de datos.
Frontend	HTML5, CSS3 y JavaScript	Estructura, estilos e interactividad de las páginas.
Framework CSS	Tailwind CSS	Sistema de estilos basado en clases de utilidad para diseño responsive.
Maquetación	Flexbox y CSS Grid	Organización visual de tarjetas, formularios, paneles y secciones.
JavaScript	JavaScript Vanilla	Interactividad personalizada en el navegador.
Micro-framework JS	Alpine.js	Interactividad ligera en menús, modales y componentes dinámicos.
Peticiones asíncronas	Fetch API / Axios	Comunicación con el servidor sin recargar completamente la página.
Compilación de assets	Vite	Compilación y optimización de archivos CSS y JavaScript.
Iconografía	Blade Lucide Icons	Iconos SVG integrados directamente en las vistas Blade.
Alertas visuales	SweetAlert2	Ventanas emergentes personalizadas para confirmaciones, errores y avisos.
Fechas	Carbon	Gestión de fechas, vencimientos, horarios, accesos y cierres de caja.
Calendario	FullCalendar	Visualización y gestión

		interactiva de clases del gimnasio.
Escáner QR	Html5-QRCode	Lectura de códigos QR mediante la cámara del dispositivo.
Generación QR	QR Server API	Generación dinámica de códigos QR personalizados para los usuarios.
Pagos	Stripe PHP SDK	Integración de flujo de pago simulado mediante Stripe.
Dependencias PHP	Composer	Instalación y gestión de paquetes PHP.
Dependencias frontend	NPM	Instalación y gestión de paquetes JavaScript y herramientas frontend.
Entorno de desarrollo	Artisan / Laravel Sail	Ejecución de comandos, migraciones, servidor local y tareas de desarrollo.
Testing	PHPUnit	Base para pruebas automatizadas del proyecto.

5.2. Justificación de las tecnologías

Laravel y PHP

Laravel se ha elegido como framework principal del proyecto porque ofrece una estructura robusta, clara y preparada para aplicaciones web completas. Su arquitectura basada en el patrón **Modelo-Vista-Controlador** permite separar correctamente la lógica de negocio, la presentación y el acceso a los datos.

En Iron Pulse, Laravel se encarga de gestionar funcionalidades esenciales como el registro e inicio de sesión de usuarios, la administración de socios, los planes de suscripción, las rutinas, las clases, las reservas, la caja, los pagos simulados con Stripe y el control de accesos mediante QR.

Además, Laravel proporciona herramientas internas que facilitan el desarrollo seguro, como protección CSRF, validación de formularios, middleware, gestión de sesiones, migraciones, seeders y hashing de contraseñas.

Blade

Blade es el motor de plantillas propio de Laravel. Se utiliza para construir las vistas de la aplicación de forma dinámica y reutilizable.

Gracias a Blade, el proyecto puede compartir estructuras comunes como el layout principal, la navegación, el footer, los botones, los formularios y otros componentes visuales. Esto evita repetir código y mejora la mantenibilidad de la aplicación.

En Iron Pulse, Blade se utiliza en páginas como el dashboard, el perfil, los planes, las rutinas, las clases, el panel de administración, la caja y el flujo de pago con Stripe.

Laravel Breeze

Laravel Breeze se ha utilizado como base para el sistema de autenticación. Este paquete proporciona de forma inicial las funcionalidades de registro, inicio de sesión, cierre de sesión, recuperación de contraseña, verificación de email y edición de perfil.

Sobre esta base se han añadido funcionalidades propias de Iron Pulse, como la gestión de datos adicionales del usuario, el uso de planes, el acceso mediante QR, el panel de administración y la relación con las membresías.

MySQL / MariaDB y Eloquent ORM

La base de datos utilizada es relacional, basada en **MySQL** o **MariaDB**. Este tipo de base de datos resulta adecuado para Iron Pulse porque el sistema trabaja con entidades relacionadas entre sí, como usuarios, planes, rutinas, clases, reservas, registros de acceso, movimientos de caja y cierres diarios.

Laravel se comunica con la base de datos mediante **Eloquent ORM**, lo que permite trabajar con modelos en lugar de escribir consultas SQL manuales de forma constante. Esto facilita la lectura del código, reduce errores y mejora la seguridad frente a inyecciones SQL.

Por ejemplo, las relaciones entre usuarios y clases, usuarios y rutinas, o caja y movimientos económicos se gestionan mediante modelos y relaciones definidas en Laravel.

HTML5, CSS3, Tailwind CSS, Flexbox y Grid

La interfaz de Iron Pulse se ha construido utilizando **HTML5** para la estructura, **CSS3** para los estilos y **JavaScript** para la interactividad.

El diseño combina **Tailwind CSS** con estilos propios en CSS. Tailwind permite crear interfaces responsive de forma rápida mediante clases de utilidad, mientras que el CSS personalizado permite ajustar la estética visual del proyecto, especialmente el estilo oscuro, deportivo y premium de Iron Pulse.

También se utilizan técnicas de maquetación modernas como **Flexbox** y **CSS Grid**, que permiten organizar correctamente tarjetas, paneles, formularios, calendarios y secciones de administración.

JavaScript, Alpine.js, Fetch API y Axios

JavaScript se utiliza para añadir interactividad a la aplicación. En Iron Pulse se emplea tanto JavaScript nativo como librerías ligeras para mejorar la experiencia del usuario.

Alpine.js permite gestionar interacciones sencillas como menús desplegables, modales, cambios de estado visuales o componentes dinámicos sin necesidad de utilizar frameworks más pesados.

La **Fetch API** y **Axios** permiten realizar comunicaciones asíncronas con el servidor, por ejemplo en procesos como el escaneo de QR, la gestión de respuestas dinámicas o determinadas acciones que no requieren recargar toda la página.

Vite

Vite se utiliza como herramienta de compilación y optimización de assets. Permite trabajar durante el desarrollo con recarga rápida de estilos y scripts, y generar posteriormente una versión optimizada para producción.

En el proyecto se utiliza junto con Laravel Vite Plugin para integrar correctamente los archivos CSS y JavaScript dentro de las vistas Blade.

Blade Lucide Icons

Para la iconografía de la interfaz se utiliza **Blade Lucide Icons**, una librería de iconos SVG integrada directamente con Blade.

Esta elección permite mantener una línea visual coherente en toda la aplicación. Además, al tratarse de iconos vectoriales, se adaptan correctamente a diferentes tamaños de pantalla sin perder calidad.

Se utilizan en botones, menús, tarjetas, paneles de administración y secciones visuales de la web.

SweetAlert2

SweetAlert2 se utiliza para sustituir las alertas básicas del navegador por ventanas emergentes más visuales y coherentes con el diseño del proyecto.

En Iron Pulse puede utilizarse para confirmar acciones importantes, mostrar errores, avisar de operaciones realizadas correctamente, alertar sobre caja cerrada o informar sobre accesos mediante QR.

Carbon

Carbon es una librería de PHP especializada en el manejo de fechas y horas. En Iron Pulse resulta especialmente útil para gestionar vencimientos de membresías, registros de entrada y salida, fechas de clases, reservas, movimientos de caja y cierres diarios.

Su uso permite trabajar con fechas de forma más clara y segura que utilizando funciones básicas de PHP.

FullCalendar

FullCalendar se utiliza en la gestión de clases para ofrecer una interfaz de calendario visual e interactiva.

Gracias a esta librería, el administrador puede organizar clases y actividades de forma más intuitiva, mientras que los usuarios pueden consultar las clases disponibles y realizar reservas reales dentro del sistema.

Html5-QRCode y QR Server API

El sistema de control de accesos de Iron Pulse se apoya en el uso de códigos QR.

Por un lado, **QR Server API** se utiliza para generar dinámicamente los códigos QR asociados a cada usuario. Estos códigos permiten identificar al socio y validar su acceso.

Por otro lado, **Html5-QRCode** permite leer códigos QR mediante la cámara del dispositivo. Esto se utiliza en el panel de administración o escáner para registrar entradas y salidas del gimnasio.

El funcionamiento se basa en el identificador del usuario: cuando se escanea el QR, el sistema comprueba si el usuario está dentro o fuera del gimnasio y registra la acción correspondiente.

Stripe PHP SDK

Stripe se utiliza para representar el flujo de contratación de planes o membresías mediante pago con tarjeta. En la versión actual del proyecto, esta integración funciona en modo de pruebas o simulación.

La aplicación permite iniciar un proceso de pago, crear una sesión de Stripe y gestionar el resultado del pago. Esto demuestra cómo funcionaría el sistema en un entorno real, aunque para producción sería necesario configurar claves reales, webhooks definitivos, validación completa de transacciones y facturación real.

5.3. Arquitectura general de la aplicación

Iron Pulse sigue una arquitectura basada en el patrón **Modelo-Vista-Controlador**, propio de Laravel.

Este patrón divide la aplicación en tres partes principales:

- **Modelo:** representa los datos del sistema y su relación con la base de datos.
- **Vista:** representa la interfaz que ve el usuario.
- **Controlador:** recibe las peticiones, ejecuta la lógica necesaria y devuelve una respuesta.

Esta arquitectura facilita el mantenimiento del proyecto, ya que cada parte tiene una responsabilidad concreta.

Modelos

Los modelos se encuentran en la carpeta **app/Models** y representan las entidades principales de la aplicación.

Modelo	Función
User	Representa a los usuarios registrados. Incluye datos personales, rol, plan asociado y datos de contacto.
Plan	Representa los planes o membresías disponibles en el gimnasio.
Routine	Representa las rutinas de entrenamiento.
Event	Representa las clases o actividades del calendario.
AccessLog	Registra las entradas y salidas de usuarios mediante QR.
CashClosure	Representa la apertura y cierre diario de caja.
CashMovement	Representa los movimientos económicos registrados en caja.

Vistas

Las vistas se encuentran en la carpeta **resources/views** y están desarrolladas con Blade. Se organizan por módulos para mantener una estructura clara.

Carpeta / vista	Función
welcome.blade.php	Página principal pública de Iron Pulse.
dashboard.blade.php	Página principal tras iniciar sesión.
auth/	Vistas de login, registro, recuperación de contraseña y verificación.
profile/	Vistas relacionadas con el perfil del usuario.
plans/	Vistas relacionadas con planes y suscripciones.
stripe/	Vistas del flujo de pago con Stripe.
routines/	Vistas de creación, edición, listado y detalle de rutinas.
events/	Vistas de clases, reservas y clases del usuario.
admin/	Vistas del panel de administración.
admin/cash/	Vistas específicas de caja e informes.

layouts/	Plantillas principales de la aplicación.
components/	Componentes reutilizables de Blade.

Controladores

Los controladores se encuentran en **app/Http/Controllers** y gestionan la lógica de las peticiones.

Controlador	Responsabilidad principal
ProfileController	Gestión del perfil del usuario.
PlanController	Visualización, suscripción y cancelación de planes.
RoutineController	Gestión de rutinas de entrenamiento.
EventController	Consulta de clases, reservas y cancelaciones.
AdminController	Panel de administración, gestión de usuarios, planes, rutinas, clases, escáner QR y accesos.
CashController	Apertura de caja, movimientos, cierres, reaperturas e informes.
StripeController	Gestión del flujo de pago simulado con Stripe.

Controladores de Auth/	Registro, login, recuperación de contraseña, confirmación y verificación de email.

Rutas

Las rutas principales se definen en el archivo **routes/web.php**. Además, las rutas de autenticación se incluyen mediante **routes/auth.php**.

La aplicación organiza sus rutas en varios bloques:

Tipo de ruta	Descripción	Ejemplos
Rutas públicas	Accesibles sin iniciar sesión.	/
Rutas de usuario autenticado	Requieren login.	/dashboard, /profile, /planes, /mis-rutinas, /clases
Rutas de administración	Gestión interna del gimnasio.	/panel-control, /usuarios, /gestion-clases, /gestion-planes, /escaner, /admin/caja
Rutas de Stripe	Flujo de pago simulado.	/stripe/planes/{plan}/comprar, /stripe/pago-exitoso, /stripe/webhook

Las rutas privadas están protegidas mediante autenticación.

5.4. Estructura de carpetas y organización del código

El proyecto sigue la estructura estándar de Laravel, lo que facilita su mantenimiento y permite que cualquier desarrollador con conocimientos del framework pueda entender rápidamente la organización del código.

Carpeta / archivo	Descripción
app/Models/	Contiene los modelos principales del sistema: User, Plan, Routine, Event, AccessLog, CashClosure y CashMovement.
app/Http/Controllers/	Contiene los controladores encargados de gestionar la lógica de cada módulo.
app/Http/Controllers/Auth/	Contiene los controladores de autenticación proporcionados por Laravel Breeze.
app/Http/Requests/	Contiene clases de validación de formularios.
resources/views/	Contiene las vistas Blade de la aplicación.
resources/views/admin/	Contiene las vistas del panel de administración.
resources/views/admin/cash/	Contiene las vistas de gestión de caja e informes.
resources/views/auth/	Contiene las vistas de autenticación.
resources/views/events/	Contiene las vistas relacionadas con clases y reservas.

resources/views/plans/	Contiene las vistas relacionadas con planes.
resources/views/profile/	Contiene las vistas del perfil del usuario.
resources/views/routines/	Contiene las vistas de rutinas.
resources/views/stripe/	Contiene las vistas del flujo de pago con Stripe.
resources/views/layouts/	Contiene las plantillas base de la aplicación.
resources/views/components/	Contiene componentes reutilizables de Blade.
resources/css/	Contiene los estilos principales de la aplicación.
resources/js/	Contiene los scripts JavaScript del frontend.
routes/web.php	Define las rutas principales de la aplicación.
routes/auth.php	Define las rutas de autenticación generadas por Breeze.
database/migrations/	Contiene las migraciones que crean y modifican las tablas de la base de datos.
database/seeders/	Contiene datos iniciales o de prueba.

public/	Contiene recursos públicos como imágenes, logos y archivos accesibles desde el navegador.
config/	Contiene los archivos de configuración del framework.
composer.json	Define las dependencias PHP del proyecto.
package.json	Define las dependencias frontend y scripts de Vite.
STRIPE_SETUP_GUIDE.md	Documento auxiliar con instrucciones de configuración de Stripe.

Esta organización permite mantener separadas las distintas responsabilidades del proyecto. Los modelos gestionan los datos, los controladores procesan la lógica, las vistas representan la interfaz y las migraciones definen la estructura de la base de datos.

5.5. Flujo general de funcionamiento

El flujo general de funcionamiento de Iron Pulse puede resumirse de la siguiente forma:

1. El visitante accede a la página principal de Iron Pulse y puede consultar información pública del gimnasio, como servicios, planes, clases, instalaciones y contacto.
2. Si desea acceder a funcionalidades privadas, el usuario debe registrarse o iniciar sesión mediante el sistema de autenticación.
3. Una vez autenticado, el usuario accede a su zona privada o dashboard, desde donde puede consultar su información personal, planes, rutinas, clases y reservas.
4. El usuario puede consultar los planes disponibles y realizar un flujo de pago simulado mediante Stripe para contratar o renovar una membresía.
5. El sistema registra la información relacionada con el plan o pago y permite asociarla al usuario correspondiente.

6. El usuario puede consultar el calendario de clases y realizar reservas reales en las actividades disponibles.
7. El usuario dispone de un código QR asociado a su identificador. Este QR se utiliza para controlar el acceso al gimnasio.
8. Cuando el QR se escanea desde el panel de administración, el sistema comprueba el estado actual del usuario y registra una entrada o una salida.
9. El administrador accede al panel de control para gestionar usuarios, planes, rutinas, clases, reservas, accesos, pagos y caja.
10. Desde el módulo de caja, el administrador puede abrir caja, registrar movimientos, cerrar caja, reabrirla si es necesario y consultar informes.
11. Todos los datos importantes quedan registrados en la base de datos, permitiendo mantener trazabilidad sobre usuarios, reservas, accesos, pagos y operaciones económicas.

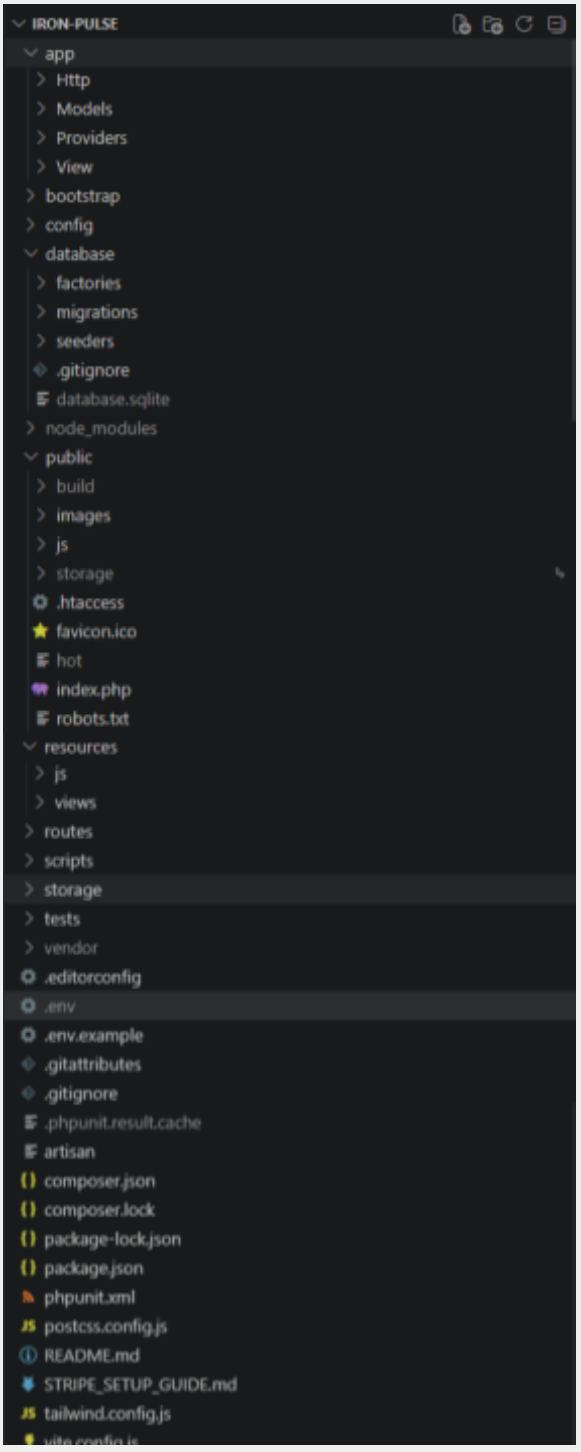


Imagen 13: *carpetas principales del proyecto*

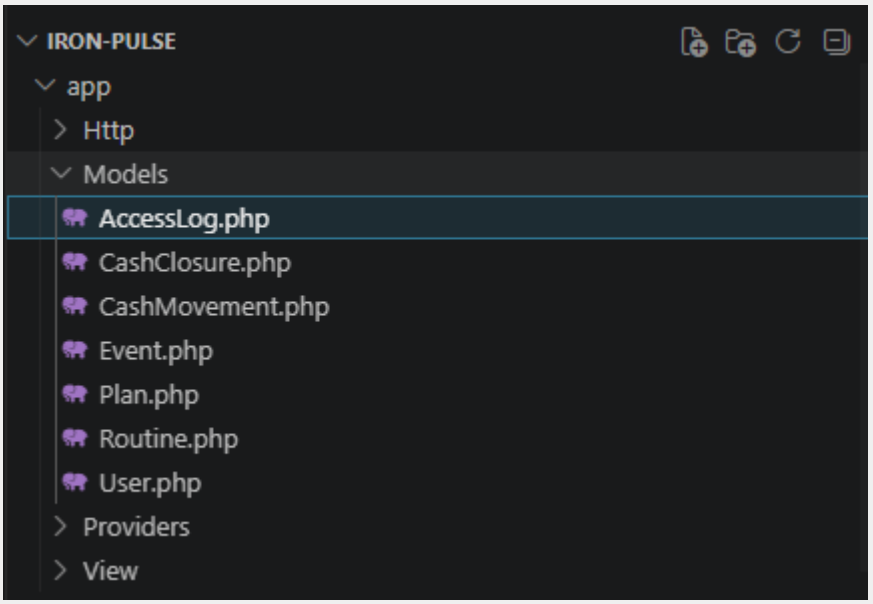


Imagen 14: *Modelos del proyecto*

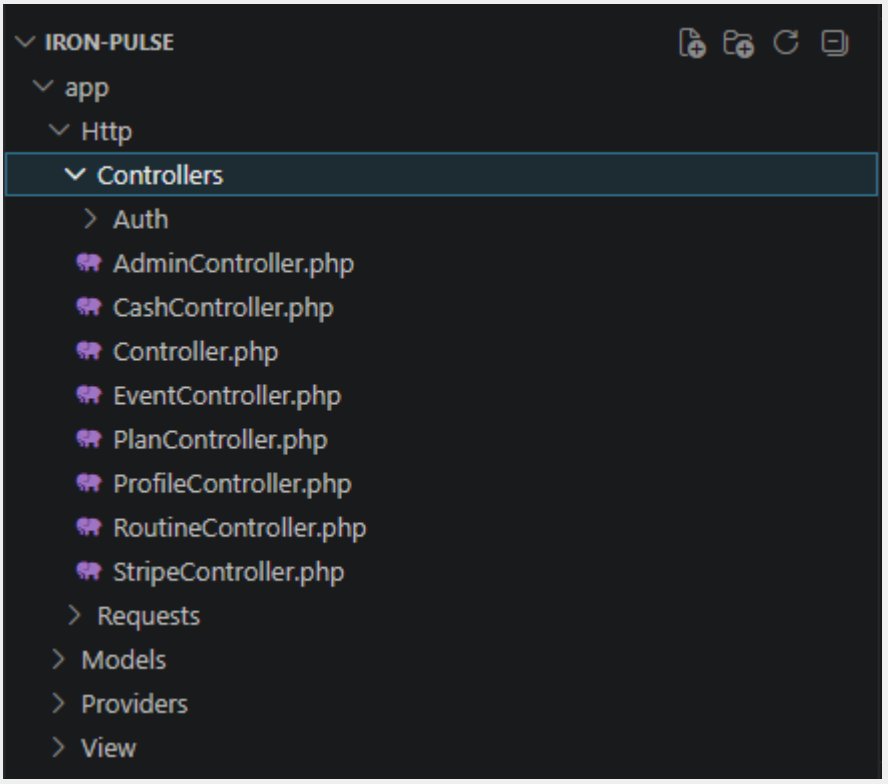


Imagen 15: *Modelos del proyecto*

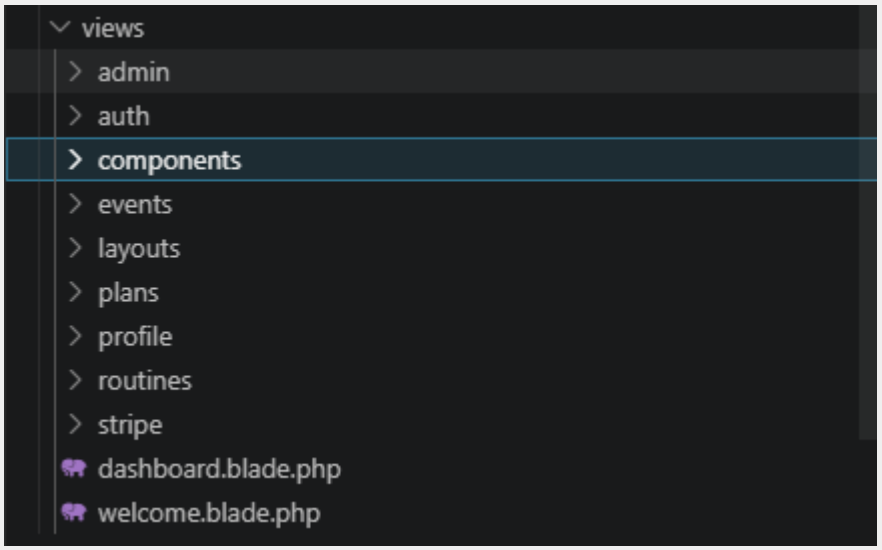


Imagen 16: *Modelos del proyecto*



Imagen 17: *Archivo web.php*

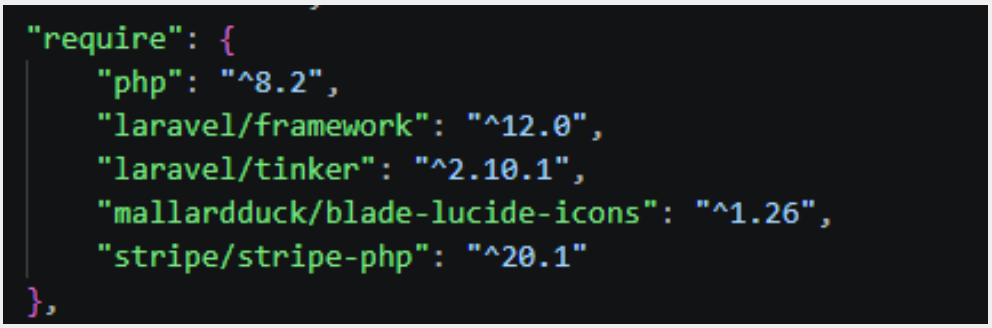


Imagen 18: *Requerimientos implícitos en el composer.json*

Las capturas de la estructura del proyecto muestran la organización interna de Iron Pulse siguiendo el estándar de Laravel. En ellas se aprecia la separación entre modelos, controladores, vistas, rutas, migraciones y recursos públicos.

La carpeta **app/Models** contiene las entidades principales del sistema, como usuarios, planes, rutinas, eventos, registros de acceso y caja. La carpeta **app/Http/Controllers** agrupa la lógica de cada módulo funcional, incluyendo administración, caja, clases, planes, perfil, rutinas y Stripe.

Por su parte, **resources/views** contiene las plantillas Blade que forman la interfaz de usuario, organizadas por secciones. Esta separación permite mantener un código más limpio y facilita la ampliación del proyecto.

El archivo **routes/web.php** centraliza las rutas principales de la aplicación, diferenciando entre rutas públicas, rutas privadas de usuario, rutas de administración y rutas relacionadas con Stripe. Finalmente, **composer.json** y **package.json** muestran

las dependencias utilizadas tanto en backend como en frontend.

6. Diseño de Base de Datos

6.1. Modelo Entidad-Relación (DER / ERD) y 6.2. Modelo Lógico Relacional

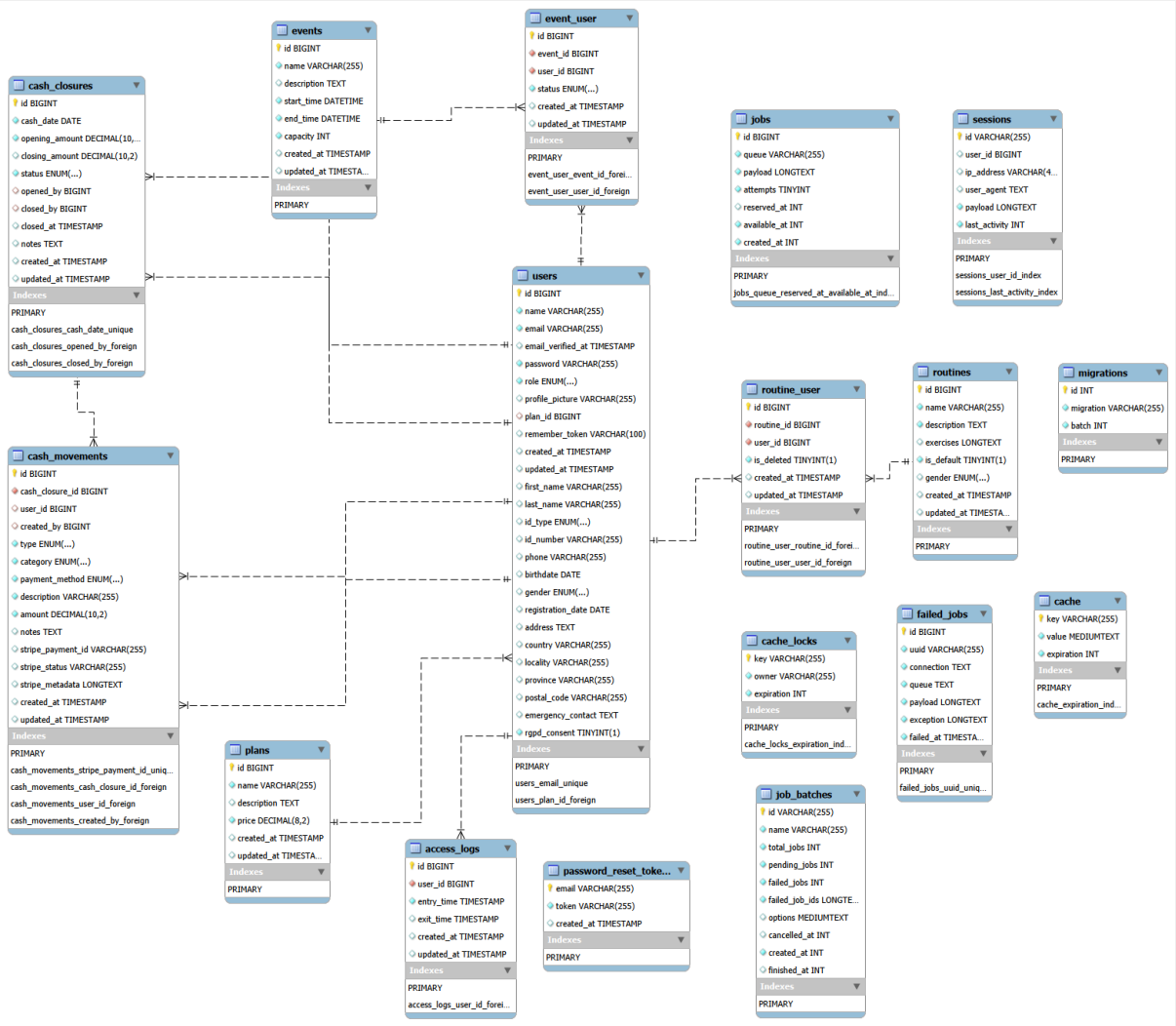


Imagen 19: *Diagrama Entidad-Relación de la base de datos del proyecto*

El modelo de datos está organizado en torno a los siguientes grupos de entidades:

1. Gestión de Usuarios y Membresías

- **users:** Es la entidad central. Almacena el perfil completo del usuario (datos personales, contacto de emergencia, dirección y consentimiento RGPD). Se relaciona con casi todas las áreas del sistema.

- **plans:** Define los tipos de suscripción disponibles (ej. Mensual, VIP). Tiene una relación de **1:N** con **users** a través de **plan_id**, indicando que un plan puede tener muchos usuarios, pero cada usuario pertenece a un solo plan.
- **access_logs:** Registra los accesos físicos al centro. Tiene una relación **1:N** con **users** (**user_id**), permitiendo auditar cuándo entra y sale cada cliente.
- **password_reset_tokens:** Tabla técnica para gestionar la recuperación de contraseñas mediante tokens vinculados al email del usuario.

2. Sistema de Entrenamiento y Rutinas

- **routines:** Almacena las plantillas de entrenamiento, incluyendo la lista de ejercicios y el género al que van dirigidas.
- **routine_user:** Tabla pivote que gestiona la relación **N:M** entre **usuarios** y **rutinas**. Permite que un usuario tenga asignadas varias rutinas a lo largo del tiempo y que una rutina sea usada por múltiples clientes. Incluye el campo **is_deleted** para bajas lógicas.

3. Actividades y Eventos

- **events:** Define clases grupales o eventos especiales, con control de horarios y capacidad máxima.
- **event_user:** Tabla intermedia para la relación **N:M** entre **users** y **events**. Gestiona las inscripciones y el status de la asistencia del usuario a un evento específico.

4. Gestión Financiera y Caja

- **cash_closures:** Controla los cortes de caja. Registra quién abrió/cerró la caja (**opened_by**, **closed_by** vinculados a **users**) y los balances finales.
- **cash_movements:** Detalla cada transacción económica. Se relaciona con:
    - **cash_closures (1:N):** Cada movimiento pertenece a un cierre de caja.
    - **users (1:N):** El campo user_id identifica al cliente del movimiento y **created_by** al empleado que lo registró.
    - Incluye integración con Stripe para pagos digitales.

5. Infraestructura de la Aplicación (Laravel)

- **sessions:** Almacena los datos de sesión de los usuarios activos, incluyendo IP y agente de usuario.
- **jobs, failed_jobs y job_batches:** Forman el sistema de colas. Permiten ejecutar tareas pesadas (como envío de emails masivos) en segundo plano sin bloquear la interfaz.
- **cache y cache_locks:** Optimizan el rendimiento guardando datos temporales de acceso frecuente.
- **migrations:** Registro técnico que controla las versiones y cambios realizados en la estructura de la base de datos.

Este esquema asegura una trazabilidad total: desde quién realizó un cobro en caja hasta qué rutina está siguiendo un usuario o cuántas veces ha asistido al gimnasio esta semana.

6.2 Diagrama de flujo de procesos

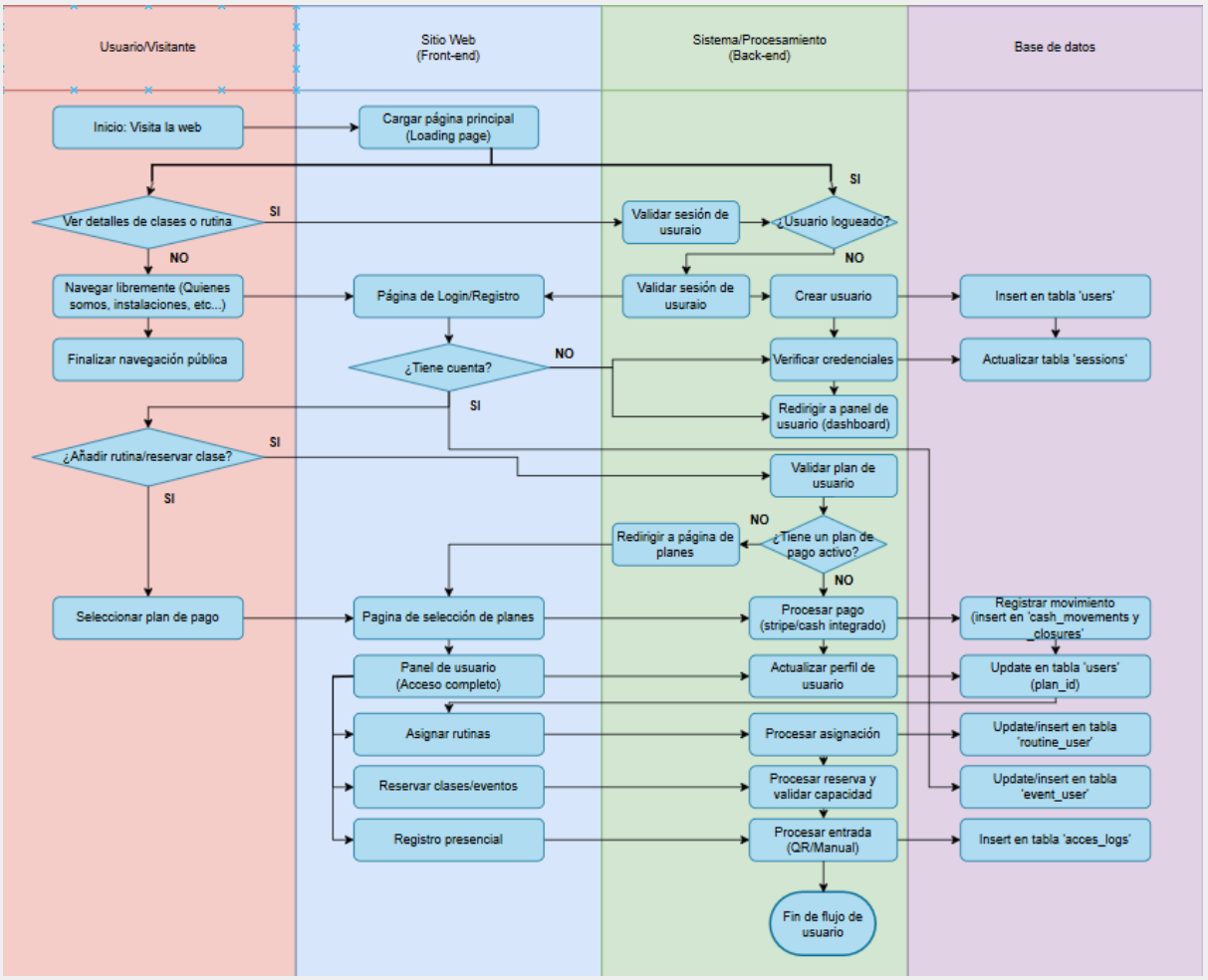


Imagen 20: *Diagrama de flujo de procesos*

El diagrama representa los procesos fundamentales de la plataforma Iron Pulse: la gestión de entrenamientos personalizados, la reserva de actividades y el control financiero de membresías. Todo el flujo está diseñado bajo un modelo de embudo que prioriza la conversión, asegurando que las funcionalidades de valor queden vinculadas a usuarios activos y con planes vigentes.

En el flujo de acceso y suscripción, el usuario navega de forma pública por la información corporativa, pero es redirigido al sistema de autenticación al intentar interactuar con servicios específicos. Una vez logueado, el sistema verifica el estado de su plan; en caso de no contar con uno, se gestiona la compra a través de una pasarela de pago (Stripe), actualizando su perfil y registrando el movimiento en la caja del centro.

En el flujo operativo de entrenamiento y clases, el cliente con plan activo puede asignar rutinas a su perfil o reservar plaza en eventos programados, validando en tiempo real la capacidad de los mismos. Finalmente, el sistema permite el registro de

acceso presencial, garantizando una trazabilidad completa desde la primera visita web hasta la asistencia física al gimnasio.

Esta estructura garantiza una monetización eficiente, mejora la seguridad de los datos mediante el control de sesiones y facilita una experiencia de usuario fluida y profesional.

6.3. Descripción de tablas principales

6.3.1. Módulo de Usuarios y Acceso

Este es el núcleo del sistema. Gestiona quiénes son los miembros y cómo acceden a la plataforma.

- **users:** Es la tabla central. Almacena la identidad de cada persona (nombre, email, contraseña), su rol en el sistema (admin, cliente, etc.) y su información personal (DNI, teléfono, dirección). Está vinculada directamente a un plan de suscripción.
- **plans:** Define los niveles de membresía disponibles (ej. Plan Gold, Plan Mensual). Contiene el nombre del plan, su descripción y el precio.
- **access_logs:** Registra cada vez que un usuario entra o sale de las instalaciones o del sistema, guardando la marca de tiempo exacta de entrada y salida.

6.3.2. Módulo de Entrenamiento y Actividades

Gestiona la oferta deportiva del gimnasio.

- **routines:** Almacena las rutinas de ejercicio. El campo exercises es clave, ya que guarda la lista técnica de movimientos que el usuario debe seguir.
- **events**: Gestiona clases grupales, talleres o eventos especiales. Incluye el horario de inicio/fin y la capacidad máxima de personas.
- **routine_user y event_user**: Son tablas de "relación". Permiten que un usuario tenga múltiples rutinas asignadas y que pueda inscribirse en varios eventos a la vez.

6.3.3. Módulo Financiero y de Caja

Controla el flujo de dinero y las transacciones.

- **cash_closures:** Representa el "Cierre de Caja". Registra con cuánto dinero se abre y se cierra un turno o día, y qué usuarios fueron los responsables de abrir y cerrar esa caja.

- **cash_movements:** Es el detalle de cada transacción. Cada vez que hay un pago (vía Stripe u otro método) o un movimiento de efectivo, se registra aquí con su monto y categoría, siempre vinculado a un cierre de caja específico.

6.3.4. Tablas de Soporte Técnico

Estas tablas aseguran que la aplicación funcione correctamente a nivel de software.

- **sessions:** Mantiene el rastro de las conexiones activas de los usuarios en la web o app, incluyendo su dirección IP y el navegador que usan.
- **failed_jobs y job_batches**: Gestionan procesos automáticos en segundo plano (como el envío masivo de correos o cobros automáticos) y registran si alguno falla.

6.4 Relaciones y cardinalidades

6.4.1. Relaciones Uno a Muchos (1:N)

- **plans 1:N users:** Un plan de suscripción puede ser contratado por muchos usuarios, pero cada usuario tiene asignado un solo plan principal.
- **users 1:N access_logs:** Un usuario genera múltiples registros de entrada y salida a lo largo del tiempo.
- **users 1:N sessions:** Un usuario puede tener varias sesiones abiertas (ej. en su móvil y en su ordenador).
- **cash_closures 1:N cash_movements:** Un cierre de caja agrupa muchos movimientos de dinero (pagos, ventas, retiros) realizados durante ese turno.
- **users 1:N cash_movements:** Un usuario (el cliente) puede realizar muchos pagos o transacciones diferentes.

6.4.2. Relaciones Muchos a Muchos (N:M)

- **users N:M events (vía event_user):**
    - Muchos usuarios pueden inscribirse a un mismo evento (clase).
    - Un usuario puede estar inscrito en muchos eventos diferentes.
- **users N:M routines (vía routine_user):**
    - Muchos usuarios pueden seguir la misma rutina de entrenamiento.
    - Un usuario puede tener asignadas varias rutinas (ej. una para cardio y otra para fuerza).

6.4.3. Relaciones de Gestión (Autoría)

- **users 1:N cash_closures:** Un empleado (admin) puede ser el responsable de abrir o cerrar muchas cajas diferentes a lo largo de su contrato.
- **users 1:N cash_movements (creado por):** Además del cliente, la tabla de movimientos registra qué empleado creó el registro del pago.

6.5 Integridad, normalización e índices

6.5.1. Integridad de Datos

- **Integridad de Entidad (PK):** Cada tabla tiene una **PRIMARY KEY (ID)**, lo que garantiza que no existan registros duplicados (ej. no habrá dos usuarios con el mismo ID).
- **Integridad Referencial (FK)**: Las líneas que conectan tablas (como user_id en access_logs) aseguran que no existan registros "huérfanos". Por ejemplo, no puedes registrar un acceso si el usuario no existe previamente en la tabla users.
- **Integridad de Dominio:** El uso de tipos de datos específicos como ENUM en los campos role o status limita lo que se puede escribir, evitando errores de dedo (ej. solo permite 'admin' o 'cliente').

6.5.2. Normalización

- **Eliminación de duplicidad:** En lugar de escribir el nombre del plan dentro de cada usuario, se ha creado una tabla plans. Así, si el precio cambia, solo lo editas en un lugar.
- **Tablas de ruptura:** El uso de event_user y routine_user demuestra que hemos normalizado las relaciones "Muchos a Muchos", separando los datos para que la base de datos sea ligera y escalable.

6.5.3. Índices

- **Búsquedas rápidas:** Tenemos índices en campos clave como email en la tabla users. Esto hace que el inicio de sesión sea instantáneo aunque tengas miles de usuarios.
- **Optimización de relaciones:** Casi todas las llaves foráneas (FK) tienen índices automáticos para que, al consultar "qué rutinas tiene este usuario", el sistema no se ralentice.
- **Índices Únicos:** Como el de cash_closures_cash_date_unique, que garantiza que no se creen dos cierres para la misma fecha por error.

7. Planificación temporal (PASAR A NRO 1)

7.1. Fases del proyecto

La planificación temporal de Iron Pulse se ha organizado en diferentes fases de trabajo, siguiendo una evolución progresiva desde el análisis inicial hasta la documentación final del proyecto. Al tratarse de una aplicación web completa, el desarrollo no se ha planteado como una única tarea, sino como un conjunto de bloques relacionados entre sí: análisis, diseño, desarrollo, pruebas, ajustes y documentación.

La planificación comienza el **20 de septiembre**, fecha en la que se inicia la definición del proyecto, la búsqueda de necesidades y la organización general del trabajo. La documentación se mantiene activa durante todo el proceso, ya que no se limita a una fase final, sino que acompaña al desarrollo desde el principio hasta la fecha actual. Esto permite registrar decisiones técnicas, cambios en funcionalidades, capturas, problemas encontrados y mejoras aplicadas.

Las fases principales del proyecto son las siguientes:

Fase 1: Análisis inicial

En esta fase se estudia la idea del proyecto, se define el objetivo general de la aplicación y se identifican los usuarios principales del sistema. También se determinan las funcionalidades necesarias para que Iron Pulse pueda funcionar como una plataforma de gestión de gimnasio.

Durante esta fase se trabajan aspectos como:

- Definición de la idea inicial.
- Identificación de los perfiles de usuario.
- Análisis de necesidades del gimnasio.
- Definición de funcionalidades principales.
- Estudio de viabilidad técnica.
- Primer planteamiento de la base de datos.

Fase 2: Diseño funcional y visual

Una vez definida la idea, se inicia el diseño de la aplicación. En esta fase se trabaja la estructura visual, la navegación, la organización de pantallas y la identidad gráfica de Iron Pulse.

Incluye tareas como:

- Diseño de wireframes.
- Diseño de mockups.
- Definición de la paleta de colores.
- Selección de tipografías.
- Diseño de componentes visuales.
- Organización del mapa de navegación.
- Definición inicial de la arquitectura MVC.

Fase 3: Diseño de base de datos y arquitectura

En esta fase se diseña la estructura técnica interna del sistema. Se definen las tablas principales, sus relaciones y la forma en la que Laravel organizará los modelos, controladores y vistas.

Incluye:

- Diseño del modelo Entidad–Relación.
- Definición del modelo lógico relacional.
- Creación de migraciones.
- Definición de relaciones entre usuarios, planes, rutinas, clases, reservas, caja y accesos.
- Organización inicial de rutas y controladores.

Fase 4: Desarrollo base de la aplicación

Esta fase corresponde al inicio del desarrollo técnico. Se configura el entorno de Laravel, la autenticación y la estructura general del proyecto.

Incluye:

- Instalación y configuración de Laravel.
- Configuración de Laravel Breeze.
- Creación del sistema de registro e inicio de sesión.
- Configuración de layouts principales.
- Creación del dashboard inicial.
- Primeras vistas públicas y privadas.

Fase 5: Desarrollo de funcionalidades principales

En esta fase se implementan los módulos principales de Iron Pulse. Es la fase más extensa del proyecto, ya que incluye la mayor parte de la lógica funcional.

Incluye:

- Gestión de usuarios.
- Gestión de planes y membresías.
- Gestión de rutinas.
- Gestión de clases y calendario.
- Sistema de reservas reales.
- Panel de administración.
- Sistema de caja.
- Control de accesos mediante QR.
- Integración del flujo de pago simulado con Stripe.

Fase 6: Pruebas, correcciones y ajustes

Tras implementar las funcionalidades principales, se realizan pruebas para comprobar que la aplicación funciona correctamente y que los datos se guardan adecuadamente.

Incluye:

- Pruebas de registro e inicio de sesión.
- Pruebas de reservas de clases.
- Pruebas de pagos simulados.
- Pruebas de caja.
- Pruebas de entrada y salida mediante QR.
- Revisión de errores visuales.
- Ajustes responsive.
- Corrección de errores detectados.

Fase 7: Documentación continua

La documentación se desarrolla de forma paralela al resto del proyecto, desde el inicio hasta el final. Esta fase incluye la redacción de la memoria, la recopilación de capturas, la elaboración de diagramas, la explicación de funcionalidades y la preparación de la presentación final.

Incluye:

- Redacción de la memoria.
- Capturas de pantalla de la aplicación.
- Explicación de tecnologías utilizadas.

- Documentación de base de datos.
- Documentación de pruebas.
- Preparación de anexos.
- Preparación de la presentación final.

7.2. Diagrama de Gantt

El siguiente diagrama de Gantt representa la planificación temporal del proyecto Iron Pulse. La planificación parte del **20 de septiembre** y se extiende hasta la fecha actual, incluyendo análisis, diseño, desarrollo, pruebas y documentación continua.

La documentación aparece como una tarea transversal, ya que se ha trabajado desde el inicio del proyecto y se mantiene activa hasta la entrega final.

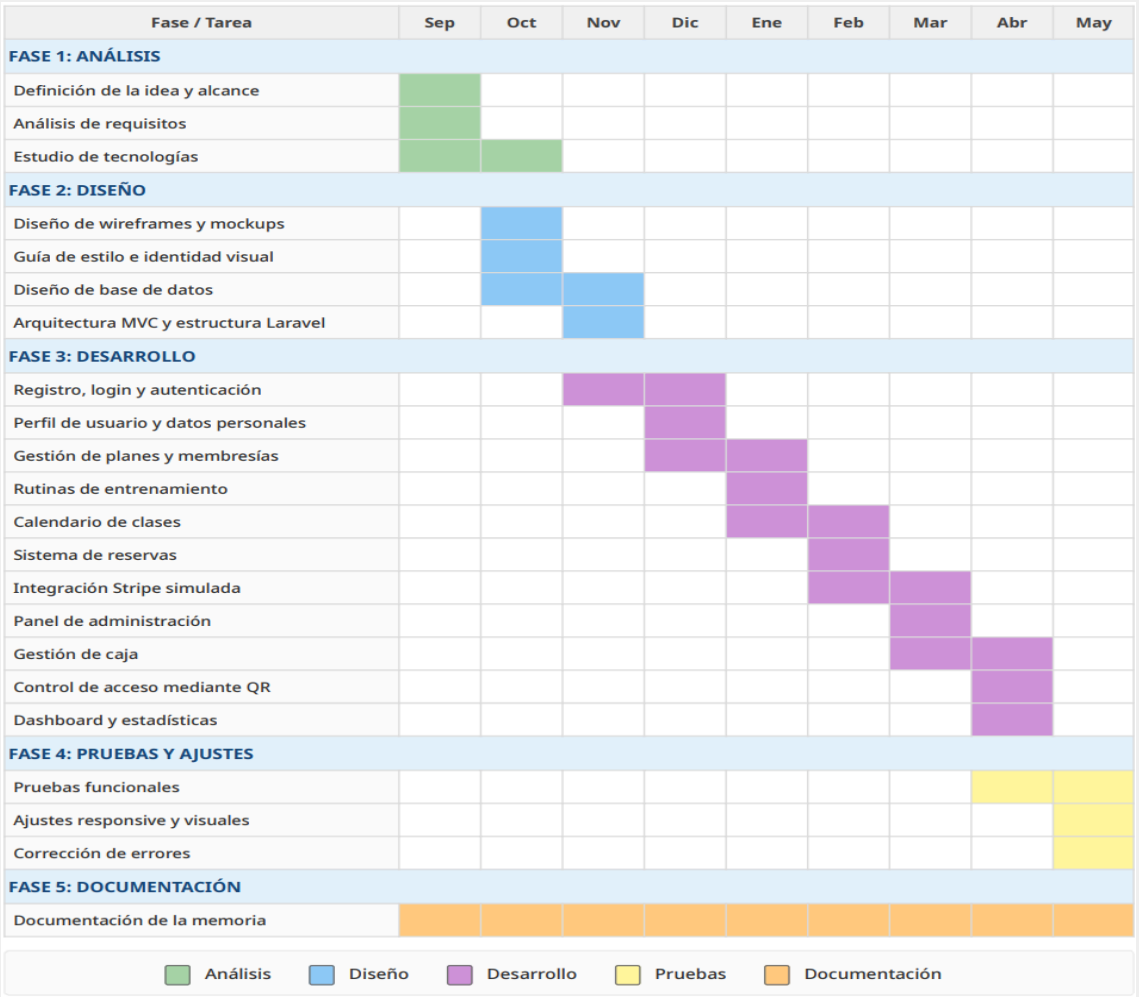


Imagen 21: *Diagrama de gantt*

7.2.1. Detalle de Tareas y Duración Estimada

Tarea	Duración	Dependencia
Definición de la idea y alcance	2 semanas	-
Análisis de requisitos	2 semanas	Definición de la idea
Estudio de tecnologías	3 semanas	-
Diseño de wireframes y mockups	3 semanas	Análisis de requisitos
Guía de estilo e identidad visual	1.5 semanas	-
Diseño de base de datos	4 semanas	Análisis de requisitos
Arquitectura MVC y estructura Laravel	2 semanas	Diseño de BBDD
Registro, login y autenticación	4 semanas	Estructura Laravel
Perfil de usuario y datos personales	2 semanas	Autenticación
Gestión de planes y membresías	3 semanas	Autenticación
Rutinas de entrenamiento	3 semanas	Perfil de usuario
Calendario de clases	3 semanas	Planes y membresías
Sistema de reservas	2 semanas	Calendario de clases
Integración Stripe simulada	4 semanas	Gestión de planes
Panel de administración	3 semanas	Autenticación (Admin)
Gestión de caja	4 semanas	Integración Stripe
Control de acceso mediante QR	2 semanas	Panel de admin
Dashboard y estadísticas	3 semanas	Panel de admin
Pruebas funcionales	4 semanas	Desarrollo completo
Ajustes responsive y visuales	2 semanas	Desarrollo completo
Corrección de errores	2 semanas	Pruebas funcionales
Documentación de la memoria	Transversal (9 meses)	-

7. Diseño de Interfaces

El diseño de interfaces de Iron Pulse se ha trabajado con el objetivo de crear una aplicación visualmente potente, moderna y coherente con el sector fitness. La identidad visual busca transmitir fuerza, energía, profesionalidad y confianza, manteniendo al mismo tiempo una navegación clara y funcional.

Durante el proceso de diseño se analizaron referencias visuales de gimnasios, plataformas deportivas, páginas de planes de suscripción y paneles de administración. A partir de esta investigación se definieron los colores principales, la tipografía, los componentes visuales, el estilo de botones, la iconografía y la estructura responsive de la aplicación.

Aunque no se desarrollaron mockups completos de alta fidelidad en una herramienta externa, el diseño final se fue construyendo directamente sobre la aplicación mediante vistas Blade, CSS, Tailwind y componentes reutilizables. Por este motivo, el resultado implementado actúa como diseño final de alta fidelidad.

7.1. Investigación inicial: benchmark, inspiración y moodboard

Antes de comenzar el desarrollo visual de Iron Pulse se realizó una investigación inicial basada en páginas web y aplicaciones relacionadas con el mundo del fitness. El objetivo era identificar patrones visuales habituales en este tipo de plataformas y adaptarlos a una propuesta propia.

Durante esta fase se analizaron principalmente:

- Páginas web de gimnasios.
- Secciones de planes y membresías.
- Interfaces de reservas de actividades.
- Paneles de administración.
- Botones de llamada a la acción.
- Tipografías deportivas.
- Iconografía relacionada con entrenamiento, fuerza, cardio y salud.
- Composiciones visuales en modo oscuro.

El moodboard recoge distintas referencias visuales utilizadas para definir la dirección estética del proyecto. En él aparecen ejemplos de botones, estilos tipográficos, paletas de colores, iconos deportivos y estructuras visuales de páginas de gimnasio.

La estética seleccionada se basa en un estilo oscuro y deportivo, con predominio del negro como color principal, blanco para contraste y rojo como color de acción. Esta combinación permite reforzar la personalidad de la marca y destacar elementos importantes como botones, precios, avisos y llamadas a la acción.

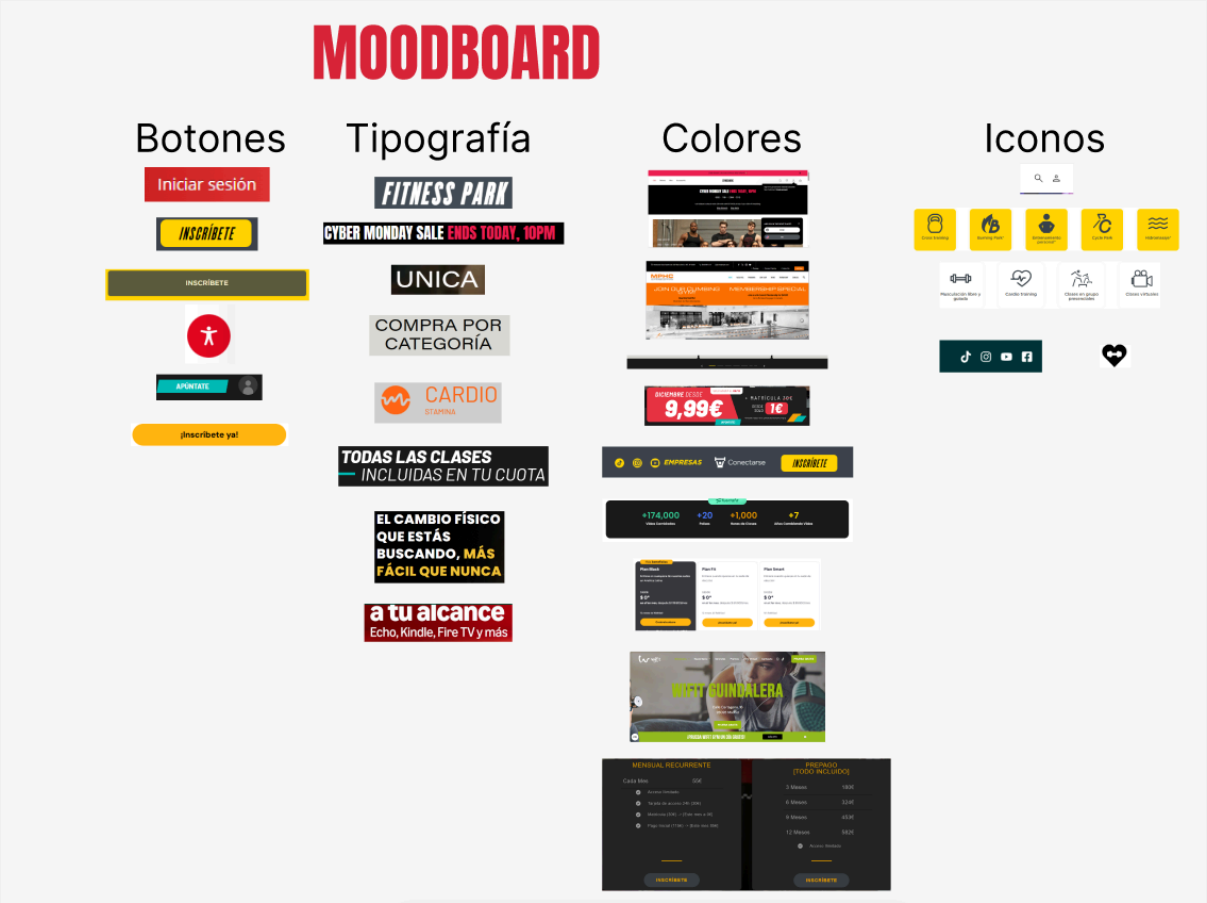


Imagen 22: *Moodboard inicial de Iron Pulse*

Fuente: elaboración propia.

En el moodboard se observan referencias de botones con colores intensos, tipografías condensadas y de alto impacto, secciones de precios, elementos visuales propios de gimnasios y diferentes tipos de iconografía deportiva. Estas referencias sirvieron como base para construir una interfaz con carácter propio, orientada al sector fitness.

7.2. Arquitectura de información

La arquitectura de información define cómo se organizan las secciones de la aplicación y cómo navega el usuario por ellas. En Iron Pulse se ha planteado una estructura diferenciada entre la parte pública, la parte privada del usuario y la zona de administración.

Esta división permite que cada tipo de usuario acceda únicamente a las funcionalidades que necesita.

7.2.1. Zona pública

La zona pública está pensada para visitantes que todavía no han iniciado sesión. Su objetivo es presentar el gimnasio, mostrar sus servicios y facilitar el registro o inicio de sesión.

Incluye las siguientes secciones:

- Página principal.
- Información del gimnasio.
- Sobre nosotros.
- Planes disponibles.
- Clases o actividades.
- Contacto.
- Registro.
- Inicio de sesión.

La página principal actúa como punto de entrada y resume la identidad de Iron Pulse, mostrando llamadas a la acción para registrarse, consultar planes o acceder al sistema.

7.2.2. Zona privada de usuario

La zona privada está dirigida a usuarios registrados. Desde esta zona, el usuario puede acceder a sus funcionalidades personales.

Incluye:

- Dashboard del usuario.
- Perfil personal.
- Plan o membresía activa.
- Rutinas de entrenamiento.
- Calendario de clases.
- Reservas realizadas.
- Código QR de acceso.
- Pago simulado de planes mediante Stripe.

Esta organización busca que el usuario pueda encontrar rápidamente las acciones principales: consultar rutinas, reservar clases, revisar su plan y acceder al gimnasio mediante QR.

7.2.3. Zona de administración

La zona de administración está dirigida al personal autorizado del gimnasio. Desde ella se gestionan los elementos internos de la aplicación.

Incluye:

- Panel de control.
- Gestión de usuarios.
- Gestión de planes.
- Gestión de rutinas.
- Gestión de clases.
- Gestión de reservas.
- Escáner QR.
- Control de entradas y salidas.
- Gestión de caja.
- Informes o cierres de caja.
- Estadísticas generales.

El panel de administración se ha diseñado para centralizar las tareas internas del gimnasio, reduciendo la necesidad de utilizar herramientas externas para gestionar socios, pagos, clases o accesos.

7.3. Mapa de navegación (literalmente rehacer)

El mapa de navegación representa la estructura general de la aplicación y la relación entre sus páginas principales.

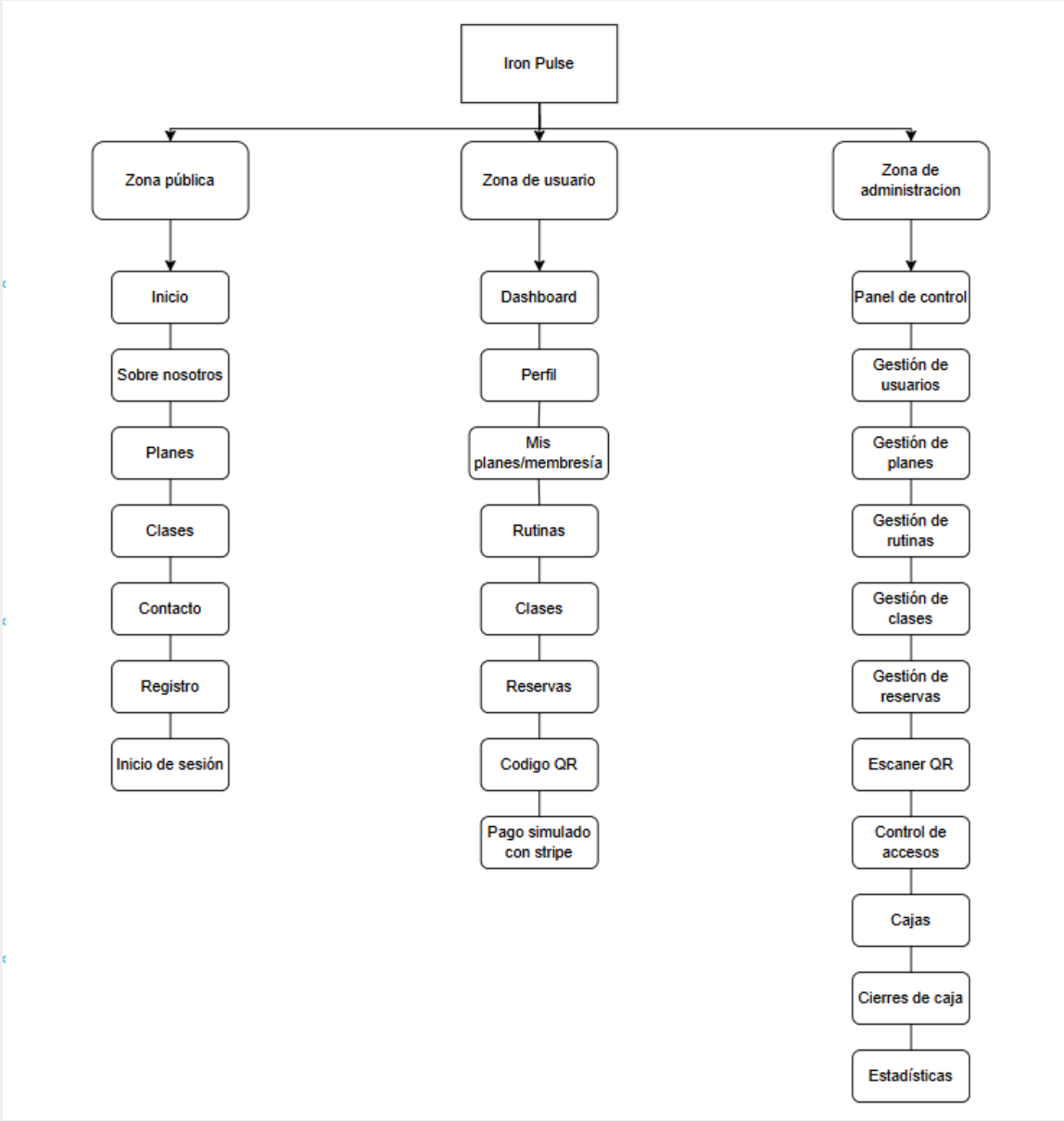


Imagen 23: *Mapa de navegación de Iron Pulse*

Este mapa muestra que la aplicación se organiza en tres grandes bloques: parte pública, área privada de usuario y administración. Esta estructura facilita la navegación y permite separar claramente las funcionalidades según el tipo de usuario.

7.4. Wireframes

Los wireframes son representaciones iniciales de la estructura de las pantallas. En el caso de Iron Pulse, se utilizaron como apoyo para definir la distribución general de los elementos antes de implementar las vistas finales.

El objetivo de los wireframes fue establecer:

- Ubicación de la barra de navegación.
- Distribución de secciones principales.
- Organización de tarjetas y bloques de contenido.
- Estructura de formularios.
- Disposición de botones de acción.
- Jerarquía visual de títulos, textos e imágenes.
- Adaptación de las pantallas a diferentes tamaños.

Los wireframes sirvieron como guía inicial, aunque posteriormente algunas pantallas fueron ajustándose durante la implementación real para mejorar la experiencia de usuario y adaptarse mejor al contenido definitivo.

7.4.1. Pantallas principales previstas

Los wireframes o estructuras iniciales contemplan las siguientes pantallas:

Pantalla	Objetivo
Página principal	Presentar Iron Pulse y dirigir al usuario hacia planes, registro o login.
Registro	Permitir la creación de una cuenta de usuario.
Inicio de sesión	Permitir el acceso de usuarios registrados.
Planes	Mostrar los planes disponibles, precios y características.

Rutinas	Mostrar rutinas de entrenamiento disponibles para el usuario.
Clases	Mostrar el calendario de actividades y permitir reservas.
Perfil	Permitir la gestión de datos personales.
QR	Mostrar el código QR asociado al usuario.
Panel admin	Centralizar las tareas administrativas.
Caja	Registrar movimientos económicos y cierres de caja.
Escáner QR	Leer códigos QR y registrar entradas o salidas.

7.4.2. Capturas de bocetos o wireframes

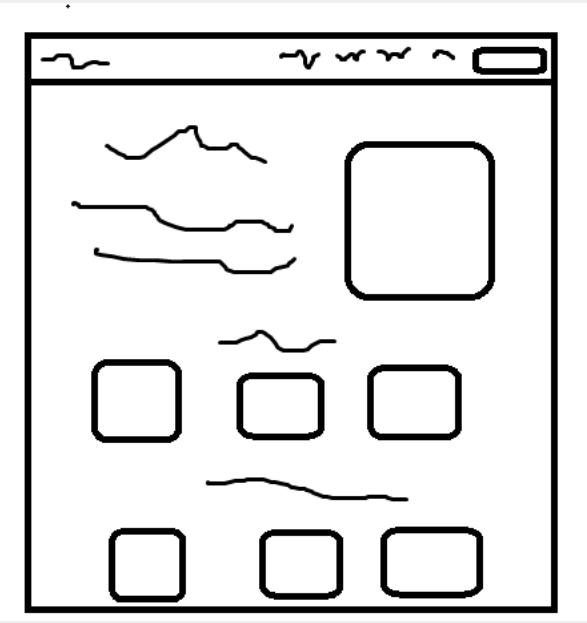


Imagen 24: *Boceto del HOME, hecho en paint*

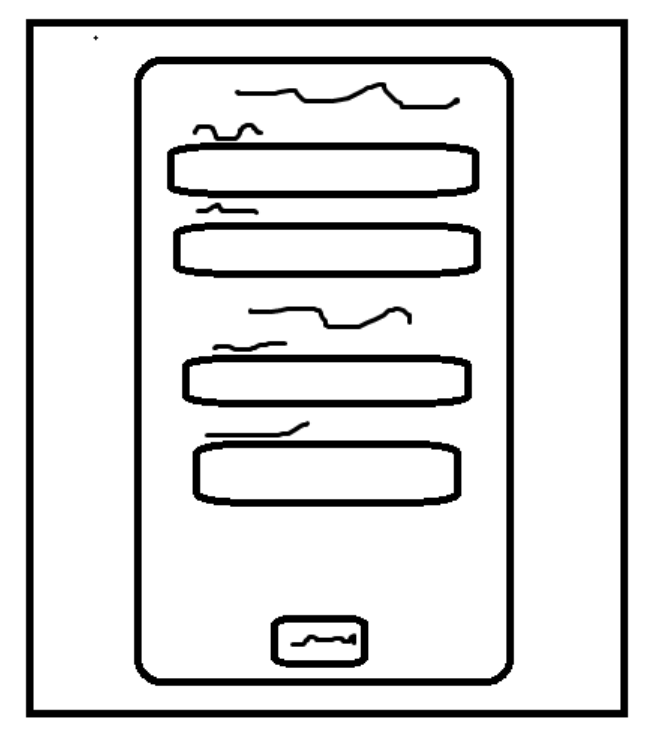


Imagen 25: *Boceto del Inicio de Sesión, hecho en paint*

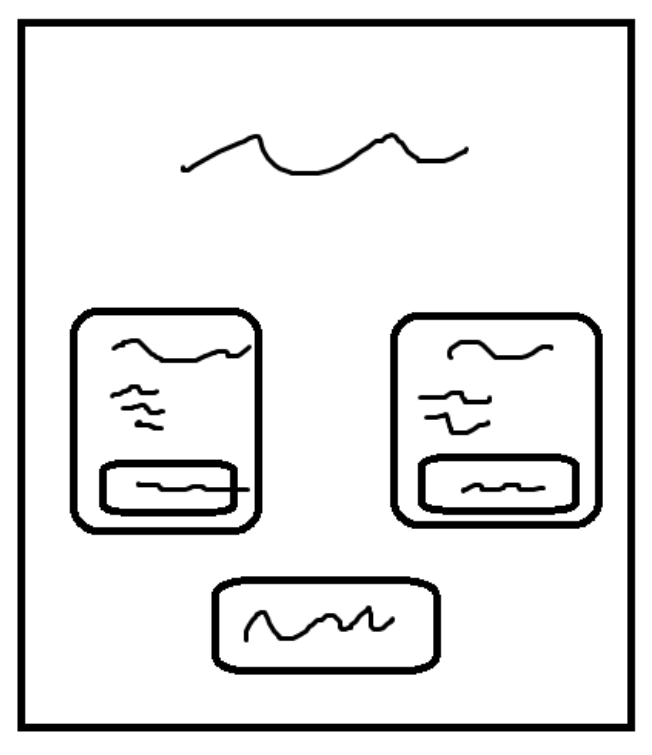


Imagen 26: *Boceto de los Planes, hecho en paint*

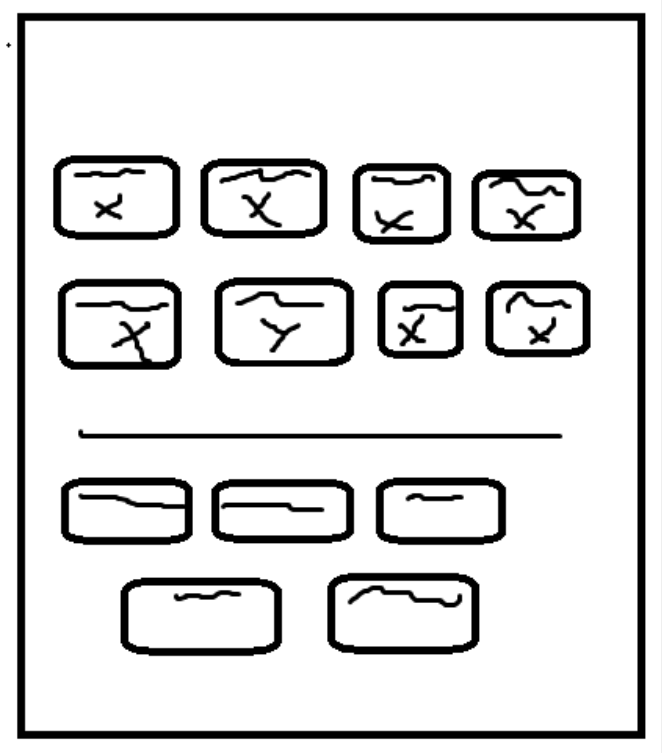


Imagen 27: *Boceto del panel de administrador, hecho en paint*

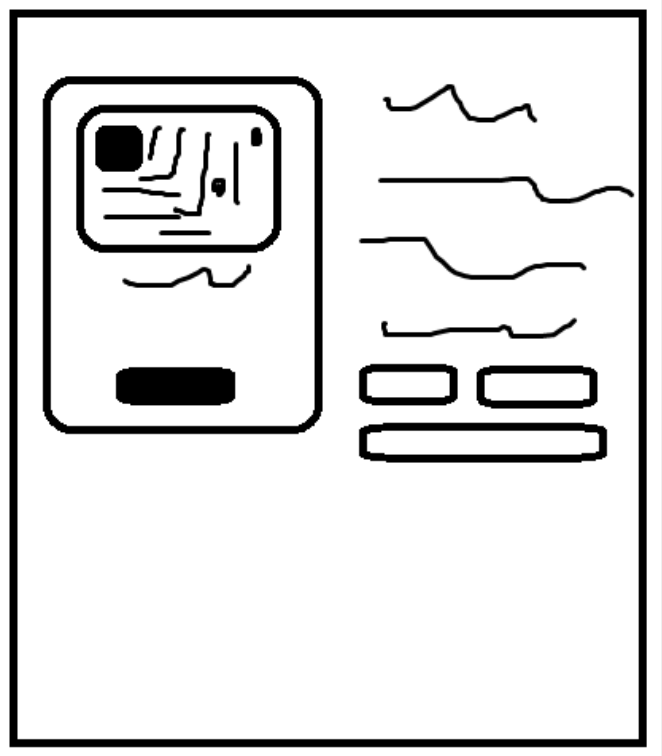


Imagen 28: *Boceto del Panel de usuario con QR, hecho en paint*

7.5. Mockups de alta fidelidad

Para el diseño de Iron Pulse se realizaron mockups de alta fidelidad mediante Figma, con el objetivo de definir de forma visual el aspecto final de la aplicación antes de trasladarlo al desarrollo. Estos mockups permitieron establecer la composición de las pantallas principales, la jerarquía visual, el uso de la paleta de colores, la tipografía, los botones, las tarjetas y la organización general de la interfaz.

Los mockups de alta fidelidad sirvieron como referencia directa para la implementación posterior en Laravel, Blade, Tailwind CSS, CSS personalizado y JavaScript. Gracias a este trabajo previo, el equipo pudo mantener una línea visual coherente en todas las secciones de la aplicación y reducir cambios importantes durante la fase de desarrollo.

El diseño realizado en Figma se centró especialmente en conseguir una estética deportiva, moderna y oscura, alineada con la identidad de Iron Pulse. Se trabajaron elementos como los contrastes entre negro, blanco y rojo, el uso de tipografías de impacto para títulos, la distribución mediante grid y la creación de componentes reutilizables para botones, tarjetas, menús y secciones de contenido.

Los mockups también ayudaron a validar decisiones de diseño antes de la programación, como la disposición de la página principal, la presentación de planes, la estructura del panel de administración y la organización visual de secciones importantes como clases, rutinas, caja y acceso mediante QR.

Para documentar este apartado se incluirán capturas de los diseños realizados en Figma y, si se considera necesario, una comparación con las pantallas finales implementadas en la aplicación.



Imagen 29: *Mockup del HOME*



Imagen 30: *Mockup del apartado de PLANES*

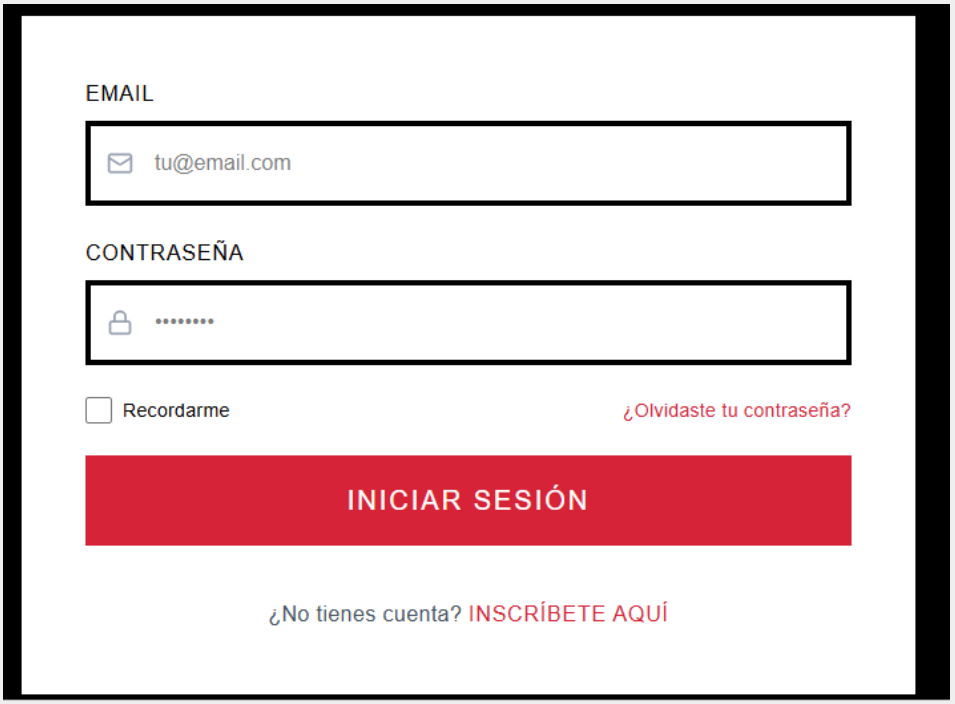


Imagen 31: *Mockup del apartado de Iniciar Sesión*



Imagen 32: *Mockup del apartado de Clases*

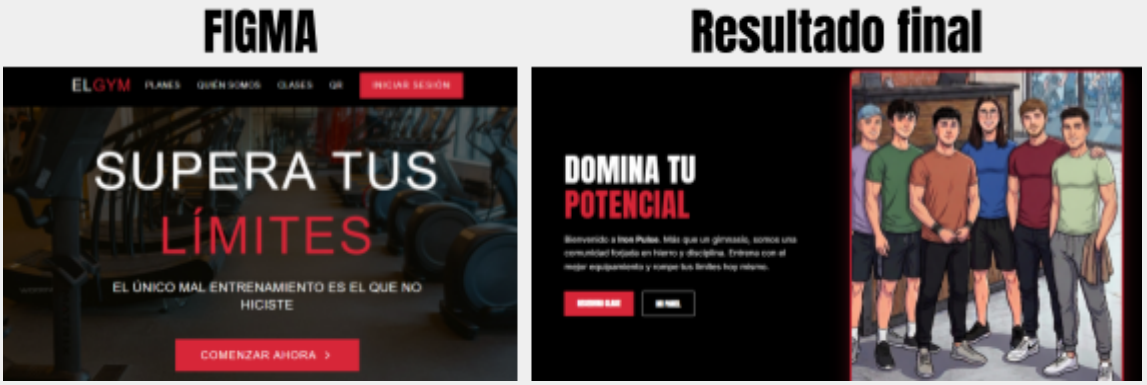


Imagen 33: *Comparativa Home planeado a Home Final*

Los mockups de alta fidelidad realizados en Figma representan una aproximación visual muy cercana al resultado final de la aplicación. En ellos se definieron los colores, tipografías, distribución de secciones, componentes principales y estilo general de Iron Pulse. Estos diseños sirvieron como guía durante el desarrollo frontend, permitiendo trasladar la propuesta visual a vistas reales construidas con Blade, Tailwind CSS, CSS personalizado y JavaScript.

7.6. Guía de estilo web

La guía de estilo define las normas visuales que se han seguido para mantener coherencia en toda la aplicación. En Iron Pulse se ha trabajado una identidad visual fuerte, basada en colores intensos, tipografía de impacto y componentes visuales simples pero reconocibles.

7.6.1. Paleta de colores

La paleta de colores de Iron Pulse se basa en tres colores principales:

Color	Código HEX	Uso principal
Negro	**#000000**	Color principal de fondo, cabeceras, paneles y secciones oscuras.
Blanco	**#FFFFFF**	Textos principales, fondos secundarios y contraste visual.
Rojo	**#D72638**	Botones, llamadas a la acción, elementos destacados y acentos visuales.

La distribución visual de la paleta se plantea aproximadamente mediante la regla **60/30/10**:

- **60% negro:** utilizado como base visual principal. Transmite fuerza, seriedad y estilo premium.
- **30% blanco:** utilizado para textos, espacios de descanso visual y contraste.
- **10% rojo:** utilizado para destacar acciones importantes, botones, precios, alertas y elementos clave.

Esta combinación permite crear una identidad visual coherente con el sector fitness, manteniendo un equilibrio entre impacto visual y legibilidad.

7.6.2. Justificación de los colores

Negro — **#000000**

El negro se utiliza como color dominante porque transmite fuerza, disciplina, elegancia y profesionalidad. Es habitual en marcas deportivas y gimnasios que buscan una imagen intensa y premium.

En Iron Pulse se aplica en fondos principales, barras de navegación, tarjetas oscuras, paneles y secciones destacadas.

Blanco — **#FFFFFF**

El blanco se utiliza para aportar claridad, contraste y equilibrio. Permite que los textos sean legibles sobre fondos oscuros y ayuda a evitar que la interfaz resulte demasiado cargada.

Se utiliza principalmente en textos, zonas de contenido, formularios y elementos secundarios.

Rojo — **#D72638**

El rojo funciona como color de acción. Se asocia con energía, intensidad, esfuerzo y motivación, conceptos muy ligados al mundo del entrenamiento.

En la aplicación se emplea en botones, enlaces destacados, precios, estados importantes y llamadas a la acción como “Iniciar sesión”, “Inscribirse” o “Reservar”.



Imagen 34: *Guía de estilo*

7.6.3. Tipografías

La identidad tipográfica de Iron Pulse combina una fuente de alto impacto para títulos con una fuente más limpia y legible para textos generales.

Tipografía	Uso
Anton	Títulos principales, secciones destacadas, precios y llamadas visuales de alto impacto.
Inter	Textos generales, formularios, tablas, menús, descripciones y contenido administrativo.

Anton

La tipografía Anton se utiliza para transmitir fuerza y carácter deportivo. Su forma condensada y gruesa la hace adecuada para titulares, nombres de secciones, banners y elementos promocionales.

Ejemplos de uso:

- Títulos principales.
- Nombres de planes.
- Frases destacadas.

- Secciones promocionales.
- Encabezados de gran tamaño.

Inter

Inter se utiliza como tipografía secundaria para garantizar buena lectura en textos largos, formularios, menús, tablas y paneles de administración.

Es una fuente moderna, limpia y muy legible en pantalla, por lo que resulta adecuada para interfaces web y aplicaciones con mucho contenido funcional.

Ejemplos de uso:

- Textos descriptivos.
- Formularios.
- Botones secundarios.
- Tablas.
- Menús.
- Panel de administración.
- Información de usuario.

7.6.4. Tamaños de fuente (revisar)

Para mantener una jerarquía visual clara, se han definido tamaños aproximados según el tipo de contenido:

Elemento	Tamaño aproximado
Título principal h1	32px
Título de sección h2	26px
Subtítulo h3	22px
Texto general	16px
Texto pequeño	12px - 14px
Botones	14px - 16px

Esta jerarquía permite diferenciar correctamente los niveles de información y facilita la lectura.

7.6.5. Componentes UI

La interfaz de Iron Pulse se compone de diferentes elementos reutilizables. Estos componentes mantienen una estética coherente en todas las pantallas.

Botones

Los botones son uno de los elementos más importantes de la interfaz, ya que guían al usuario hacia acciones concretas.

Tipos de botones utilizados:

Tipo de botón	Uso
Botón principal	Acciones importantes como registrarse, reservar, pagar o confirmar.
Botón secundario	Acciones alternativas como volver, cancelar o editar.
Botón de peligro	Acciones destructivas como eliminar o cancelar reserva.
Botón administrativo	Acciones internas del panel admin.

Los botones principales utilizan normalmente fondo rojo, texto blanco y bordes definidos para destacar sobre el fondo oscuro.

Ejemplos:

- Iniciar sesión.
- Inscríbete.
- Reservar clase.
- Pagar plan.
- Confirmar.
- Escanear QR.

Formularios

Los formularios se utilizan en procesos como registro, login, edición de perfil, creación de rutinas, creación de clases, gestión de usuarios y caja.

Características:

- Campos claramente etiquetados.
- Separación visual entre bloques.
- Botones de acción visibles.
- Mensajes de error comprensibles.
- Validación de datos.
- Diseño adaptado a móvil y escritorio.

Tarjetas

Las tarjetas permiten presentar información agrupada de forma clara. Se utilizan especialmente en:

- Planes de suscripción.
- Rutinas.
- Clases.
- Estadísticas.
- Usuarios.
- Movimientos de caja.
- Panel de administración.

Las tarjetas suelen incluir título, descripción, icono, dato destacado y botón de acción.

Menús

La navegación se organiza mediante menús diferenciados según el tipo de usuario:

- Menú público para visitantes.
- Menú privado para usuarios registrados.
- Menú de administración para personal autorizado.

El objetivo es que cada usuario vea solo las secciones relevantes para su rol.

Iconos (poner los de lucid icons)

La iconografía se utiliza para mejorar la comprensión visual de las acciones y secciones. Los iconos ayudan a identificar rápidamente elementos como usuarios, rutinas, pesas, calendario, caja, QR o estadísticas.

Se ha buscado una iconografía simple, lineal y coherente, evitando mezclar estilos distintos.

7.6.6. Sistema de grid y responsive design

El diseño se apoya en un sistema de rejilla para organizar correctamente el contenido en diferentes resoluciones. En el moodboard se contemplan referencias para pantallas verticales y horizontales, teniendo en cuenta medidas aproximadas como:

- Pantallas verticales: **90 x 768.**
- Pantallas horizontales: **1353 x 120.**

Aunque estas medidas se usan como referencia visual en el diseño, la aplicación está pensada para adaptarse a diferentes dispositivos mediante CSS responsive, Flexbox, Grid y utilidades de Tailwind.

El sistema responsive permite que los elementos se reorganicen según el tamaño de pantalla:

- En escritorio, las secciones pueden mostrarse en varias columnas.
- En tablet, se reduce la cantidad de columnas y se ajustan márgenes.
- En móvil, los elementos se apilan verticalmente para mejorar la lectura.

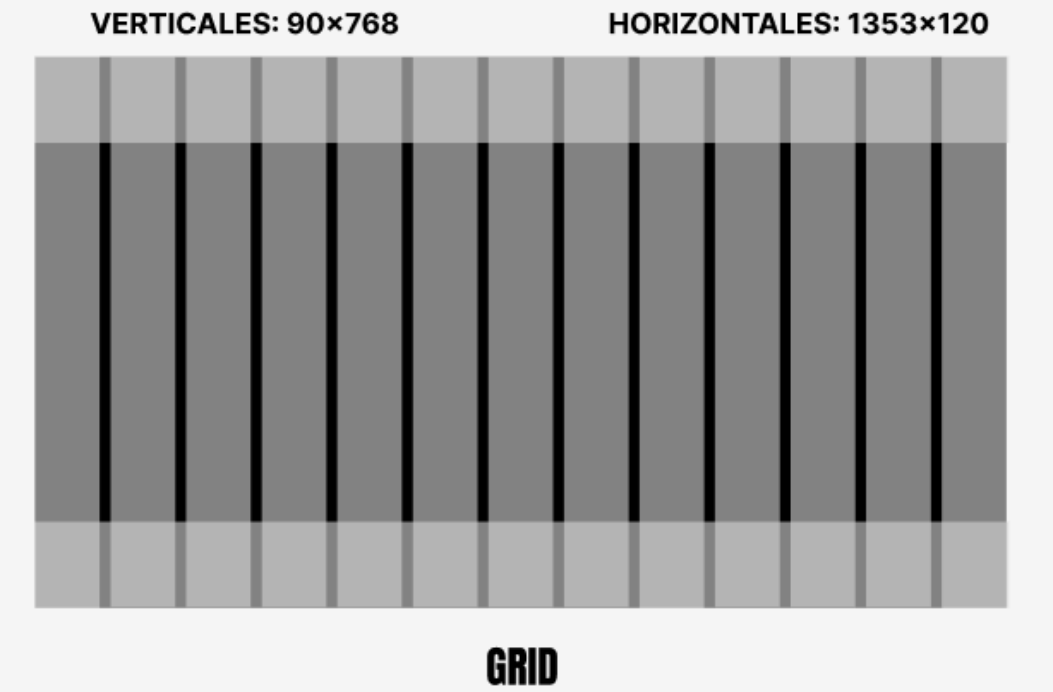


Imagen 35: *Grid utilizado en el proyecto*

7.6.7. Accesibilidad aplicada al diseño

Durante el diseño de la interfaz se han tenido en cuenta principios básicos de accesibilidad para mejorar la experiencia de uso.

Medidas aplicadas:

- Contraste alto entre fondo y texto.
- Tamaños de fuente legibles.
- Botones con suficiente tamaño para ser pulsados cómodamente.
- Formularios con etiquetas claras.
- Uso de iconos acompañados de texto cuando la acción puede no ser evidente.
- Jerarquía visual clara mediante títulos, subtítulos y secciones.
- Adaptación a diferentes tamaños de pantalla.

Estas medidas permiten que la aplicación sea más cómoda de utilizar y reducen errores durante procesos importantes como registro, reserva de clases, pago simulado o control de acceso mediante QR.

9. Desarrollo Frontend

9.1. Estructura HTML semántica

El desarrollo frontend de Iron Pulse se ha realizado mediante vistas Blade dentro de Laravel. La interfaz se organiza a través de plantillas reutilizables, componentes Blade, HTML5, CSS personalizado, Tailwind CSS, JavaScript, Alpine.js y diferentes librerías externas para funcionalidades concretas como el calendario, las alertas visuales, Stripe y el escáner QR.

La estructura principal de la aplicación se encuentra en **resources/views/layouts/app.blade.php.** Este layout define la base común de las páginas internas de Iron Pulse, incluyendo la configuración del documento HTML, la carga de estilos y scripts mediante Vite, la barra de navegación, el contenido principal y el footer general.

La estructura base del layout es la siguiente:



```blade
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <meta name="csrf-token" content="{{ csrf_token() }}">
        <title>{{ config('app.name', 'Iron Pulse') }}</title>
        @vite(['resources/css/app.css', 'resources/js/app.js'])
    </head>

    <body class="antialiased bg-principal text-secundario min-h-screen flex flex-col">
        @include('layouts.navigation')

        @isset($header)
            <header>
                {{ $header }}
            </header>
        @endisset

        <main class="container main-content">
            {{ $slot }}
        </main>

        <footer>
            <!-- Footer general -->
        </footer>
    </body>
</html>
```

Imagen 36: *Estructura base del layout*

Esta estructura permite que todas las vistas que utilizan **<x-app-layout>** compartan la misma navegación, el mismo contenedor principal y el mismo pie de página. De esta forma se evita repetir código y se mantiene una experiencia visual coherente en toda la aplicación.

Las vistas se encuentran organizadas por carpetas dentro de **resources/views**, separando las diferentes zonas funcionales del proyecto:

Carpeta / vista	Función principal
welcome.blade.php	Página principal pública de Iron Pulse.
dashboard.blade.php	Panel principal del usuario tras iniciar sesión.
auth/	Vistas de autenticación: login, registro, recuperación y verificación.

profile/	Edición de datos personales, contraseña y eliminación de cuenta.
plans/	Visualización de planes y resultado de suscripción.
stripe/	Vistas relacionadas con el flujo de pago simulado mediante Stripe.
routines/	Creación, edición, listado y detalle de rutinas.
events/	Calendario de clases, reservas y clases del usuario.
admin/	Panel de administración, usuarios, planes, clases, escáner y accesos.
admin/cash/	Gestión de caja, movimientos, cierres e informes.
components/	Componentes reutilizables de Blade.
layouts/	Plantillas base y navegación.

El proyecto utiliza etiquetas semánticas como **header, main, section, footer, form, label, input, button** y encabezados jerárquicos. En algunas vistas también se emplean **div** con estilos personalizados para construir tarjetas, dashboards y paneles visuales. Esta combinación permite mantener una estructura clara y adaptar el diseño a las necesidades visuales del proyecto.

Los formularios están presentes en varias partes de la aplicación: registro, login, perfil, creación de usuarios, planes, rutinas, clases y caja. En ellos se utilizan campos de entrada, etiquetas, validaciones y protección CSRF propia de Laravel.

Ejemplos de formularios del frontend:

- Formulario de registro de usuario.
- Formulario de inicio de sesión.
- Formulario de edición de perfil.
- Formulario de creación de rutinas.
- Formulario de creación de clases.
- Formulario de asignación de planes.
- Formulario de movimientos de caja.
- Formulario de pago simulado con Stripe.

9.2. Estilos CSS

La parte visual de Iron Pulse se construye mediante una combinación de Tailwind CSS, clases personalizadas, estilos propios en **resources/css/app.css** y estilos específicos dentro de algunas vistas Blade. Esta combinación permite aprovechar la rapidez de Tailwind y, al mismo tiempo, mantener una identidad visual personalizada para el gimnasio.

El archivo principal de estilos es **resources/css/app.css**. En él se definen las fuentes, variables globales, colores, tipografías, botones, layout, navegación, responsive, calendario, tablas, formularios y otros elementos reutilizables.

Variables globales

El proyecto define variables CSS en **:root** para mantener coherencia visual en toda la aplicación:

```
:root {
    /* Paleta de Colores */
    --color-principal: #000000; /* 60% - Fuerza, seriedad, elegancia */
    --color-secundario: #ffffff; /* 30% - Limpieza, elegancia */
    --color-terciario: #d72638; /* 10% - Importancia, pasión, energía (Acento) */

    /* Tipografías */
    --font-heading: "Anton", sans-serif;
    --font-body: "Inter", sans-serif;

    /* Tamaños de Fuente */
    --text-h1: 32px;
    --text-h2: 26px; /* Ajustado al valor de tu imagen (26px) */
    --text-h3: 20px; /* Ajustado al valor de tu imagen (22px) -> En la imagen dice 22px, ajustado aquí */
    --text-p: 12px;

    /* Espaciados y Grid (Referencia básica) */
    --gap-standard: 1rem;
    --container-max-width: 1353px; /* Basado en tu grilla horizontal */
}
```

Imagen 37: *Captura de una parte del CSS*

Estas variables permiten reutilizar colores, fuentes y tamaños de texto sin repetir valores manualmente. La paleta mantiene el estilo definido en la guía visual: negro como color principal, blanco como color secundario y rojo como color de acento.

Tipografía

El proyecto importa las fuentes Anton e Inter desde Google Fonts:

```
4   @import url("https://fonts.googleapis.com/css2?family=Anton&family=Inter:wght@400;500;700&display=swap");
```

Imagen 38: *Ejemplo de importación de las fuentes del proyecto*

Anton se utiliza en títulos, encabezados y elementos de impacto visual. Inter se utiliza en textos generales, formularios, paneles y contenido administrativo.

Botones

El CSS define botones reutilizables como **.btn-primary, .btn-secondary, .btn-danger, .btn-add y .btn-add-day.**

Clase	Uso
.btn-primary	Botón principal rojo, utilizado en acciones importantes.
.btn-secondary	Botón secundario con borde blanco.
.btn-danger	Botón para acciones destructivas o de advertencia.
.btn-add	Botón circular para añadir elementos.
.btn-add-day	Botón circular usado en acciones relacionadas con días o planificación.

Estos botones utilizan transiciones y efectos **hover** para mejorar la interacción visual.

Layout, Flexbox y Grid

Iron Pulse utiliza Flexbox y CSS Grid para organizar la interfaz. En **app.css** se definen clases como **.container, .main-content, .flex, .flex-col, .flex-grow, .flex-center** y **.grid.**

La clase **.grid** define una rejilla de 12 columnas inspirada en la guía de estilo:

```
214  /* Grilla básica de 12 columnas para replicar la de Figma */
215  .grid {
216      display: grid;
217      grid-template-columns: repeat(12, 1fr);
218      gap: 20px; /* Separación entre columnas */
219  }
```

Imagen 39: *Demostración de la utilización del grid en el proyecto*

Además, muchas vistas utilizan estilos propios con **display: grid, display: flex** y **grid-template-columns** para organizar secciones como la home, las tarjetas de planes, el dashboard, la caja y el panel de administración.

Tablas, formularios y calendario

El CSS también incluye estilos específicos para:

- Tablas administrativas mediante la clase **.table**.
- Campos de formulario mediante la clase **.input.**
- Grid de ejercicios mediante **.exercise-grid**.
- Calendario FullCalendar mediante clases .**fc, .fc-toolbar-title, .fc-button-primary, .fc-event**, etc.

Esto permite adaptar visualmente librerías externas y elementos funcionales al estilo oscuro de Iron Pulse.

Uso de estilos inline

Además de las clases CSS, varias vistas Blade utilizan estilos inline para ajustar elementos concretos de la interfaz. Esto se observa en pantallas como **welcome.blade.php, dashboard.blade.php, plans/index.blade.php, events/index.blade.php** y vistas de administración.

Aunque estos estilos permiten ajustar rápidamente el diseño, como mejora futura se podría refactorizar parte de ellos a clases reutilizables para mejorar la mantenibilidad.

9.3. Interactividad con JavaScript

La interactividad del frontend se implementa mediante JavaScript, Alpine.js, Axios, Fetch API y librerías externas cargadas en determinadas vistas.

El archivo **resources/js/app.js** importa **bootstrap.js** y activa Alpine.js:

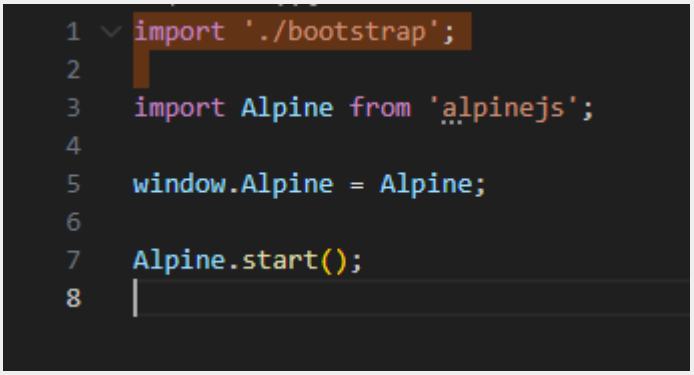


Imagen 40: *captura del apartado en el archivo app.js*

El archivo **resources/js/bootstrap.js** configura Axios para peticiones HTTP asíncronas:



Imagen 41: *captura del apartado en el archivo bootstrap.js*

Alpine.js

Alpine.js se utiliza principalmente en componentes interactivos de la interfaz, especialmente en la navegación. Por ejemplo, el menú de usuario desplegable utiliza atributos como **x-data, x-show** y **x-transition** para mostrar y ocultar opciones sin recargar la página.

FullCalendar

FullCalendar se utiliza para la visualización y gestión de clases. Aparece tanto en la zona de usuario como en la zona de administración.

En la vista de usuario **events/index.blade.php**, el calendario carga los eventos desde atributos **data-*** y utiliza el archivo **public/js/calendar-client.js**. Este script permite:

- Mostrar clases en formato calendario.
- Diferenciar clases disponibles, llenas, pasadas o ya reservadas.
- Buscar clases por nombre.
- Filtrar para mostrar solo las clases reservadas por el usuario.
- Reservar o cancelar clases mediante confirmaciones visuales.

En la zona de administración, la vista **admin/events/index.blade.php** permite gestionar clases, seleccionar días del calendario, autocompletar fechas en el formulario y eliminar clases existentes.

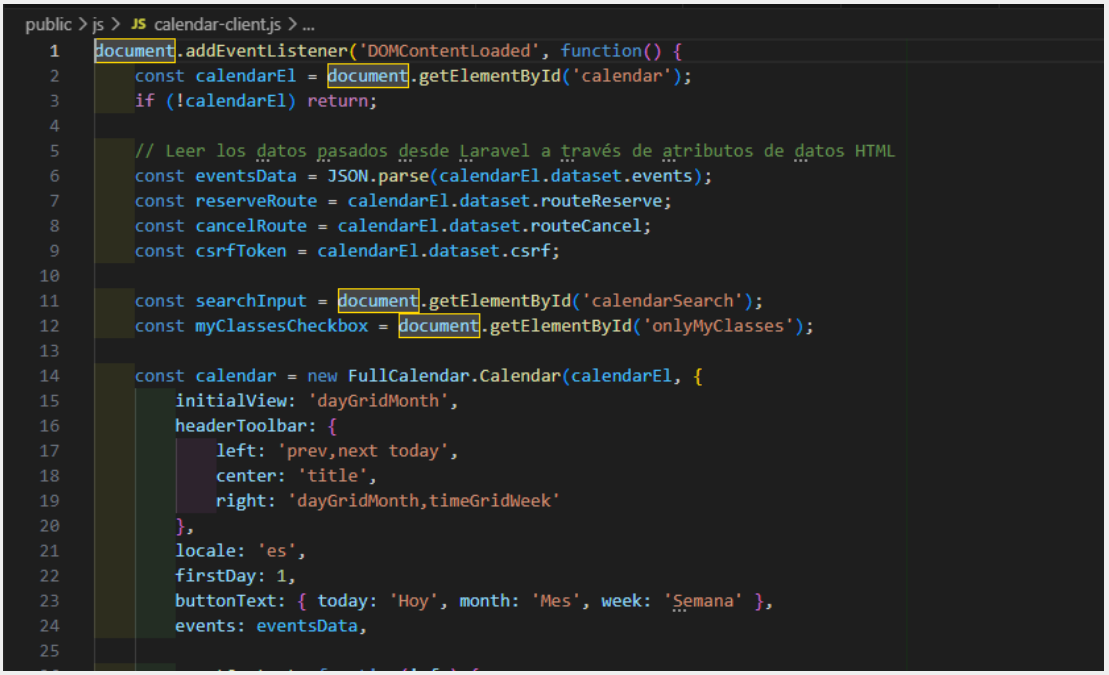


Imagen 42: *Imagen de parte del calendario*

SweetAlert2

SweetAlert2 se utiliza para mejorar la comunicación con el usuario mediante ventanas emergentes personalizadas. Se emplea en acciones como:

- Confirmar reservas de clase.
- Cancelar reservas.
- Avisar de clases llenas o finalizadas.

- Confirmar cancelación de plan.
- Mostrar mensajes de éxito.

Esto sustituye parcialmente los avisos básicos del navegador por una experiencia visual más coherente con el diseño oscuro de Iron Pulse.

Html5-QRCode y escáner de accesos

La vista **admin/scanner.blade.php** utiliza la librería Html5-QRCode para acceder a la cámara del dispositivo y leer códigos QR. Cuando se detecta un QR, el frontend envía una petición **fetch()** al servidor mediante la ruta **admin.scanner.process.**

El flujo del escáner es el siguiente:

1. Se inicia la cámara al cargar la pantalla.
2. La librería detecta y decodifica el código QR.
3. El escáner se detiene para evitar lecturas repetidas.
4. Se envía el contenido del QR al backend mediante **fetch()**.
5. El servidor devuelve una respuesta JSON.
6. La vista muestra el resultado con un color, título y mensaje.
7. El administrador puede reiniciar el escáner para leer otro código.

Stripe y pago simulado

Las vistas de Stripe utilizan JavaScript para iniciar el flujo de pago simulado. En **stripe/plan-register.blade.php** se carga Stripe mediante:

```
<x-app-layout>
    <script src="https://js.stripe.com/v3/"></script>
```

Imagen 43: *Importación de stripe en el proyecto*

Después, mediante **fetch()**, la vista solicita al backend la creación de una sesión de checkout y redirige al usuario a Stripe con **stripe.redirectToCheckout().**

Este flujo permite simular el proceso de contratación de un plan o membresía.

9.4. Diseño responsive

Iron Pulse se ha desarrollado con un enfoque responsive para que pueda utilizarse correctamente en ordenadores, tablets y dispositivos móviles. Esto es importante

porque muchos usuarios accederán desde el móvil para consultar planes, reservar clases o mostrar el QR de acceso al gimnasio.

El responsive se consigue mediante:

- Clases responsive de Tailwind CSS.
- Media queries en **resources/css/app.css.**
- Flexbox y CSS Grid.
- Menú móvil adaptado.
- Reducción de tamaños de texto en pantallas pequeñas.
- Formularios y botones adaptados al ancho disponible.
- Tablas con scroll horizontal.
- Ajustes específicos para FullCalendar.

El archivo **app.css** incluye dos bloques principales de responsive:

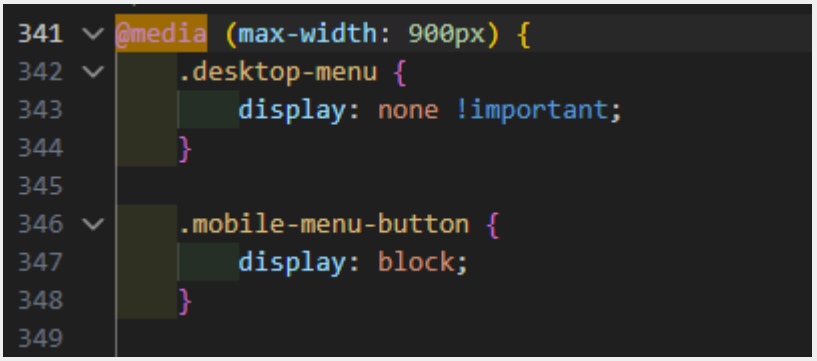


Imagen 44: *Demostración de diseño responsive en el proyecto*

En pantallas pequeñas se reduce el tamaño de títulos, se ajustan los contenedores, los botones pasan a ocupar más anchura, los grids se convierten en una sola columna y las tablas permiten desplazamiento horizontal. También se adapta el calendario para hacerlo más usable en móvil.

Además, la navegación dispone de clases como **.desktop-menu, .mobile-menu-button** y **.mobile-menu-panel**, que permiten mostrar un menú de escritorio o un menú móvil según el tamaño de pantalla.

Adaptación por dispositivo

Dispositivo	Adaptación aplicada
Escritorio	Diseño amplio, uso de varias columnas, dashboards completos, tablas y calendarios visibles.
Tablet	Reducción de columnas y adaptación de tarjetas, formularios y navegación.
Móvil	Elementos apilados, menú móvil, botones más grandes, tablas con scroll y calendario simplificado.

Las pantallas más importantes para móvil son el login, el registro, los planes, el calendario de clases, las reservas y el QR de acceso, ya que son las más utilizadas por un usuario del gimnasio desde su teléfono.

9.5. Accesibilidad aplicada

Durante el desarrollo frontend de Iron Pulse se han aplicado buenas prácticas básicas de accesibilidad para facilitar el uso de la aplicación. La interfaz utiliza alto contraste, textos claros, botones visibles, formularios etiquetados y una navegación diferenciada según el tipo de usuario.

Medida	Aplicación en Iron Pulse
Contraste visual	Fondo negro, texto blanco y rojo como color de acento.
Etiquetas en formularios	Uso de **label** en formularios de registro, login, clases, perfil, caja y Stripe.

Textos alternativos	Uso de **alt** en imágenes como el logo, la imagen principal y el QR.
Jerarquía visual	Uso de títulos destacados con Anton y subtítulos/textos con Inter.
Botones identificables	Botones con textos claros como “Reservar Clase”, “Ver Planes”, “Pagar con Stripe” o “Escanear siguiente”.
Navegación por roles	Separación entre usuario normal y administrador.
Mensajes de estado	Alertas visuales para confirmaciones, errores, clases llenas o acciones completadas.
Responsive	Adaptación a dispositivos móviles para mejorar la usabilidad táctil.

Formularios accesibles

Los formularios utilizan etiquetas descriptivas y campos organizados de forma clara. Esto permite que el usuario entienda qué información debe introducir y reduce errores durante procesos como registro, edición de perfil, creación de clases, pago simulado o gestión de caja.

Contraste y legibilidad (poner captura del WAVE)

La combinación de negro, blanco y rojo permite mantener una lectura clara. El rojo se reserva para acciones importantes y elementos destacados, evitando saturar la interfaz. Los textos principales se muestran en blanco o gris claro sobre fondos oscuros, manteniendo una buena separación visual.

Mejoras futuras de accesibilidad

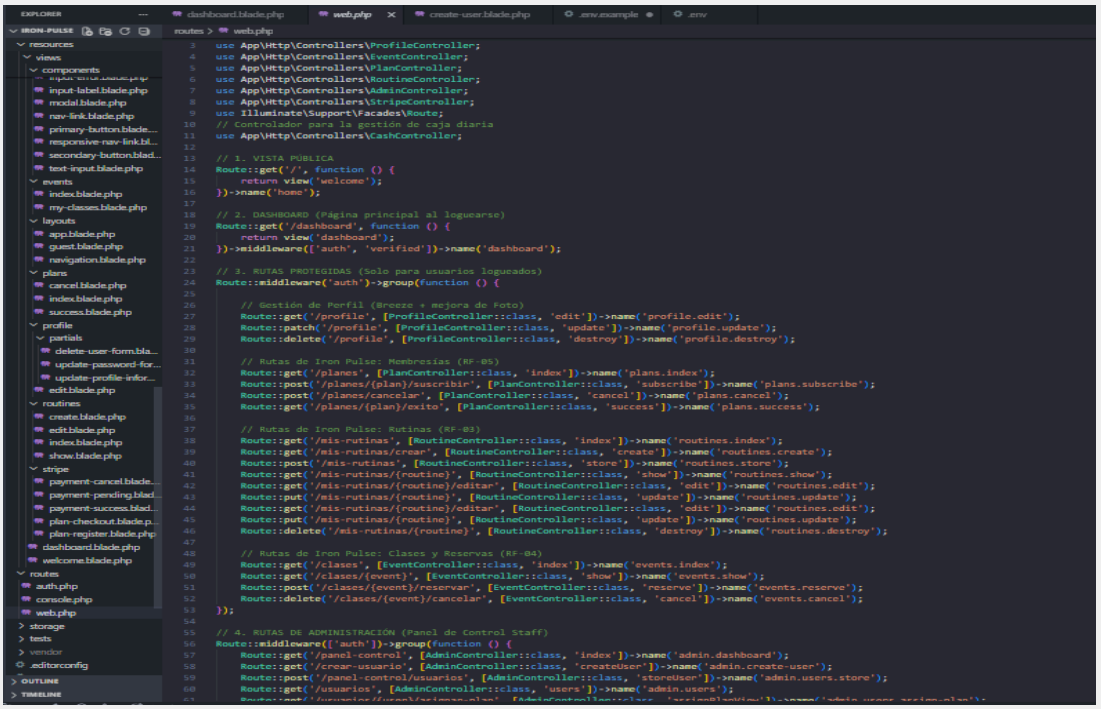
Como mejora futura, se podría realizar una revisión completa de accesibilidad para comprobar aspectos como navegación con teclado, foco visible en todos los elementos interactivos, atributos ARIA en componentes dinámicos, contraste exacto según WCAG y validación automática con herramientas como Lighthouse o WAVE.

10. Desarrollo Backend

10.1. Rutas principales

El backend de Iron Pulse se organiza principalmente a través de los archivos **routes/web.php** y **routes/auth.php**. En ellos se definen las URLs de la aplicación, el controlador encargado de procesar cada petición y los middlewares que protegen las zonas privadas.

La aplicación separa sus rutas en varios bloques: rutas públicas, rutas de usuario autenticado, rutas de administración, rutas de caja, rutas de Stripe y rutas de autenticación generadas por Laravel Breeze.



```php
use App\Http\Controllers\ProfileController;
use App\Http\Controllers\EventController;
use App\Http\Controllers\PlanController;
use App\Http\Controllers\RoutineController;
use App\Http\Controllers\AdminController;
use App\Http\Controllers\StripeController;
use Illuminate\Support\Facades\Route;
// Controlador para la gestión de caja diaria
use App\Http\Controllers\CashController;

// 1. VISTA PÚBLICA
Route::get('/', function () {
    return view('welcome');
})->name('home');

// 2. DASHBOARD (Página principal al loguearse)
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

// 3. RUTAS PROTEGIDAS (Solo para usuarios logueados)
Route::middleware('auth')->group(function () {

    // Gestión de Perfil (Breeze + mejora de Foto)
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');

    // Rutas de Iron Pulse: Membresías (RF-05)
    Route::get('/planes', [PlanController::class, 'index'])->name('plans.index');
    Route::post('/planes/{plan}/suscribir', [PlanController::class, 'subscribe'])->name('plans.subscribe');
    Route::post('/planes/cancelar', [PlanController::class, 'cancel'])->name('plans.cancel');
    Route::get('/planes/{plan}/exito', [PlanController::class, 'success'])->name('plans.success');

    // Rutas de Iron Pulse: Rutinas (RF-03)
    Route::get('/mis-rutinas', [RoutineController::class, 'index'])->name('routines.index');
    Route::get('/mis-rutinas/crear', [RoutineController::class, 'create'])->name('routines.create');
    Route::post('/mis-rutinas', [RoutineController::class, 'store'])->name('routines.store');
    Route::get('/mis-rutinas/{routine}', [RoutineController::class, 'show'])->name('routines.show');
    Route::get('/mis-rutinas/{routine}/editar', [RoutineController::class, 'edit'])->name('routines.edit');
    Route::put('/mis-rutinas/{routine}', [RoutineController::class, 'update'])->name('routines.update');
    Route::get('/mis-rutinas/{routine}/editar', [RoutineController::class, 'edit'])->name('routines.edit');
    Route::put('/mis-rutinas/{routine}', [RoutineController::class, 'update'])->name('routines.update');
    Route::delete('/mis-rutinas/{routine}', [RoutineController::class, 'destroy'])->name('routines.destroy');

    // Rutas de Iron Pulse: Clases y Reservas (RF-04)
    Route::get('/clases', [EventController::class, 'index'])->name('events.index');
    Route::get('/clases/{event}', [EventController::class, 'show'])->name('events.show');
    Route::post('/clases/{event}/reservar', [EventController::class, 'reserve'])->name('events.reserve');
    Route::delete('/clases/{event}/cancelar', [EventController::class, 'cancel'])->name('events.cancel');
});

// 4. RUTAS DE ADMINISTRACIÓN (Panel de Control Staff)
Route::middleware(['auth'])->group(function () {
    Route::get('/panel-control', [AdminController::class, 'index'])->name('admin.dashboard');
    Route::get('/crear-usuario', [AdminController::class, 'createUser'])->name('admin.create-user');
    Route::post('/panel-control/usuarios', [AdminController::class, 'storeUser'])->name('admin.users.store');
    Route::get('/usuarios', [AdminController::class, 'users'])->name('admin.users');
```

Imagen 45: *Primera parte de rutas principales*

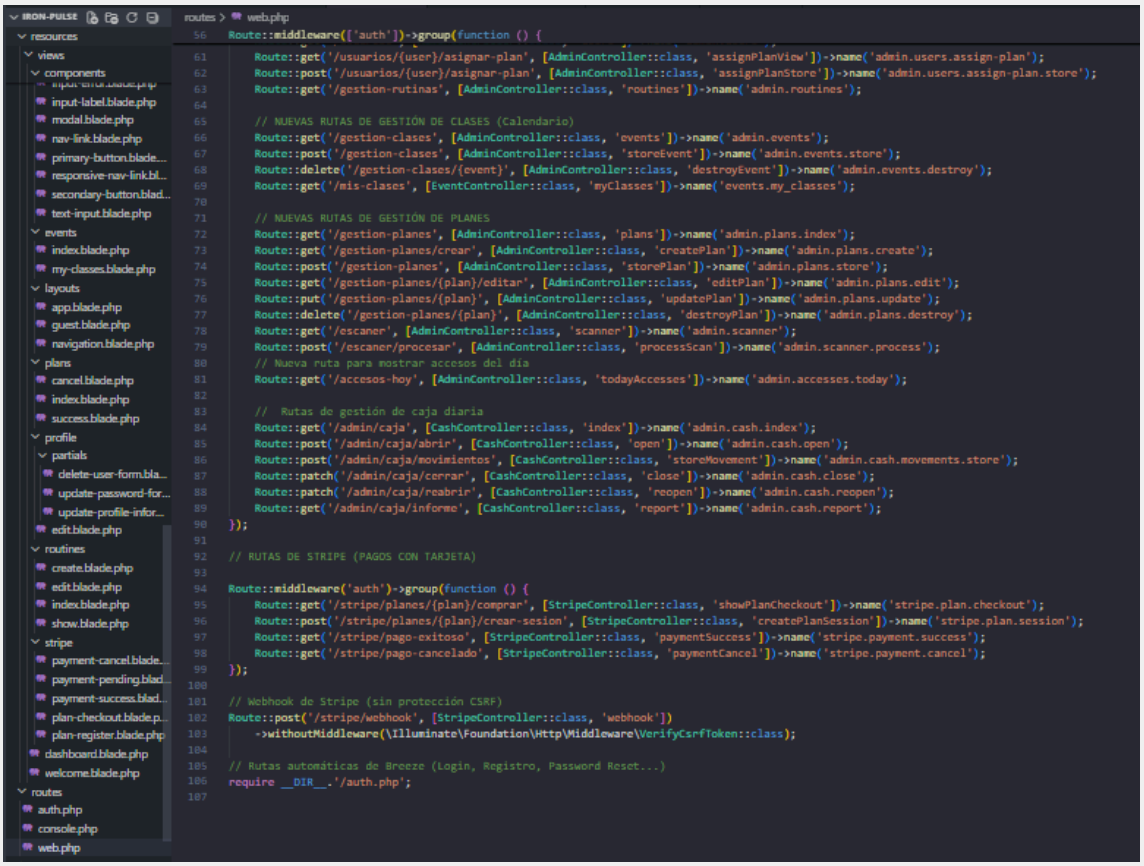


Imagen 46: *Segunda parte de rutas principales*

Rutas públicas

La ruta pública principal es la página de inicio de Iron Pulse:

Método	Ruta	Controlador / Acción	Nombre
GET	/	Vista welcome	home

Esta ruta permite acceder a la landing page del gimnasio sin necesidad de iniciar sesión.

Rutas de usuario autenticado

Las rutas privadas del usuario se agrupan bajo el middleware **auth**, lo que significa que solo pueden acceder a ellas los usuarios que han iniciado sesión.

Método	Ruta	Controlador / Acción	Función
GET	/dashboard	Vista dashboard	Panel principal del usuario.
GET	/profile	ProfileController@edit	Editar perfil.
PATCH	/profile	ProfileController@update	Actualizar perfil.
DELETE	/profile	ProfileController@destroy	Eliminar cuenta.
GET	/planes	PlanController@index	Mostrar planes disponibles.
POST	/planes/{plan}/suscribir	PlanController@subscribe	Suscribir usuario a un plan.
POST	/planes/cancelar	PlanController@cancel	Cancelar plan actual.
GET	/planes/{plan}/exito	PlanController@success	Mostrar confirmación de suscripción.
GET	/mis-rutinas	RoutineController@index	Listar rutinas del usuario.

GET	/mis-rutinas/crear	RoutineController@create	Formulario de nueva rutina.
POST	/mis-rutinas	RoutineController@store	Guardar rutina.
GET	/mis-rutinas/{routine}	RoutineController@show	Ver detalle de rutina.
GET	/mis-rutinas/{routine}/editar	RoutineController@edit	Editar rutina.
PUT	/mis-rutinas/{routine}	RoutineController@update	Actualizar rutina.
DELETE	/mis-rutinas/{routine}	RoutineController@destroy	Eliminar rutina.
GET	/clases	EventController@index	Mostrar calendario de clases.
GET	/clases/{event}	EventController@show	Ver clase concreta.
POST	/clases/{event}/reservar	EventController@reserve	Reservar clase.
DELETE	/clases/{event}/cancelar	EventController@cancel	Cancelar reserva.
GET	/mis-clases	EventController@myClasses	Ver clases reservadas.

Rutas de administración

Las rutas de administración también están protegidas mediante auth. Además, dentro del AdminController se comprueba el rol del usuario mediante Auth::user()→role !== 'admin'. Si el usuario no es administrador, se le redirige al dashboard con un mensaje de acceso denegado.

Método	Ruta	Controlador / Acción	Función
GET	/panel-control	AdminController@index	Dashboard administrativo.
GET	/crear-usuario	AdminController@createUser	Formulario de alta de socio.
POST	/panel-control/usuarios	AdminController@storeUser	Guardar nuevo usuario.
GET	/usuarios	AdminController@users	Listado y búsqueda de usuarios.
GET	/usuarios/{user}/asignar-plan	AdminController@assignPlanView	Vista para asignar plan.
POST	/usuarios/{user}/asignar-plan	AdminController@assignPlanStore	Guardar asignación de plan.
GET	/gestion-rutinas	AdminController@routines	Gestión de rutinas.

GET	/gestion-clases	AdminController@events	Gestión de clases.
POST	/gestion-clases	AdminController@storeEvent	Crear clase.
DELETE	/gestion-clases/{event}	AdminController@destroyEvent	Eliminar clase.
GET	/gestion-planes	AdminController@plans	Listar planes.
GET	/gestion-planes/crear	AdminController@createPlan	Formulario de nuevo plan.
POST	/gestion-planes	AdminController@storePlan	Guardar plan.
GET	/gestion-planes/{plan}/editar	AdminController@editPlan	Editar plan.
PUT	/gestion-planes/{plan}	AdminController@updatePlan	Actualizar plan.
DELETE	/gestion-planes/{plan}	AdminController@destroyPlan	Eliminar plan.
GET	/escaner	AdminController@scanner	Vista del escáner QR.
POST	/escaner/procesar	AdminController@processScan	Procesar QR escaneado.

GET	/accesos-hoy	AdminController@todayAccesses	Ver accesos del día.

Rutas de caja

Las rutas de caja están dentro del bloque de administración y se gestionan mediante **CashController**.

Método	Ruta	Controlador / Acción	Función
GET	/admin/caja	CashController@index	Pantalla principal de caja.
POST	/admin/caja/abrir	CashController@open	Abrir caja.
POST	/admin/caja/movimientos	CashController@storeMovement	Registrar movimiento.
PATCH	/admin/caja/cerrar	CashController@close	Cerrar caja.
PATCH	/admin/caja/reabrir	CashController@reopen	Reabrir caja.
GET	/admin/caja/informe	CashController@report	Informe imprimible de caja.

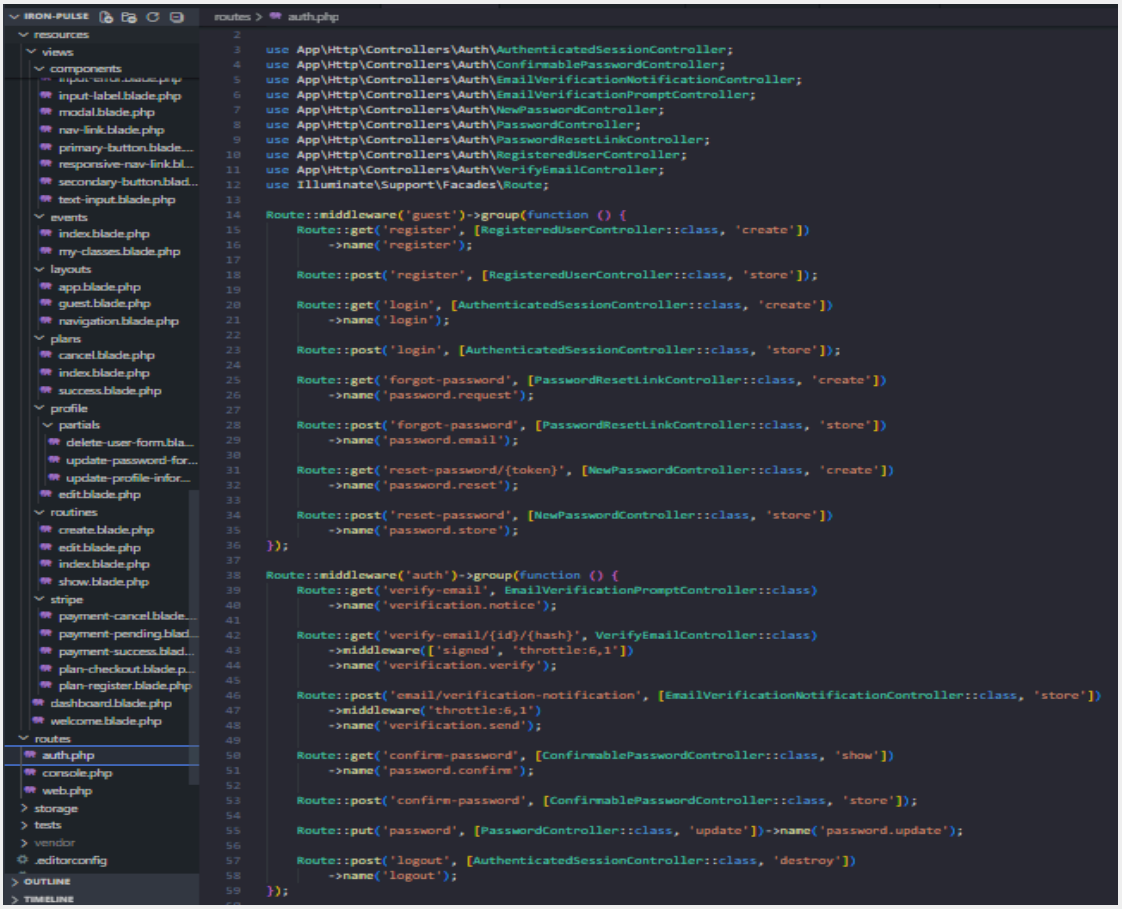
Rutas de Stripe

El flujo de pago simulado mediante **Stripe** también se encuentra protegido por auth, excepto el webhook, que se deja sin CSRF para poder recibir eventos externos de Stripe.

Método	Ruta	Controlador / Acción	Función
GET	/stripe/planes/{plan}/comprar	StripeController@showPlanCheckout	Mostrar resumen previo al pago.
POST	/stripe/planes/{plan}/crear-sesion	StripeController@createPlanSession	Crear sesión de Stripe Checkout.
GET	/stripe/pago-exitoso	StripeController@paymentSuccess	Confirmar pago y activar plan.
GET	/stripe/pago-cancelado	StripeController@paymentCancel	Mostrar pago cancelado.
POST	/stripe/webhook	StripeController@webhook	Recibir eventos de Stripe.

Rutas de autenticación

Las rutas de autenticación se cargan desde routes/auth.php y proceden de Laravel Breeze. Incluyen registro, login, recuperación de contraseña, verificación de email, confirmación de contraseña, actualización de contraseña y cierre de sesión.



```php
use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\ConfirmablePasswordController;
use App\Http\Controllers\Auth\EmailVerificationNotificationController;
use App\Http\Controllers\Auth\EmailVerificationPromptController;
use App\Http\Controllers\Auth\NewPasswordController;
use App\Http\Controllers\Auth\PasswordController;
use App\Http\Controllers\Auth\PasswordResetLinkController;
use App\Http\Controllers\Auth\RegisteredUserController;
use App\Http\Controllers\Auth\VerifyEmailController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');

    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');

    Route::post('login', [AuthenticatedSessionController::class, 'store']);

    Route::get('forgot-password', [PasswordResetLinkController::class, 'create'])
        ->name('password.request');

    Route::post('forgot-password', [PasswordResetLinkController::class, 'store'])
        ->name('password.email');

    Route::get('reset-password/{token}', [NewPasswordController::class, 'create'])
        ->name('password.reset');

    Route::post('reset-password', [NewPasswordController::class, 'store'])
        ->name('password.store');
});

Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)
        ->name('verification.notice');

    Route::get('verify-email/{id}/{hash}', VerifyEmailController::class)
        ->middleware(['signed', 'throttle:6,1'])
        ->name('verification.verify');

    Route::post('email/verification-notification', [EmailVerificationNotificationController::class, 'store'])
        ->middleware('throttle:6,1')
        ->name('verification.send');

    Route::get('confirm-password', [ConfirmablePasswordController::class, 'show'])
        ->name('password.confirm');

    Route::post('confirm-password', [ConfirmablePasswordController::class, 'store']);

    Route::put('password', [PasswordController::class, 'update'])->name('password.update');

    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});
```

Imagen 47: *Rutas de autentificación*

10.2. Controladores

El backend de Iron Pulse está organizado en varios controladores, cada uno responsable de una parte concreta de la aplicación. Esta separación permite mantener el código más ordenado y facilita el mantenimiento del proyecto.

Controlador	Responsabilidad principal
ProfileController	Gestión del perfil del usuario, actualización de datos, foto de perfil y eliminación de cuenta.

PlanController	Visualización de planes, suscripción, cancelación y página de éxito.
RoutineController	Gestión de rutinas del usuario, rutinas predeterminadas, creación, edición y eliminación.
EventController	Calendario de clases, generación automática de clases, reservas y cancelación de reservas.
AdminController	Dashboard administrativo, usuarios, planes, clases, rutinas, escáner QR y accesos del día.
CashController	Gestión de caja diaria, apertura, movimientos, cierre, reapertura e informes.
StripeController	Creación de sesiones de Stripe, confirmación de pagos, webhooks y registro de pagos en caja.
Auth/*	Controladores de Laravel Breeze para login, registro, contraseña y verificación.

ProfileController

ProfileController gestiona la pantalla de perfil del usuario. Permite mostrar el formulario de edición, actualizar el nombre y correo mediante **ProfileUpdateRequest**, subir una imagen de perfil y eliminar la cuenta del usuario.

Cuando el usuario elimina su cuenta, el sistema solicita la contraseña actual, cierra la sesión, elimina el usuario, invalida la sesión y regenera el token para evitar problemas de seguridad.

PlanController

PlanController se encarga de mostrar los planes disponibles y gestionar la suscripción básica del usuario.

Sus funciones principales son:

- **index():** obtiene todos los planes desde la tabla plans.
- **subscribe()**: si el plan tiene precio, redirige al flujo de Stripe; si es gratuito, asigna el plan directamente al usuario.
- **cancel():** elimina el plan_id del usuario.
- **success():** muestra una vista de confirmación.

RoutineController

RoutineController gestiona las rutinas de entrenamiento. El controlador distingue entre rutinas propias del usuario y rutinas predeterminadas. Las rutinas se almacenan en la tabla routines, y la relación con los usuarios se gestiona mediante la tabla pivote **routine_user**.

Una característica importante es el método privado **cleanDays()**, que limpia la estructura de días y ejercicios antes de guardarla. Este método elimina días vacíos, ejercicios incompletos y limita el máximo de días a siete.

El controlador permite:

- Listar rutinas del usuario.
- Mostrar rutinas predeterminadas no eliminadas por el usuario.
- Crear rutinas nuevas.
- Editar rutinas propias.
- Crear una copia editable cuando se modifica una rutina predeterminada.
- Marcar rutinas predeterminadas como eliminadas para un usuario mediante el campo **is_deleted** de la tabla pivote.

EventController

EventController gestiona el calendario de clases y las reservas. Incluye un horario semanal fijo mediante la propiedad **$weeklySchedule**, donde se definen clases como Yoga, Pilates, Boxeo, CrossFit, Spinning y Zumba, con día, hora de inicio, hora de fin y aforo.

El método privado **syncEventsForNextWeeks()** genera automáticamente clases futuras para las próximas semanas, evitando duplicados y descartando eventos pasados.

Sus funciones principales son:

- **index():** sincroniza las clases futuras y envía los eventos al calendario.
- **reserve():** permite reservar una clase si no ha pasado, si el usuario no está ya apuntado y si queda aforo.
- **cancel():** elimina la reserva del usuario.
- **myClasses():** muestra las clases futuras reservadas por el usuario.

El controlador también restringe el acceso al calendario para usuarios sin membresía activa, salvo que sean administradores.

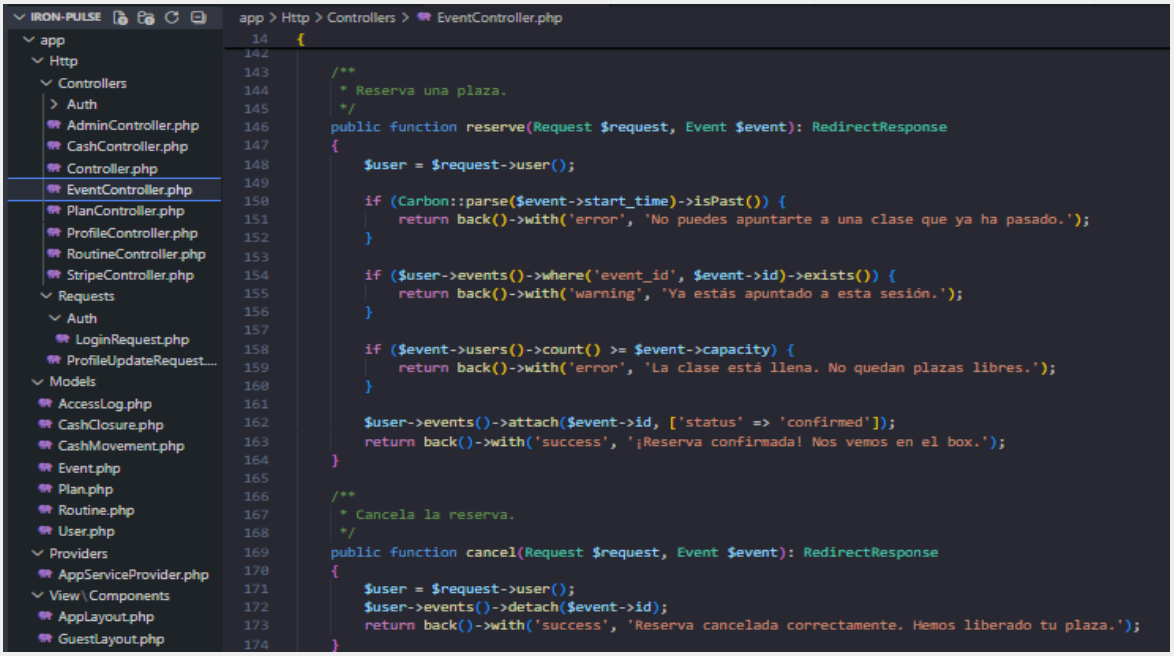


Imagen 48: *Métodos del EventController*

AdminController

AdminController centraliza la lógica del panel de administración. En casi todos sus métodos se comprueba que el usuario tenga rol admin. Si no lo tiene, se redirige al dashboard con un mensaje de acceso denegado.

Este controlador permite:

- Mostrar estadísticas del dashboard administrativo.
- Crear usuarios manualmente.
- Buscar usuarios por nombre, email o ID.
- Asignar planes a usuarios existentes.
- Gestionar clases del calendario.
- Gestionar planes.
- Mostrar el escáner QR.
- Procesar entradas y salidas mediante QR.
- Mostrar los accesos del día agrupados por usuario.

El dashboard calcula datos como usuarios totales, socios activos, accesos del día, usuarios actualmente dentro, total de rutinas, total de clases y planes disponibles.

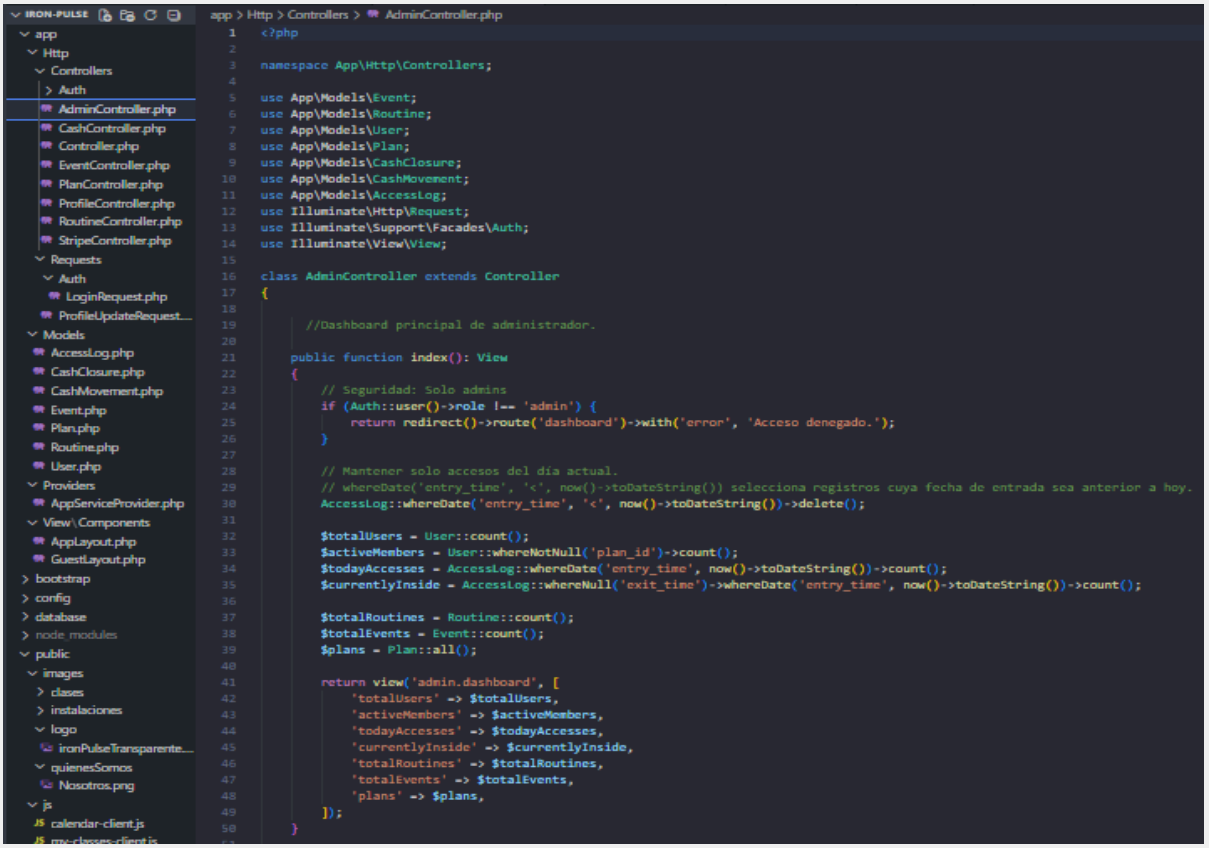


Imagen 49: *Métodos del AdminController*

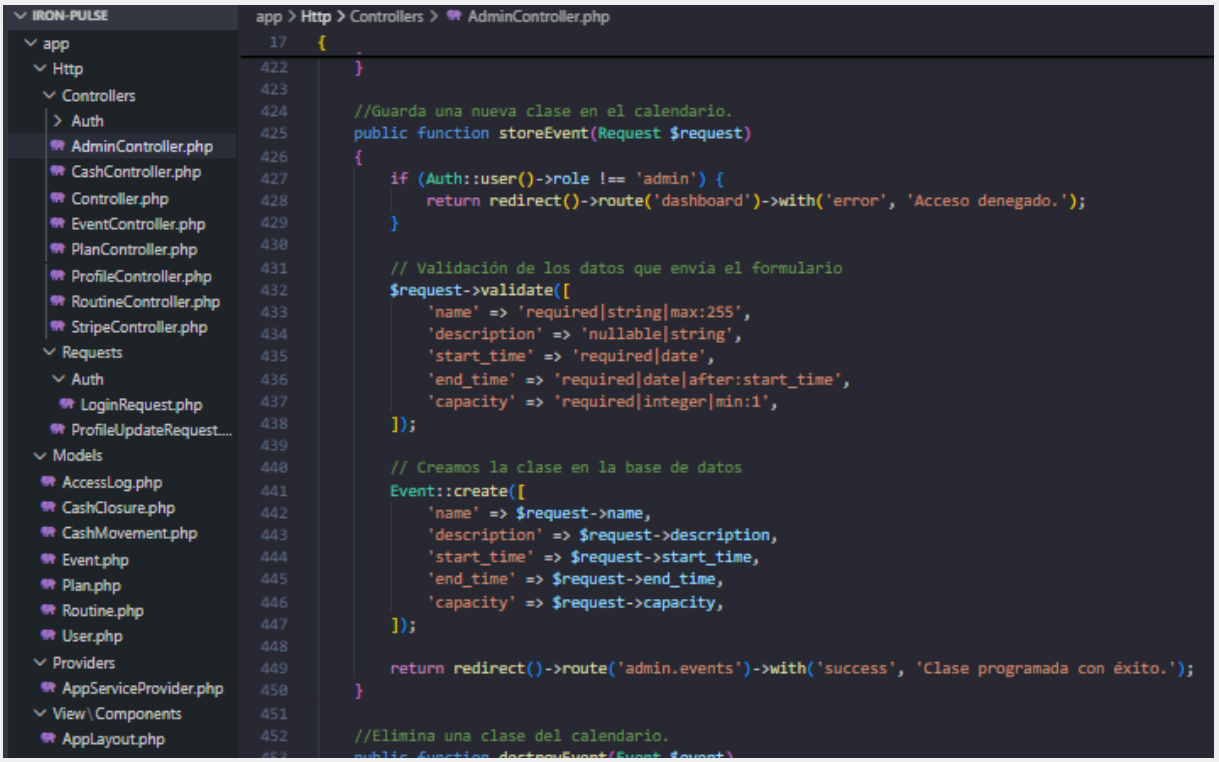


Imagen 50: *Otros métodos del AdminController*

CashController

CashController gestiona toda la lógica de caja diaria. Permite seleccionar una fecha, consultar la caja de ese día, ver movimientos, calcular ingresos por método de pago, abrir caja, registrar movimientos, cerrar caja, reabrir caja e imprimir un informe.

Los movimientos se dividen por tipo (income o expense) y método de pago (cash, card, stripe, other). El sistema calcula automáticamente:

- Total de ingresos.
- Total de gastos.
- Ingresos en efectivo.
- Ingresos con tarjeta.
- Ingresos por Stripe.
- Otros ingresos.
- Saldo actual calculado.
- Diferencia entre saldo calculado y cierre real.

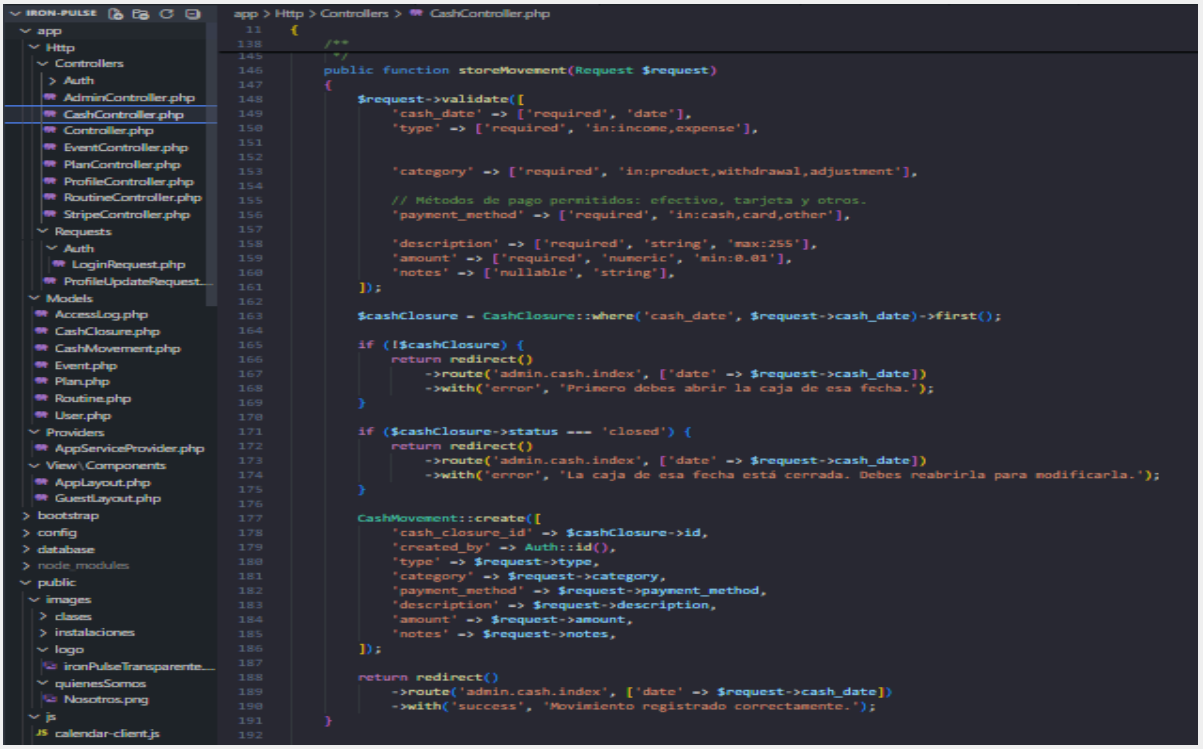


Imagen 51: *Métodos del CashController*

StripeController

StripeController gestiona el flujo de pago simulado con Stripe. En su constructor configura la clave secreta de Stripe mediante config('services.stripe.secret').

Sus funciones principales son:

- Mostrar el resumen previo al pago de un plan.
- Crear una sesión de Stripe Checkout.
- Validar el retorno desde Stripe mediante session_id.
- Comprobar que la sesión pertenece al usuario autenticado.
- Activar el plan del usuario cuando el pago está confirmado.
- Registrar el movimiento de pago en caja si existe una caja abierta.
- Procesar webhooks de Stripe.
- Marcar pagos fallidos si Stripe informa de error.

El método **fulfillCheckoutSession()** actualiza el **plan_id** del usuario y crea un movimiento en **cash_movements** con **payment_method = stripe**, evitando duplicados mediante **firstOrCreate()** sobre **stripe_payment_id**.

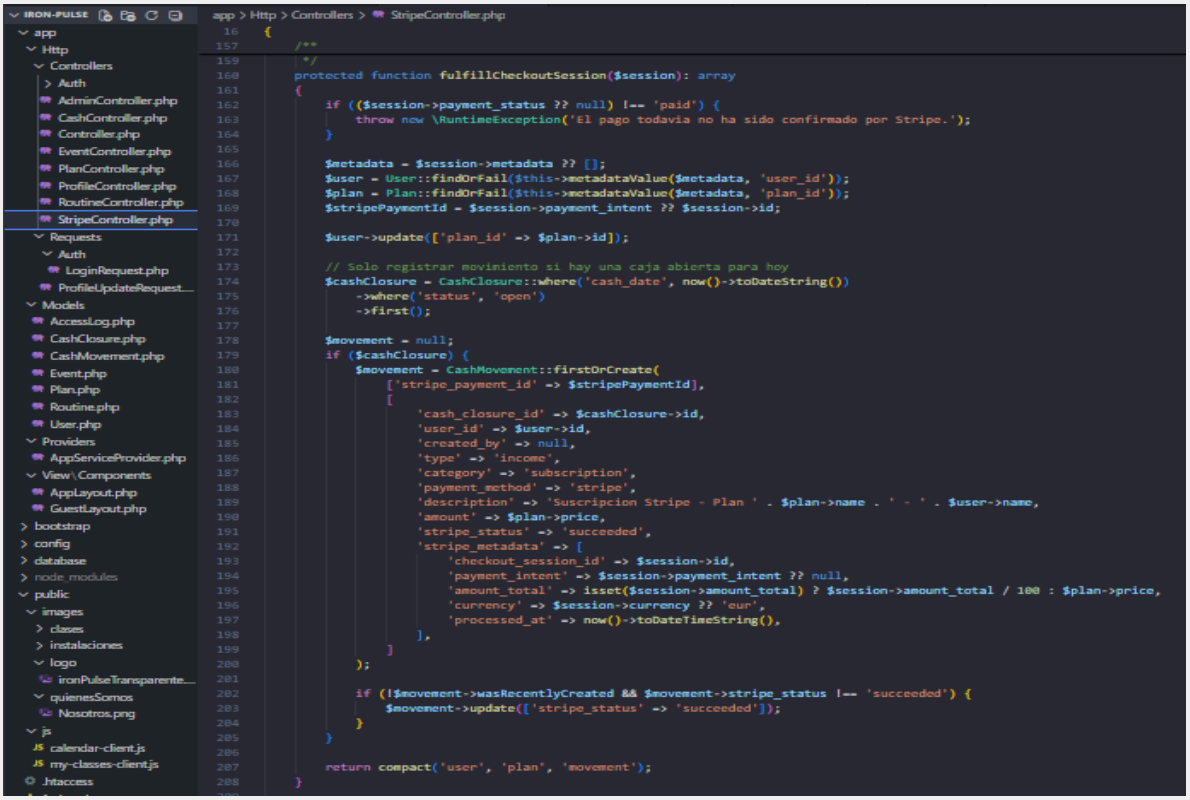


Imagen 52: *Métodos del StripeController*

10.3. Validación de formularios

La validación de formularios se realiza principalmente mediante **$request→validate()** en los controladores y mediante clases **FormRequest** en partes específicas como login y perfil.

Validaciones de autenticación

El registro de usuarios se valida en **RegisteredUserController**. Se solicitan datos como nombre, apellidos, email único, contraseña confirmada, tipo de documento, número de documento, fecha de nacimiento y aceptación RGPD.

Campo	Validación
first_name	Obligatorio, texto, máximo 255 caracteres.
last_name	Obligatorio, texto, máximo 255 caracteres.
email	Obligatorio, email, minúsculas, único en usuarios.
password	Obligatoria, confirmada y siguiendo las reglas por defecto de Laravel.
profile_picture	Opcional, imagen, máximo 2 MB.
id_type	Obligatorio, valores permitidos: DNI, NIE o Pasaporte.
id_number	Obligatorio, texto, máximo 20 caracteres.

birthdate	Obligatoria, fecha.
rgpd_consent	Obligatorio y aceptado.

Validaciones del perfil

El perfil utiliza **ProfileUpdateRequest**, donde se validan el nombre y el correo electrónico. El correo debe ser único, pero ignorando el ID del propio usuario para que pueda guardar su perfil sin cambiarlo.

Además, si el usuario sube una imagen de perfil, se valida que sea una imagen y que no supere los 2 MB.

Validaciones de rutinas

En **RoutineController**, al crear o editar rutinas se validan:

Campo	Validación
name	Obligatorio, texto, máximo 255 caracteres.
description	Opcional, texto.
dias	Opcional, array.

Después de la validación, el método **cleanDays**() limpia los días y ejercicios para evitar guardar datos vacíos o incompletos.

Validaciones de clases

En **AdminController@storeEvent**, al crear una clase se valida:

Campo	Validación
name	Obligatorio, texto, máximo 255 caracteres.
description	Opcional, texto.
start_time	Obligatorio, fecha.
end_time	Obligatorio, fecha posterior a start_time.
capacity	Obligatorio, entero, mínimo 1.

Validaciones de planes

En la creación y edición de planes se valida:

Campo	Validación
name	Obligatorio, texto, máximo 255 caracteres.
description	Opcional, texto.
price	Obligatorio, numérico, mínimo 0.

Validaciones de caja

En **CashController**, cada operación de caja tiene sus propias reglas:

Acción	Campos validados
Abrir caja	cash_date obligatorio tipo fecha, opening_amount numérico mínimo 0.
Registrar movimiento	cash_date, type, category, payment_method, description, amount y notes.
Cerrar caja	cash_date, closing_amount y notes.
Reabrir caja	cash_date.

Los movimientos de caja solo permiten tipos income y expense, y métodos de pago cash, card y other en movimientos manuales. Los pagos por Stripe se registran desde StripeController con payment_method = stripe.

Validaciones de Stripe

En StripeController@paymentSuccess se valida que llegue un **session_id** obligatorio y de tipo texto. Después, el backend recupera la sesión desde Stripe y comprueba que el **user_id** de la metadata coincida con el usuario autenticado.

Validaciones de reservas

En **EventController@reserve**, además de validar mediante lógica interna, el sistema comprueba tres condiciones antes de reservar:

1. Que la clase no haya pasado.
2. Que el usuario no esté ya apuntado.
3. Que la clase no haya alcanzado su capacidad máxima.

Estas comprobaciones evitan reservas duplicadas, reservas en clases finalizadas y sobrepasar el aforo.

10.4. Gestión de sesiones y autenticación

La autenticación de Iron Pulse está basada en Laravel Breeze. Este paquete proporciona una estructura completa para registro, inicio de sesión, cierre de sesión, recuperación de contraseña, verificación de email, confirmación de contraseña y actualización de contraseña.

Registro

El registro se gestiona mediante **RegisteredUserController**. Cuando un usuario se registra, el sistema valida los datos, guarda la imagen de perfil si existe, crea el usuario, lanza el evento Registered y autentica automáticamente al usuario mediante:

Auth::login($user);

El usuario se crea con rol user por defecto.

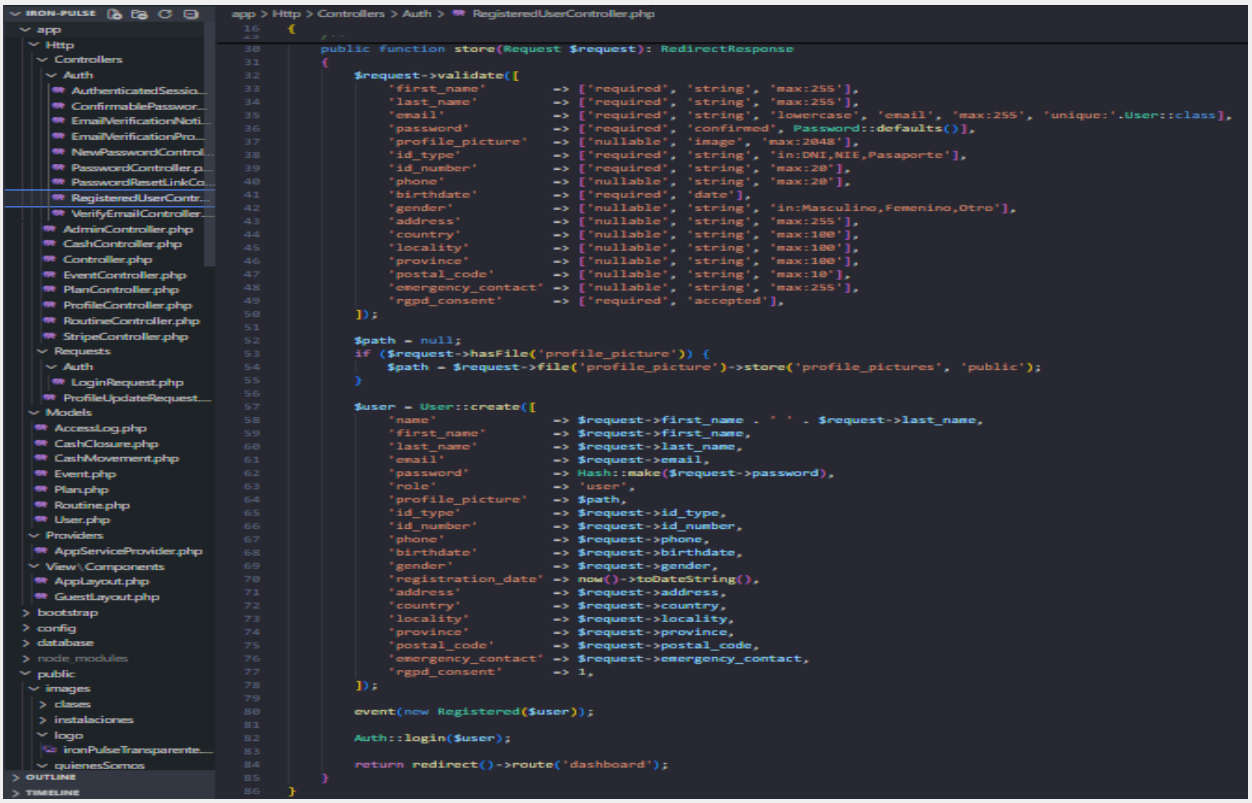


Imagen 53: *Parte de código de RegisterUserController*

Inicio de sesión

El inicio de sesión se gestiona mediante **AuthenticatedSessionController** y **LoginRequest**. **LoginRequest** valida email y contraseña, intenta autenticar al usuario mediante Auth::attempt() y aplica limitación de intentos mediante **RateLimiter**.

Si el login es correcto, se regenera la sesión con:

$request→session()→regenerate();

Esta medida ayuda a evitar ataques de fijación de sesión.

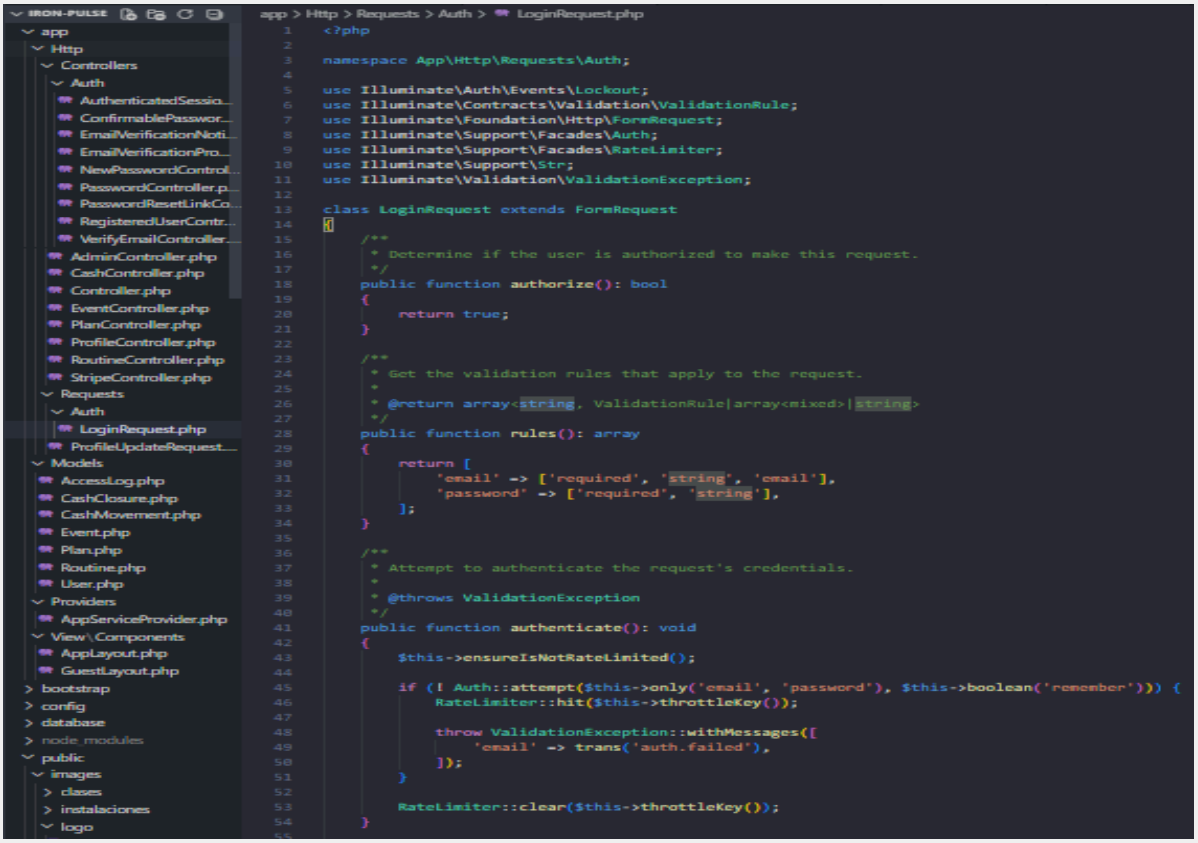


Imagen 54: *Parte de código de LoginRequest*

Cierre de sesión

El cierre de sesión se realiza mediante **AuthenticatedSessionController@destroy**. El sistema cierra la sesión, invalida la sesión actual y regenera el token CSRF:

Auth::guard('web')→logout();

$request→session()→invalidate();

$request→session()→regenerateToken();

Protección de rutas

Las rutas privadas utilizan el middleware auth. Además, el dashboard principal utiliza auth y verified.

La zona de administración está protegida por auth y por comprobaciones manuales del rol admin dentro del AdminController.

Sesiones de Laravel

Laravel utiliza la tabla **sessions** para gestionar sesiones cuando la configuración del proyecto lo requiere. De esta forma, el sistema puede mantener la sesión del usuario autenticado y controlar el acceso a las rutas privadas.

10.5. Seguridad básica

Iron Pulse incorpora varias medidas de seguridad propias de Laravel y otras implementadas directamente en el código del proyecto.

Hashing de contraseñas

Las contraseñas de los usuarios se almacenan de forma segura mediante hashing. En el registro público se utiliza:

Hash::make($request→password)

En el modelo User también se define el cast:

'password' ⇒ 'hashed'

En el alta manual de usuarios desde administración se genera una contraseña inicial basada en el documento del usuario y se guarda mediante bcrypt().

Protección CSRF

Los formularios de Laravel utilizan protección CSRF mediante tokens. Además, las peticiones asíncronas incluyen el token CSRF cuando es necesario, por ejemplo en el escáner QR.

La única excepción es la ruta /stripe/webhook, que se excluye de CSRF mediante **withoutMiddleware**(VerifyCsrfToken::class) porque debe recibir peticiones externas de Stripe.

Control de acceso

Las rutas privadas requieren autenticación mediante auth. En el panel administrativo, los métodos del AdminController comprueban que el usuario tenga rol admin. Si no lo tiene, se redirige al dashboard con un mensaje de error.

Como mejora futura, se recomienda centralizar esta comprobación mediante un middleware específico de administrador, para evitar repetir la misma condición en múltiples métodos.

Protección contra fuerza bruta en login

LoginRequest utiliza **RateLimiter** para limitar los intentos de inicio de sesión. Si se superan demasiados intentos, el sistema bloquea temporalmente nuevas pruebas y lanza un evento Lockout.

Validación de datos

La aplicación valida los datos recibidos antes de almacenarlos en la base de datos. Esto reduce errores y evita entradas incorrectas en formularios críticos como registro, caja, planes, clases, rutinas y pagos.

Uso de Eloquent ORM

La aplicación utiliza Eloquent ORM para trabajar con la base de datos mediante modelos. Esto ayuda a evitar consultas SQL manuales y reduce el riesgo de inyección SQL.

Protección en pagos Stripe

El sistema comprueba la configuración de claves de Stripe antes de crear sesiones de pago. En el retorno de pago, se recupera la sesión desde Stripe y se verifica que el **`user_id`** de la metadata coincida con el usuario autenticado. En el webhook, se valida la firma mediante **`Stripe-Signature`** y **`STRIPE_WEBHOOK_SECRET`.**

Control de acceso mediante QR

El escáner QR valida el formato esperado del código, comprueba que el usuario exista y verifica que tenga un plan activo. Además, diferencia entre entrada y salida para evitar que un usuario entre dos veces sin haber salido o salga sin haber entrado.

10.6. Panel de administración

El panel de administración es una de las partes principales del backend de Iron Pulse. Está gestionado principalmente por **AdminController** y se accede desde la ruta /panel-control.

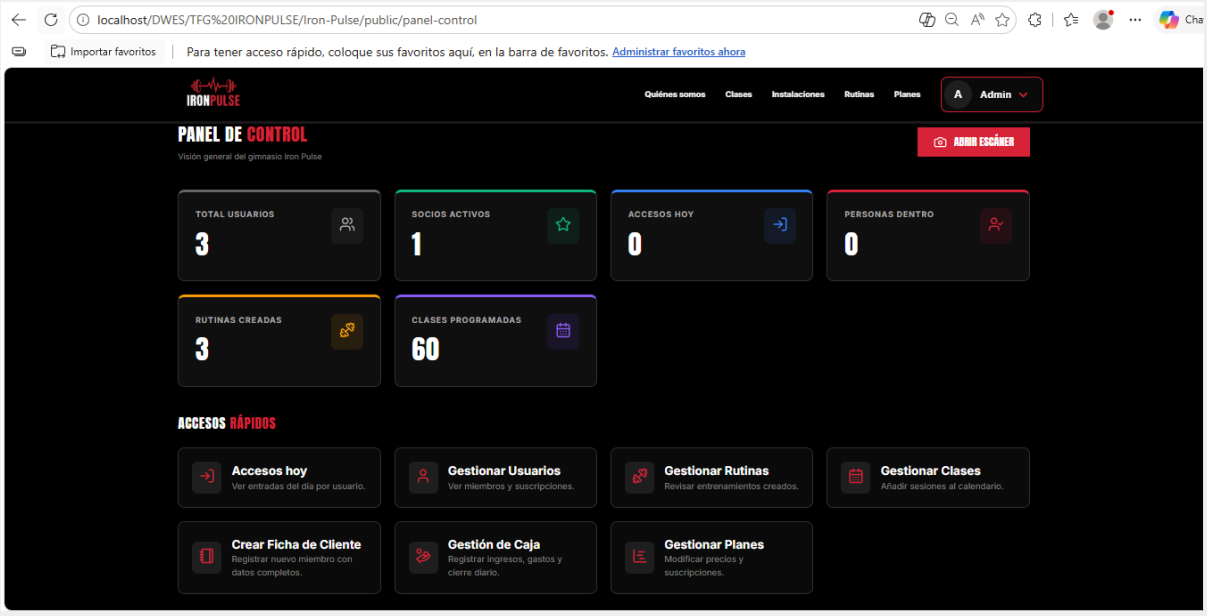


Imagen 55: *Panel de administración*

Dashboard administrativo

El dashboard muestra estadísticas generales del gimnasio:

Indicador	Descripción
Usuarios totales	Número total de usuarios registrados.
Socios activos	Usuarios con un plan_id asignado.
Accesos del día	Registros de entrada del día actual.

Usuarios dentro	Usuarios con entrada abierta y sin salida registrada.
Rutinas totales	Número total de rutinas existentes.
Clases totales	Número total de clases registradas.
Planes	Lista de planes disponibles.

Antes de calcular los accesos del día, el controlador elimina registros antiguos de **access_logs** cuya fecha de entrada sea anterior al día actual. Esto permite que el contador del dashboard se centre en la actividad del día.

Gestión de usuarios

Desde administración se pueden listar usuarios, buscarlos por nombre, correo o ID, crear nuevos usuarios manualmente y asignarles un plan.

Al crear un usuario desde administración se validan sus datos personales, documento, email, dirección, plan, método de pago y aceptación **RGPD**. Además, el sistema comprueba que exista una caja abierta antes de registrar el cobro físico del plan.

Cuando se asigna un plan a un usuario existente, también se registra un movimiento de caja si la caja está abierta.

Gestión de planes

El panel permite crear, editar y eliminar planes. Cada plan tiene nombre, descripción y precio. Estos planes se muestran posteriormente en la zona pública/privada del usuario y pueden utilizarse en el flujo de suscripción o pago simulado.

Gestión de clases

El administrador puede gestionar las clases del calendario. Puede visualizar eventos, crear nuevas clases con nombre, descripción, fecha de inicio, fecha de fin y aforo, y eliminar clases existentes.

Gestión de rutinas

Desde administración se puede acceder a la gestión de rutinas. El sistema trabaja con rutinas propias de usuario y rutinas predeterminadas, que pueden mostrarse a los usuarios según la lógica definida en **RoutineController**.

Escáner QR y accesos

El panel incluye una vista de escáner QR en **/escaner**. Esta pantalla permite leer el código QR de un usuario mediante la cámara del dispositivo. El backend procesa el contenido del QR, valida usuario, plan y acción, y registra una entrada o salida en **access_logs**.

También existe una vista de accesos del día en /accesos-hoy, donde se muestran los accesos agrupados por usuario y se calcula el total de accesos registrados durante la jornada.

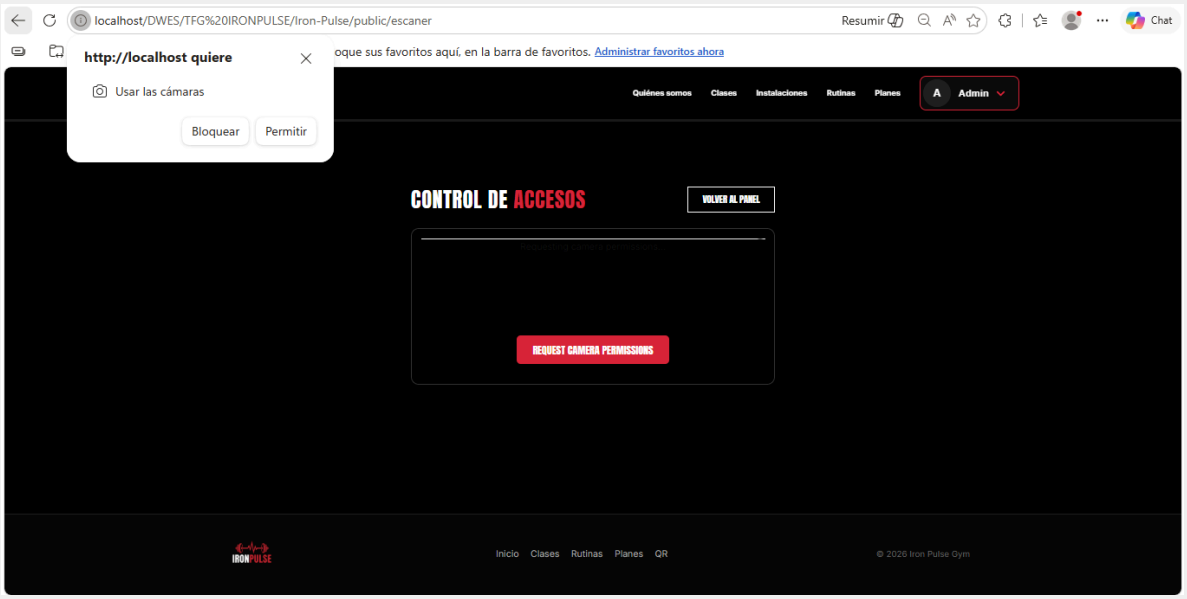


Imagen 56: *Escáner QR y accesos*

Gestión de caja

La caja se gestiona desde /admin/caja mediante **CashController**. El administrador puede:

- Consultar una caja por fecha.
- Abrir caja.
- Registrar ingresos o gastos.
- Cerrar caja.
- Reabrir caja.
- Consultar el historial de cajas.
- Generar un informe imprimible.

La caja está conectada con los pagos de membresías y con Stripe. Cuando se crea un usuario con plan, se asigna un plan a un socio o se confirma un pago de Stripe con caja abierta, el sistema puede registrar el movimiento correspondiente en **cash_movements.**

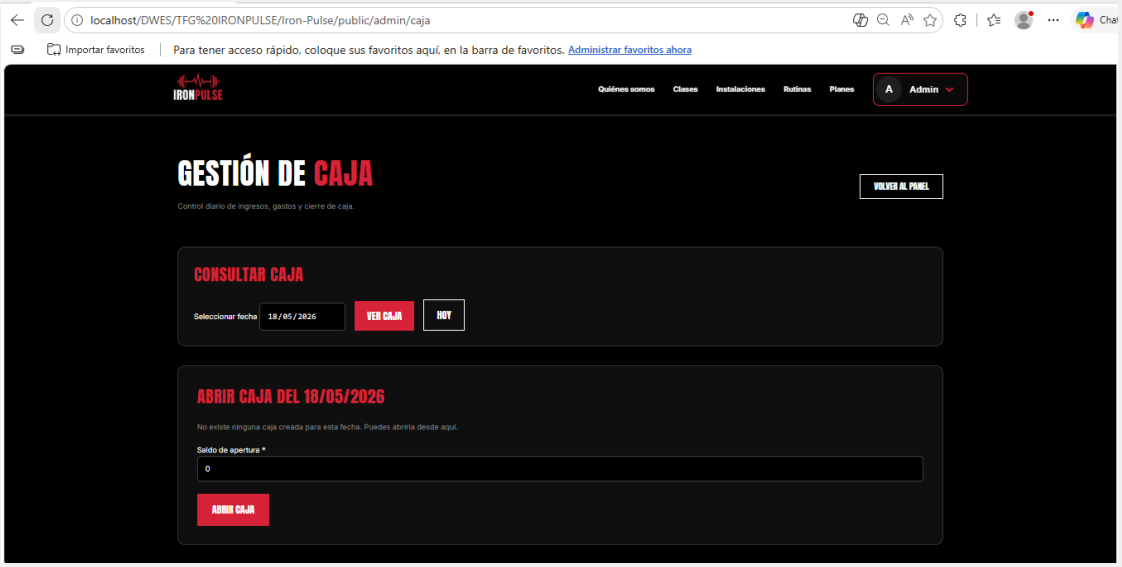


Imagen 57: *Gestión de caja*

11. Integración con Base de Datos

11.1 CRUDs implementados

El sistema sigue el patrón de diseño **MVC (Modelo-Vista-Controlador)** de Laravel. Para gestionar los datos definidos en el diagrama de la base de datos, se han desarrollado controladores específicos que administran el ciclo de vida (Creación, Lectura, Actualización y Borrado) de cada entidad.

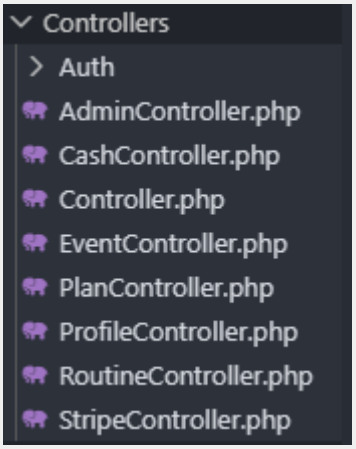


Imagen 58: *Listado de controladores más representativos*

Gestión de Entrenamientos (RoutineController)

Creación y Relación: En el método store, no solo se crea la rutina en la tabla routines, sino que se vincula automáticamente al usuario mediante la tabla pivote **routine_user** usando el método attach() de Eloquent.

Lectura Inteligente: El método index realiza consultas filtradas para diferenciar entre rutinas predeterminadas del gimnasio y las personalizadas por el usuario.

Actualización Protegida: Si un usuario intenta editar una rutina predeterminada, el controlador crea una copia personalizada en la base de datos para no alterar los datos globales.

Borrado Lógico: Se implementa un sistema donde las rutinas predeterminadas no se borran físicamente, sino que se marcan como ocultas para el usuario específico (**is_deleted**).

Control Financiero (CashController y StripeController)

Estos controladores gestionan la integridad de las tablas económicas del diagrama (**cash_movements** y **cash_closures**):

- **Persistencia de Pagos:** Se registran los ingresos vinculándolos al user_id correspondiente, asegurando que cada movimiento financiero tenga trazabilidad.
- **Integración Externa:** El StripeController actúa como un CRUD automatizado que, tras confirmar un pago externo, actualiza el campo plan_id en la tabla users.

Gestión de Eventos y Accesos (EventController y AccessLogController)

- **Inscripciones:** Permiten el registro de usuarios en clases específicas mediante la tabla **event_user**.
- **Auditoría:** Gestionan la lectura y creación de registros en **access_logs** para llevar un control de aforo en tiempo real.

Ejemplos de puntos importantes:

```php
public function store(Request $request): RedirectResponse
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'nullable|string',
        'dias' => 'nullable|array',
    ]);

    // Limpiar días vacíos y limitar a máximo 7
    $dias = $this->cleanDays($request->dias ?? [], 7);

    $routine = Routine::create([
        'name' => $validated['name'],
        'description' => $validated['description'] ?? '',
        'exercises' => $dias,
    ]);

    $request->user()->routines()->attach($routine->id);

    return redirect()->route('routines.index')->with('success', 'Rutina creada');
}
```

Imagen 59: *Método store del controlador de rutina*

```php
public function update(Request $request, Routine $routine): RedirectResponse
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'nullable|string',
        'dias' => 'nullable|array',
    ]);

    // Limpiar dias vacios y limitar a máximo 7
    $dias = $this->cleanDays($request->dias ?? [], 7);

    if ($routine->is_default) {
        // Si es predeterminada, crear una nueva rutina para el usuario
        $newRoutine = Routine::create([
            'name' => $validated['name'],
            'description' => $validated['description'] ?? '',
            'exercises' => $dias,
            'is_default' => false,
            'gender' => null,
        ]);

        $request->user()->routines()->attach($newRoutine->id);

        // Marcar la predeterminada como eliminada para este usuario
        $request->user()->routines()->syncWithoutDetaching([$routine->id => ['is_deleted' => true]]);
    } else {
        // Actualizar la rutina normal
        $routine->update([
            'name' => $validated['name'],
            'description' => $validated['description'] ?? '',
            'exercises' => $dias,
        ]);
    }

    return redirect()->route('routines.index')->with('success', 'Rutina actualizada');
}
```

Imagen 60. *Método update del controlador de rutina*

11.2 Conexiones

La comunicación entre la aplicación Laravel y el motor de base de datos MySQL se gestiona de manera centralizada.

- **Configuración de Entorno:** Se utiliza el archivo .env para definir las credenciales de conexión, lo que permite que el sistema sea portable y seguro al no subir contraseñas al repositorio de código.
- **Abstracción de Datos:** Gracias al uso de Eloquent, el sistema se conecta automáticamente a las tablas definidas en el diagrama Diagrama_ED_iron_pulse.jpg, traduciendo las clases de PHP en consultas SQL eficientes.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=iron_pulse
DB_USERNAME=root
```

Imagen 61: *Configuración de la conexión a la base de datos en el archivo de entorno*

11.3 Consultas

Para la comunicación con la base de datos MySQL, el sistema utiliza **Eloquent ORM**. Esta capa de abstracción permite realizar peticiones complejas mediante una sintaxis orientada a objetos, lo que garantiza un código más limpio, legible y protegido contra inyecciones SQL.

A. Relaciones entre Tablas y Unión de Colecciones

El sistema aprovecha las relaciones definidas en los modelos para recuperar datos vinculados de forma eficiente. En lugar de realizar múltiples peticiones pesadas, se obtienen las colecciones de datos necesarias y se procesan en el servidor para ofrecer una respuesta unificada al usuario.

```php
$routines = $userRoutines->merge($defaultRoutines)
    ->unique(function ($routine) {
        return $routine->is_default
            ? $routine->name . '|' . $routine->gender
            : 'user-' . $routine->id;
    })
    ->values();
```

Imagen 62: *Procesamiento y unificación de colecciones de datos mediante métodos de Eloquent*

B. Consultas con Filtros Dinámicos y Tablas Pivote

Se han implementado consultas que aplican filtros lógicos en tiempo real dependiendo del contexto del usuario autenticado. Estas consultas permiten interactuar no solo con las tablas principales, sino también con las tablas pivote (como **routine_user**), filtrando registros basados en estados específicos (como si una rutina ha sido marcada como eliminada o si es predeterminada).

```php
$userRoutines = $user->routines()->where('is_default', false)->wherePivot('is_deleted', false)->get();
```

Imagen 63. *Consulta de Eloquent utilizando filtros de columna y condiciones sobre relaciones Many-to-Many*

11.4 Control de errores

Para garantizar que la base de datos MySQL mantenga su integridad, el sistema implementa una capa de validación de datos en el servidor antes de ejecutar cualquier operación de escritura (INSERT o UPDATE).

- **Validación de Atributos:** Se utiliza el motor de validación de Laravel para asegurar que los campos obligatorios (como el nombre de la rutina o los ejercicios) no lleguen vacíos y cumplan con los tipos de datos definidos en el diagrama (strings, integers, etc.).
- **Seguridad y Robustez:** Al validar los datos en el controlador, se evitan errores fatales de SQL (como el error de "campo no puede ser nulo") y se protege el sistema contra inyecciones de código malicioso.
- **Feedback al Usuario:** Si los datos no cumplen las reglas, el sistema detiene la ejecución automáticamente y devuelve un mensaje de error, impidiendo que se guarde información incompleta en las tablas.

```php
$validated = $request->validate([
    'name' => 'required|string|max:255',
    'description' => 'nullable|string',
    'dias' => 'nullable|array',
]);
```

Imagen 64: *Implementación de reglas de validación para asegurar la integridad de los registros*

12. Pruebas (Testing)

En este apartado se describen las pruebas realizadas sobre la aplicación web del gimnasio con el objetivo de verificar el correcto funcionamiento del sistema, detectar errores y asegurar una experiencia de usuario adecuada.

Las pruebas se realizaron durante distintas fases del desarrollo utilizando diferentes navegadores, dispositivos y tipos de usuario.

12.1. Pruebas de interfaz

Objetivo

Comprobar que la interfaz gráfica de la aplicación web del gimnasio se visualiza correctamente y mantiene un comportamiento responsive en distintos navegadores y dispositivos.

Navegadores utilizados

Las pruebas de interfaz se realizaron utilizando los siguientes navegadores:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Opera

Las pruebas permitieron comprobar la compatibilidad visual y funcional de la aplicación en diferentes motores de renderizado.

Dispositivos y resoluciones utilizados

Dispositivo	Resolución utilizada	Tipo de prueba
Ordenador	1920×1080	Versión escritorio
iPhone SE	Responsive móvil	Versión móvil
iPad	Responsive tablet	Versión tablet

Aspectos comprobados

Página principal

Se verificó:

- Correcta carga de la página
- Visualización adecuada de imágenes y textos
- Correcta alineación de botones y tarjetas
- Adaptación responsive en móvil y tablet

Resultado obtenido:

> La página principal se visualiza correctamente en todos los navegadores probados y se adapta adecuadamente a diferentes tamaños de pantalla.

Capturas:

Evidencia 1: Vista Móvil (iPhone SE / 375 x 667 px)

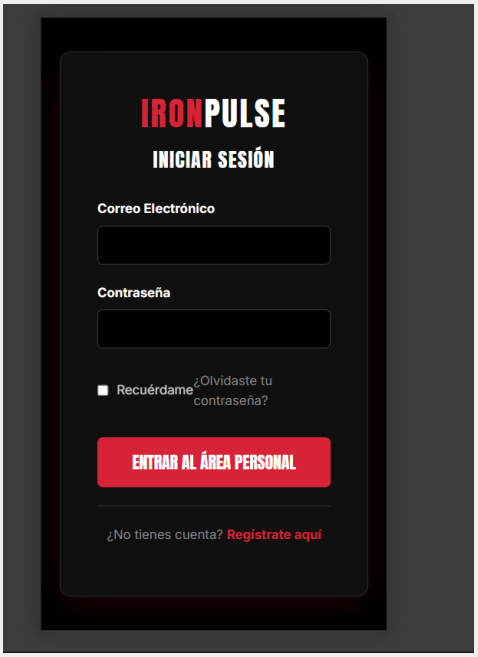


Imagen 65: Vista desde el móvil del inicio de sesión



Imagen 66: *Vista desde el movil de la página principal*

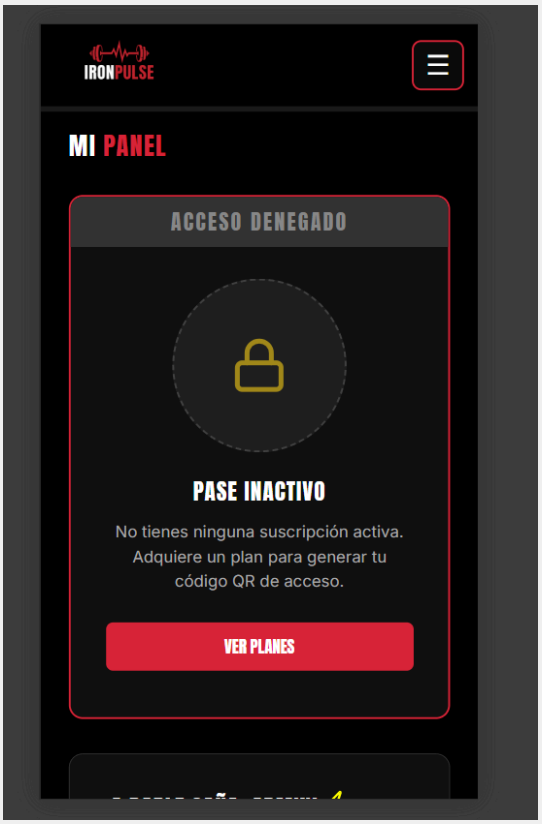


Imagen 67: *Vista desde el móvil del panel del usuario*

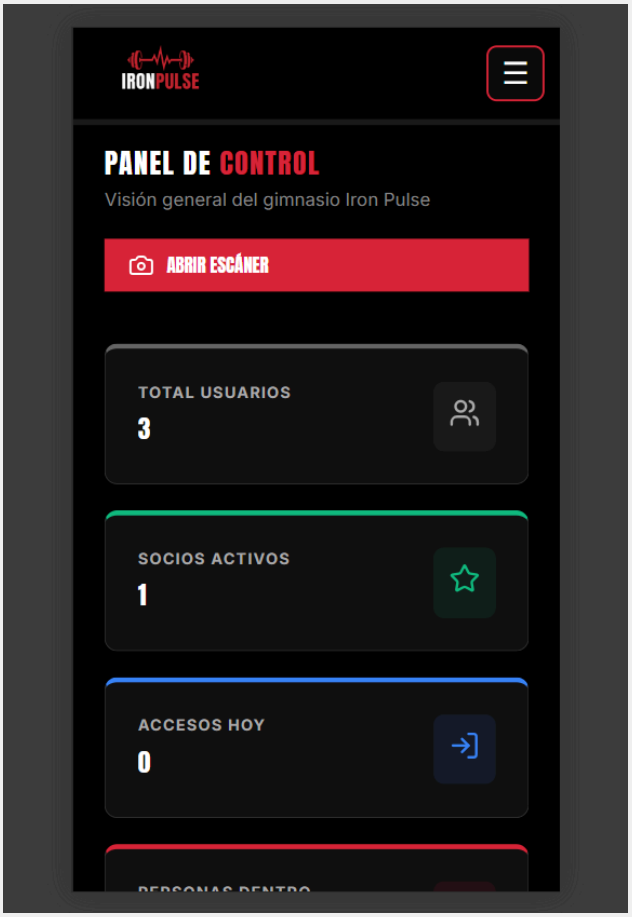


Imagen 68: *Vista desde el móvil del panel de control del administrador*

Evidencia 2: Vista Tablet (iPad / 768 x 1024 px)

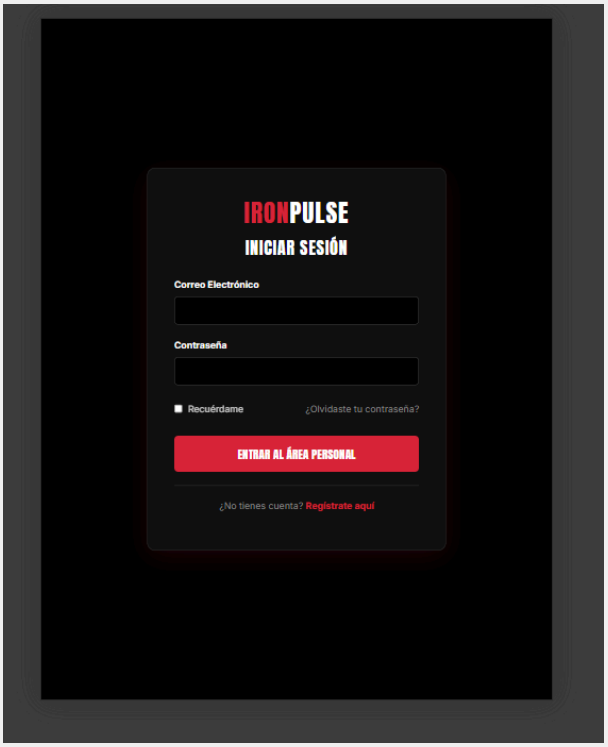


Imagen 69: *Vista desde la tablet del inicio de sesión*

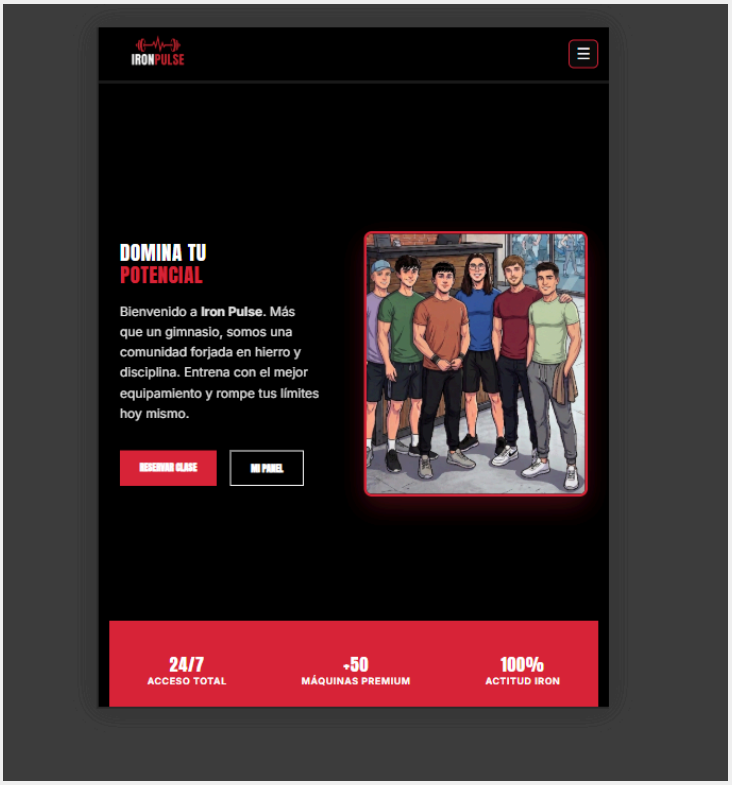


Imagen 70: *Vista desde la tablet de la página principal*

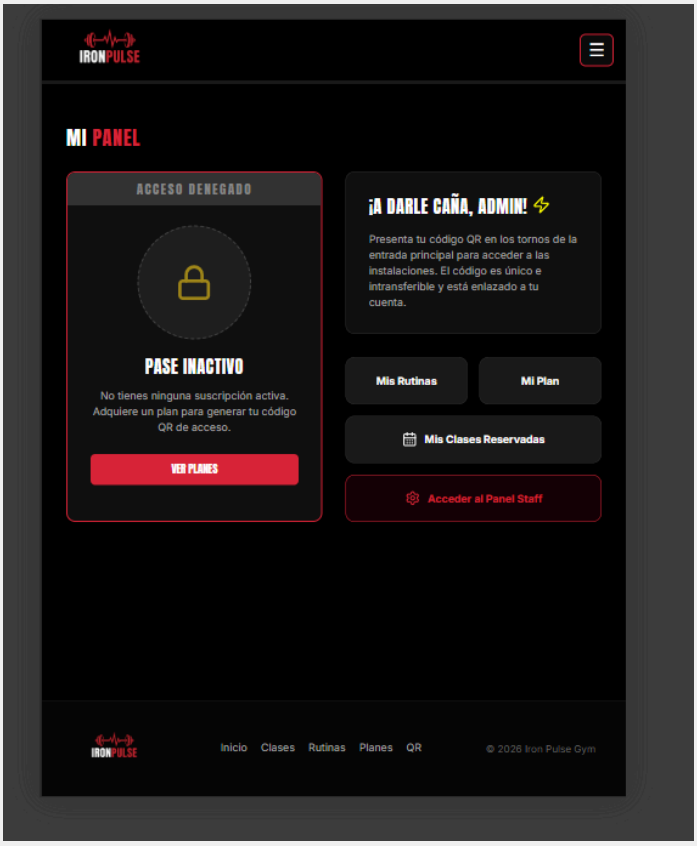


Imagen 71: *Vista desde la tablet del panel del usuario*

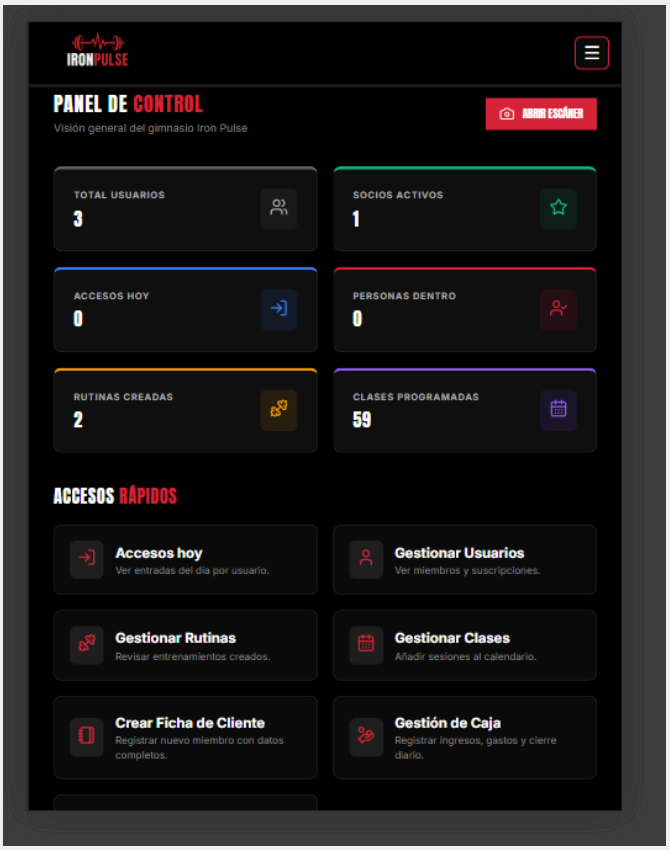


Imagen 72: *Vista desde la tablet del panel de control del administrador*

Evidencia 3: Vista Desktop (1920×1080 px)

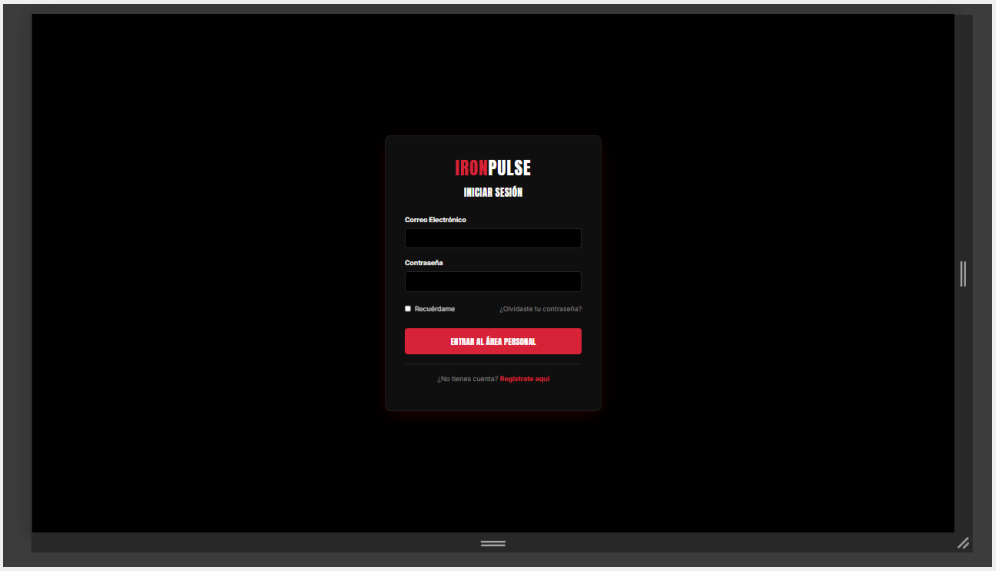


Imagen 73: *Vista desde el ordenador del inicio de sesión*

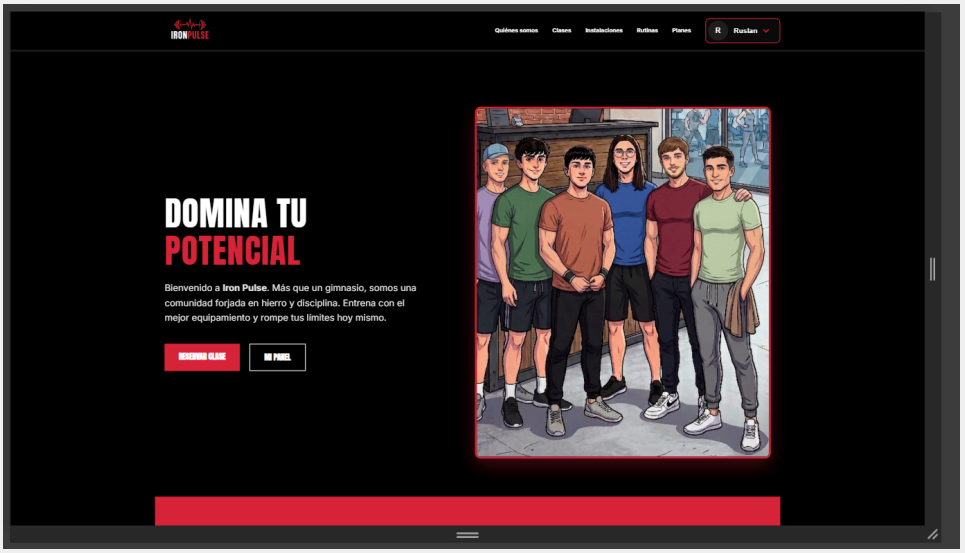


Imagen 74: *Vista desde el ordenador de la página principal*

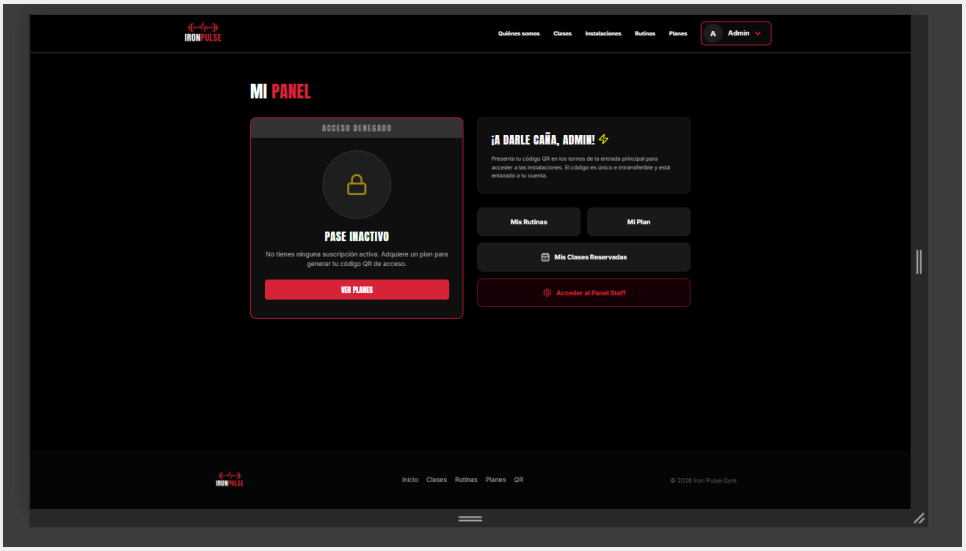


Imagen 75: *Vista desde el ordenador del panel del usuario*

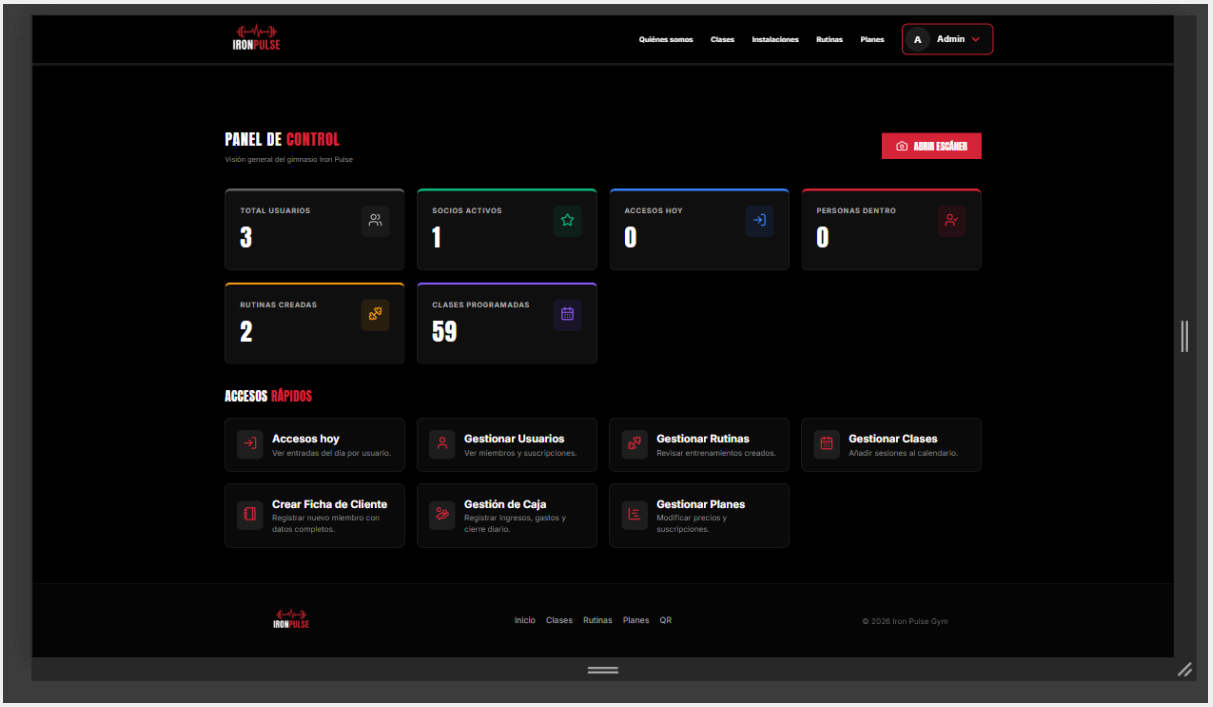


Imagen 76: *Vista desde el ordenador del panel de control del administrador*

12.2. Pruebas funcionales

Objetivo

Verificar que todas las funcionalidades implementadas en la aplicación funcionan correctamente según los requisitos definidos.

Metodología

Las pruebas funcionales se realizaron manualmente ejecutando distintos casos de uso con usuarios de prueba.

Cada prueba incluye:

- Funcionalidad
- Acción realizada
- Resultado esperado
- Resultado obtenido
- Estado final

Tabla de pruebas funcionales

ID	Funcionalidad	Acción realizada	Resultado esperado	Resultado obtenido	Estado
PF-01	Registro de usuario	Crear una cuenta nueva	Usuario registrado correctamente	Registro completado	Correcto
PF-02	Inicio de sesión	Introducir credenciales válidas	Acceso al panel usuario	Acceso correcto	Correcto
PF-03	Reserva de clases	Reservar una clase disponible	Clase reservada	Reserva realizada	Correcto
PF-04	Validación formulario	Dejar campos vacíos	Mostrar mensajes de error	Mensajes mostrados	Correcto
PF-05	Eliminación de usuarios	Eliminar usuario desde admin	Usuario eliminado	Eliminación correcta	Correcto

12.3. Pruebas de usabilidad

Objetivo

Evaluar la facilidad de uso y comprensión de la aplicación por parte de usuarios externos.

Procedimiento

Se realizaron pruebas con varios usuarios para comprobar:

- Facilidad de navegación
- Claridad de botones y menús
- Rapidez para realizar reservas
- Comprensión general de la interfaz

Resultados obtenidos

Los usuarios pudieron:

- Registrarse sin dificultad
- Reservar clases fácilmente
- Navegar entre secciones de forma intuitiva

También se detectaron pequeñas mejoras:

- Aumentar tamaño de algunos botones en móvil
- Mejorar mensajes de validación
- Se mejoró el formulario de registro de usuario

Conclusión

La aplicación presenta una interfaz intuitiva y fácil de utilizar tanto en ordenador como en dispositivos móviles.

12.4. Pruebas de seguridad

Objetivo

Verificar que la aplicación es segura frente a accesos no autorizados, manipulación de datos y vulnerabilidades comunes en aplicaciones web.

Prueba 1: Protección de rutas privadas (Middleware Auth)

Objetivo: Verificar que el sistema impide el acceso a la zona de usuario si no hay una sesión activa.

Paso	Acción	Resultado Esperado
1	Cerrar sesión en la aplicación.	El usuario vuelve al índice.
2	Escribir manualmente en la URL: /dashboard.	Redirección automática a /login.

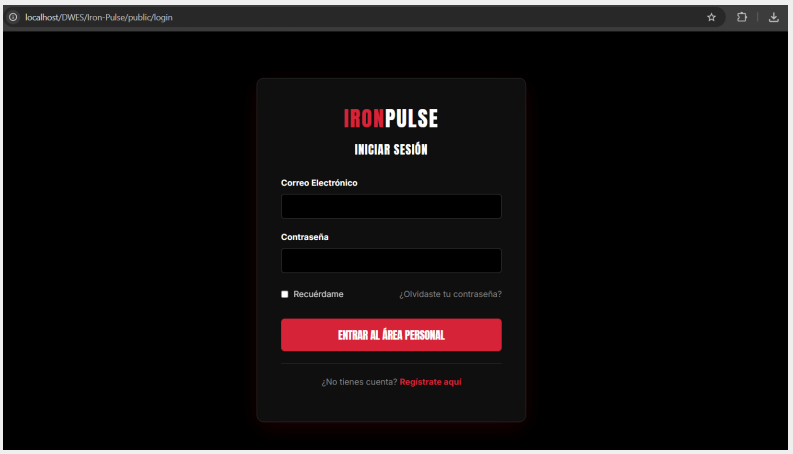


Imagen 77: *Redirección a la vista del login si no tienes sesion iniciada*

Prueba 2: Restricción de privilegios (Control de Roles)

Objetivo: Asegurar que un usuario con rol de "Cliente" no puede acceder a las herramientas del "Administrador".

Paso	Acción	Resultado Esperado
1	Iniciar sesión con una cuenta de cliente estándar.	Acceso correcto al perfil de cliente.

2	Intentar acceder a la URL: /admin (o la ruta de gestión de caja).	Error 403 Forbidden o Acceso Denegado.

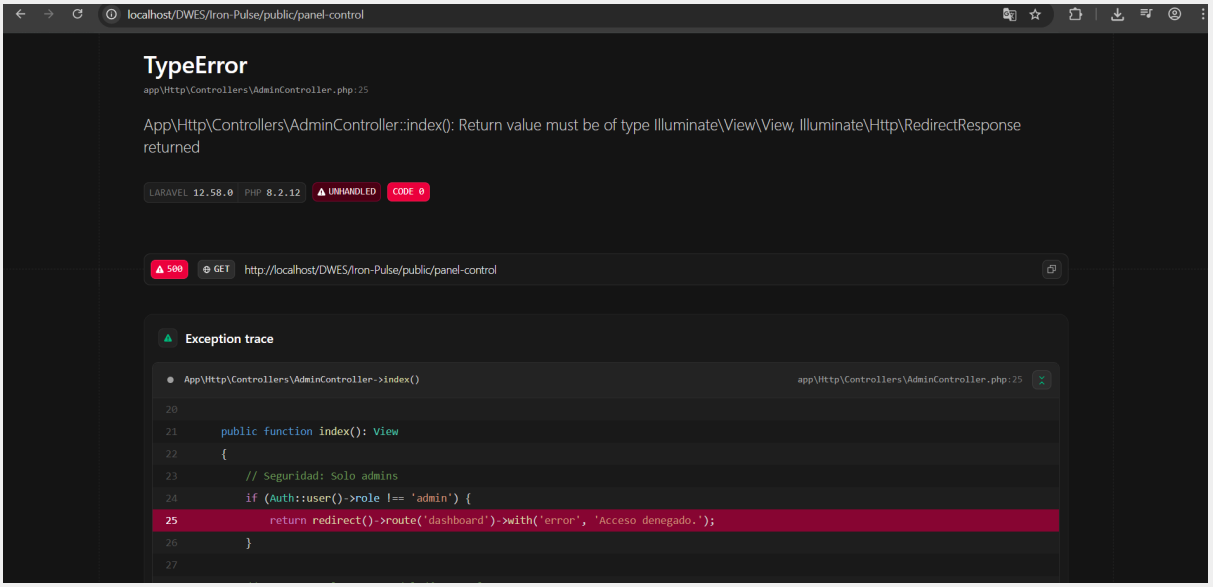


Imagen 78: *Error al intentar acceder a una ruta que solo el administrador tiene permiso.*

Prueba 3: Integridad de datos (Validación de Formularios)

Objetivo: Comprobar que el backend rechaza datos malformados o incompletos, evitando errores en la base de datos.

Paso	Acción	Resultado Esperado
1	Ir al formulario de "Caja" o "Registro".	Carga del formulario.
2	Pulsar "Enviar" con los campos vacíos o con letras en campos numéricos.	El sistema detiene el envío y muestra alertas.

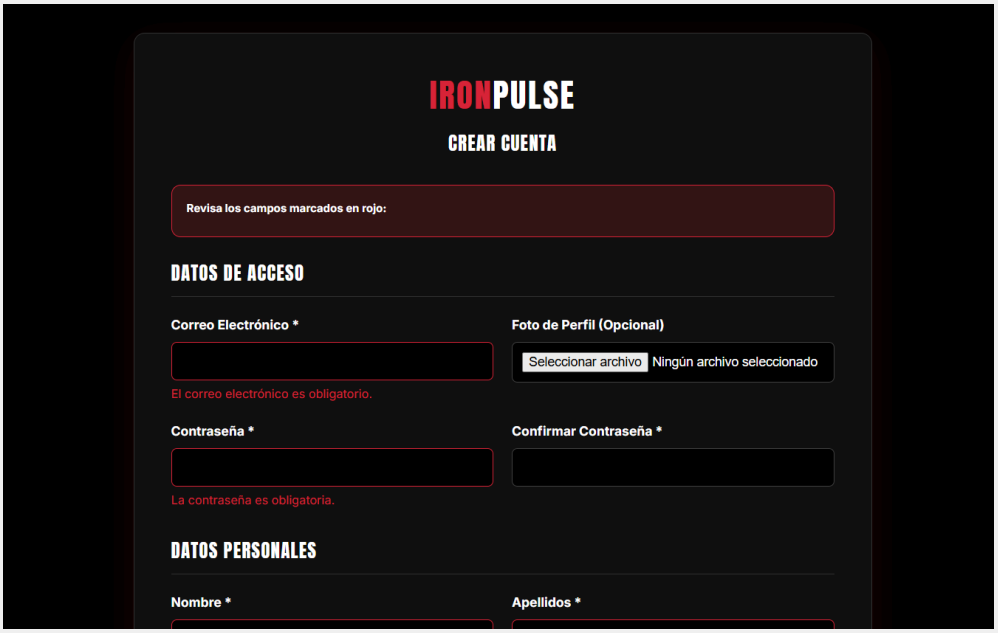


Imagen 79: *Mensajes de error al intentar registrarte omitiendo campos obligatorios.*

Prueba 4: Seguridad en Transacciones (Pasarela Stripe)

Objetivo: Confirmar que los pagos se realizan bajo protocolo seguro y que el sistema valida la respuesta externa.

Paso	Acción	Resultado Esperado
1	Seleccionar un plan y proceder al pago.	Redirección a Stripe.
2	Verificar la URL y el certificado de seguridad.	Conexión cifrada (HTTPS) y entorno seguro.

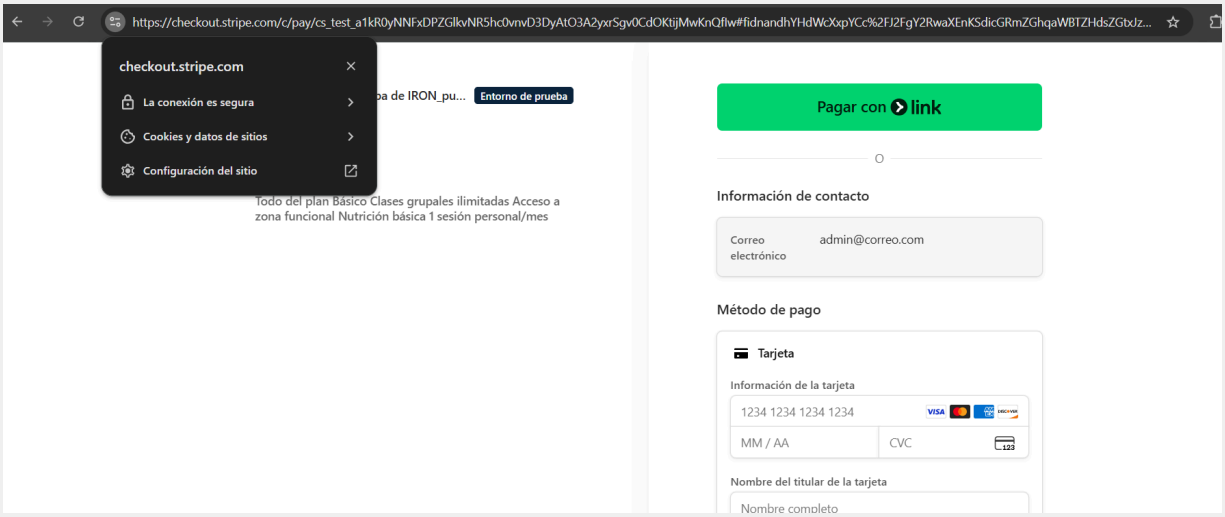


Imagen 80: *Conexión segura y cifrada (https) al realizar un pago con Stripe.*

Conclusión

Las pruebas de seguridad demuestran que la aplicación controla correctamente el acceso de usuarios, protege las rutas sensibles y valida la información introducida, reduciendo el riesgo de accesos no autorizados o manipulación de datos.

12.5. Errores detectados y solucionados

Durante la fase de pruebas se registran los errores detectados en la aplicación con el objetivo de documentar su origen, impacto y solución aplicada.
Este registro permite mantener trazabilidad de las incidencias y comprobar la evolución del sistema.

A continuación se muestra una tabla de incidencias preparada para su cumplimentación:

ID	Error detectado	Descripción	Causa	Solución
ER-01	Letra descuadrada en rutinas	La letra se descuadra en rutinas hasta que creas una nueva para otro dia	Problema de estilos en la visualización de las rutinas antiguas.	Se ajustaron los estilos para que todas las rutinas se muestren con el mismo formato.

13. Reflexión final

Propuestas de Mejora

Aunque el sistema actual cumple con los requisitos iniciales definidos para la gestión del gimnasio, durante el desarrollo hemos identificado diversas mejoras que podrían implementarse en un futuro para mejorar la experiencia del usuario (UX) y optimizar la administración del centro.

13.1. Módulo de Tienda y Sistema de Fidelización

En la tienda que tenemos en el gimnasio poner a la venta de productos relacionados con el fitness, como suplementación deportiva (proteínas, creatina, vitaminas), equipamiento y *merchandising* del gimnasio.

- **Descuentos para socios:** Para fomentar la retención y fidelización, el sistema aplicará automáticamente descuentos o tarifas especiales a aquellos usuarios que tengan una suscripción activa como clientes del gimnasio.
- **Gestión de inventario:** Permitirá al administrador llevar un control del stock de estos productos directamente desde la misma plataforma.

13.2. Añadir Control de asistencia.

Para optimizar la ocupación de las clases y evitar que queden plazas vacías debido a reservas no utilizadas, hemos pensado en implementar un sistema automatizado de control de asistencia.

- **Monitorización de reservas:** El sistema registrará cuando un usuario reserva una plaza de forma recurrente y no acude a la clase sin haber cancelado su asistencia con la antelación requerida.
- **Alertas preventivas:** Al acumular un número determinado de ausencias injustificadas, el usuario recibirá una alerta automática advirtiéndole de que está bloqueando plazas que otros clientes podrían aprovechar.
- **Penalizaciones temporales:** Si el comportamiento persiste, el sistema aplicará una penalización automática, inhabilitando la capacidad del usuario para reservar nuevas clases durante un tiempo determinado (por ejemplo, durante 48 horas o una semana).
- **Desbloqueo administrativo:** En casos extremos o de reincidencia continuada, la función de reserva quedará totalmente restringida hasta que el usuario se comunique de forma directa con un profesor o administrador del centro para justificar las ausencias y restablecer sus permisos.

13.3. Añadir Rol "Entrenador"

Actualmente, las acciones internas están limitadas al rol de Administrador. Hemos pensado que añadir un nuevo perfil: **Profesor / Entrenador** puede ser necesario ya que si un gimnasio no quiere que sus empleados controlen y tengan acceso a toda la web pues tengan ese rol para poner clases y alguna cosa más que nos puedan pedir.

- **Principio de mínimo privilegio:** Este rol tendrá los permisos necesarios para gestionar su área de trabajo (crear nuevas clases, añadirlas al calendario del gimnasio, modificar horarios y pasar lista de asistencia), sin comprometer la seguridad del sistema.
- **Restricción de datos sensibles:** A diferencia del Administrador, el Entrenador no tendrá acceso a la configuración global del sistema, datos de facturación del gimnasio o gestión integral de la base de datos de todos los clientes.

13.4. Sistema Inteligente de Notificaciones y Alertas

Para mejorar la comunicación entre el gimnasio y los clientes,añadir un sistema de notificaciones (que podría integrar alertas *Push*, correos electrónicos o notificaciones). Este sistema contaría con dos casos de uso principales:

- **Alertas de modificación de servicio:** Si una clase tiene cambio de horario, de profesor, o es cancelada, el sistema enviará una notificación automática y en tiempo real **únicamente** a los usuarios que ya estaban inscritos en dicha sesión.
- **Suscripción a novedades :** Los usuarios podrán configurar sus preferencias para recibir avisos cuando se publiquen nuevas clases en el calendario de sus disciplinas favoritas, permitiéndoles apuntarse rápidamente antes de que se agoten las plazas.

13.5. Gestión de Listas de Espera

Para aquellas clases que estén llenas. Si un usuario inscrito cancela su asistencia, el sistema notificará automáticamente al primer usuario de la lista de espera para que pueda ocupar esa plaza liberada,para asegurar que las plazas se completen y los usuarios vean una comunicación activa.

13.6. Planes de Suscripción Flexibles y Descuentos por Fidelidad

Para incentivar la retención de clientes a largo plazo y ofrecer mayor comodidad en los pagos siempre que el gimnasio lo quiera eso no lo decidimos nosotros, se sugiere implementar un sistema avanzado de gestión de suscripciones que recompense al cliente.

- **Bonos de larga duración:** Posibilidad de que los usuarios adquieran paquetes de varios meses de una sola vez (por ejemplo, planes trimestrales, semestrales o anuales) aplicando automáticamente un precio global más reducido en comparación con el pago mensual estándar.
- **Recompensas por renovación continua:** El sistema detectará si un usuario renueva su plan mensual de forma ininterrumpida. Al alcanzar un número determinado de meses seguidos, se le podrá aplicar de forma automática un porcentaje de descuento en sus siguientes cuotas como premio a su fidelidad.
- **Renovación automática y acumulable:** Se añadirá la opción de que los clientes puedan activar la auto-renovación de su cuota. De este modo, al finalizar su mes actual, el sistema automáticamente cobra al cliente sin necesidad de realizar el proceso de compra manualmente, evitando que se quede sin suscripción sin darse cuenta.
- **Configuración a medida :** Todas estas opciones estarán bajo el control absoluto de la administración del gimnasio. Desde su panel, el administrador podrá decidir si desea habilitar o deshabilitar estas funciones, así como configurar los porcentajes de descuento, los periodos exactos y las condiciones de los bonos, adaptando la plataforma a la estrategia comercial específica del centro.

15. Bibliografía

Alpine.js. (s. f.). **Documentación oficial de Alpine.js.** Consultado para documentar el uso de comportamiento interactivo ligero directamente en el marcado HTML

Carbon. (s. f.). **Documentación oficial de Carbon PHP DateTime API Extension.** Consultado para justificar la gestión de fechas y horas en reservas, accesos, vencimientos y cierres de caja.

Castiñeiras Devesa, R. (2025). **Trabajo de Fin de Grado: Cristalería 9-ART**. IES La Mar, 2º DAW. Memoria consultada como referencia formal para la organización de la documentación, estructura de apartados, estilo de presentación, uso de diagramas, índice de imágenes y redacción técnica de la memoria.

Diagrams.net. (s. f.). **Herramienta de creación de diagramas.** Utilizado para la elaboración del mapa de navegación, diagramas funcionales o esquemas de apoyo.

Figma. (s. f.). **Herramienta de diseño de interfaces.** Utilizado para la creación de mockups de alta fidelidad y definición visual del proyecto.

FullCalendar. (s. f.). **Documentación oficial de FullCalendar.** Consultado para la implementación y explicación del calendario interactivo de clases.

Google Fonts. (s. f.). **Catálogo de tipografías web.** Utilizado para la selección e importación de las tipografías Anton e Inter en la interfaz de Iron Pulse.

Html5-QRCode. (s. f.). **Documentación oficial / repositorio de Html5-QRCode.** Consultado para la integración del escáner QR mediante cámara en el panel de administración.

Laravel. (s. f.). **Documentación oficial de Laravel: Eloquent ORM.** Consultado para la implementación y explicación de modelos, relaciones y operaciones con base de datos mediante Eloquent.

Laravel. (s. f.). **Documentación oficial de Laravel: CSRF Protection.** Consultado para justificar la protección CSRF aplicada a formularios y peticiones del proyecto.

Laravel. (s. f.). **Documentación oficial de Laravel: Starter Kits / Breeze**. Consultado para documentar el sistema de autenticación basado en Laravel Breeze, incluyendo registro, login y vistas iniciales de autenticación.

OpenAI. (2026). **ChatGPT** [Modelo de lenguaje de inteligencia artificial]. Herramienta utilizada como apoyo durante el desarrollo de la memoria para resolver dudas puntuales, organizar contenidos, revisar redacción y mejorar la explicación técnica del proyecto.

Stripe. (s. f.). **Documentación oficial de Stripe: pagos y Checkout Sessions.** Consultado para la integración del flujo de pago simulado y la redirección a Checkout.

Stripe. (s. f.). **Documentación oficial de Stripe: Webhooks.** Consultado para explicar la recepción de eventos de pago y la validación de eventos externos.

Stripe. (s. f.). **Documentación oficial de Stripe: Testing.** Consultado para justificar el uso de modo de pruebas o simulación de pagos dentro del proyecto académico.

SweetAlert2. (s. f.). **Documentación oficial de SweetAlert2.** Consultado para justificar el uso de ventanas emergentes personalizadas, responsivas y accesibles en lugar de alertas nativas del navegador.

Tailwind CSS. (s. f.). **Documentación oficial de Tailwind CSS.** Consultado para justificar el uso de clases de utilidad en el desarrollo de interfaces responsive y componentes visuales.